OPERATIONAL CORRESPONDENCE BETWEEN LAMBDA ENCODINGS IN INTERNAL PI AND ALTERNATING OPERATIONAL GAMES

GUILHÈME JABER, DAVIDE SANGIORGI

Editorial scope. One active lambda/Internal-pi/alternating OGS operational-correspondence group including call-by-value and call-by-name variants, full strong/weak/optimized-encoding arguments and three active appendices. Concurrent/WB/complete-trace results remain original same-source context, not additional targets. Inactive appendices and cutLMCSsecondRound lambda-store/full-abstraction blocks stay inactive and are not counted. Original abstract still mentions lambda-store full abstraction; preserve that source text while making no such corpus target claim.

Allowed standard prerequisites (editorial).

- Author-cited standard foundations. All local definitions, main supporting constructions and complete source bibliography are retained; cited standard foundations remain original references, with no new proof or assumption.

Original abstract. We establish a tight connection between two models of the λ-calculus, namely Milner's encoding into the π-calculus (precisely, the Internal π-calculus), and operational game semantics (OGS). We first investigate the operational correspondence between the behaviours of the encoding provided by π and OGS. We do so for various LTSs: the standard LTS for π and a new 'concurrent' LTS for OGS; an 'output-prioritised' LTS for π and the standard alternating LTS for OGS. We then show that the equivalences induced on λ-terms by all these LTSs (for π and OGS) coincide. We also prove that when equivalence is based on complete traces, the 'concurrent' and 'alternating' variants of OGS also coincide with the 'well-bracketed' variant.

These connections allow us to transfer results and techniques between π and OGS. In particular: we import up-to techniques from π onto OGS; we derive congruence and compositionality results for OGS from those of π; we transport the notion of complete traces from OGS onto π, obtaining a new behavioural equivalence that yields a full abstraction result for the encoding of λ-terms with respect to contexts written in a λ-calculus extended with store. The study is illustrated for both call-by-value and call-by-name.

1. Introduction

The topic of the paper is the comparison between *Operational Game semantics* (OGS) and the *π-calculus*, as generic models or frameworks for the semantics of higher-order languages.

Game semantics [HO00, AJM00] provides intensional models of higher-order languages, where the denotation of a program brings up its possible interactions with the surrounding context. Distinct points of game semantics are the rich categorical structure and the emphasis on compositionality. Game semantics provides a modular characterization of higher-order languages with computational effects like control operators [Lai97], mutable store [AM97, AHM98] or concurrency [GM04, Lai06]. This gives rise to the "Semantic Cube" [Abr97], a characterization of the absence of such computational effects in

Date: arxiv:2504.18227v4; CC BY 4.0.

terms of appropriate restrictions on the interactions, with conditions like *alternation, well-bracketing, visibility* or *innocence*. For instance, well-bracketing corresponds to the absence of control operators like call/cc.

Game semantics has spurred Operational Game Semantics (OGS) [JPR09, JR05, Lai07, GT12, LS14], as a way to describe the interactions of a program with its environment by embedding programs into appropriate configurations and then defining rules that turn such configurations into an LTS. Besides minor differences in the representation of causality between actions, the main distinction with "standard" game semantics is in the way in which the denotation of programs is obtained: via an LTS, rather than, compositionally, by induction on the structure of the programs (or their types). It is nonetheless possible to establish a formal correspondence between these two representations [LS14].

OGS is particularly effective on higher-order programs. To avoid being too intensional, functional values exchanged between the program and its environment are represented as atoms, seen as free variables. Therefore OGS configurations include open terms. The basic actions in the LTS produced by OGS represent the calls and returns of functions between a program and its environment. The OGS semantics has been shown fully-abstract, that is, to characterize observational equivalence, for a wide class of programming languages, including effectful subsets of ML [Lai07, JT16], fragments of Java [JR05], aspect-oriented programs [JPR09]. The conditions in the above-mentioned Semantic Cube (alternation, well-bracketing, etc.) equally apply to OGS.

In this paper, we consider forms of OGS for the pure untyped call-by-value λ-calculus, which enforce some of such conditions. Specifically, we consider:

- an *Alternating* OGS, where only one term can be run at a time, and the control of the interactions alternates between the term and the environment;
- a *Concurrent* OGS, where multiple terms can be run in parallel;
- a *Well-Bracketed* OGS, where the calling convention between a term and its environment follows a stack discipline.

The π-calculus is the paradigmatical name-passing calculus, that is, a calculus where names (a synonym for 'channels') may be passed around. In the literature about the π-calculus, and more generally in Programming Language theory, Milner's work on functions as processes [Mil92], which shows how the evaluation strategies of *call-by-name λ-calculus* and *call-by-value λ-calculus* [Abr89, Plo75] can be faithfully mimicked, is generally considered a landmark. The work promotes the π-calculus to be a model for higher-order programs, and provides the means to study λ-terms in contexts other than the purely sequential ones and with the instruments available to reason about processes. In the paper, π-calculus is actually meant to be the *Internal π-calculus* (Iπ), a subset of the original π-calculus in which only fresh names may be exchanged among processes [San95]. The use of Iπ avoids a few shortcomings of Milner's encodings, notably for call-by-value; e.g., the failure of the β_v rule (i.e., the encodings of $(\lambda x.\, M)V$ and $M\{^V/_x\}$ may be behaviourally distinguishable in π).

Further investigations into Milner's encodings [San93,San00,DHS18] have revealed what is the equivalence induced on λ-terms by the encodings, whereby two λ-terms are equal if their encodings are behaviourally equivalent (i.e., bisimilar) Iπ terms. In call-by-value, this equivalence is *eager normal-form bisimilarity* [Las05], a tree structure proposed by Lassen (and indeed sometimes referred to as 'Lassen's trees') as the call-by-value counterpart of Böhm Trees (or Lévy-Longo Trees).

In a nutshell, when used to give semantics to a language, major strengths of the π-calculus are its algebraic structure and the related algebraic properties and proof techniques; major strengths of OGS are its proximity to the source language — the configurations of OGS are built directly from the terms of the source language, as opposed to an encoding as in the π-calculus — and its flexibility — the semantics can be tuned to account for specific features of the source language like control operators or references.

The general goal of this paper is to show that there is a tight and precise correspondence between OGS and π-calculus as models of programming languages, and that such a correspondence may be profitably used to take advantage of the strengths of the two models. We carry out the above program

in the specific case of (untyped) call-by-value λ-calculus, Λ_{V}, which is richer and (as partly suggested above) with some more subtle aspects than call-by-name. We also sketch at the end of the paper a similar correspondence for the untyped call-by-name λ-calculus. Analogies and similarities between game semantics and π-calculus have been pointed out in various papers in the literature (e.g., [HO95, BHY01]; see Section 14), and used to, e.g., explain game semantics using π-like processes, and enhance type systems for π-terms. In this paper, in contrast, we carry out a direct comparison between the two models, on their interpretation of functions.

We take the (arguably) canonical representations of Λ_{V} into Iπ and OGS. The latter representation is Milner's encoding, rewritten in Iπ. We consider two variant behaviours for the Iπ terms, respectively produced by the ordinary LTS of Iπ, and by an 'output-prioritised' LTS, opLTS, in which input actions may be observed only in the absence of outputs and internal actions. Intuitively, the opLTS is intended to respect sequentiality constraints in the Iπ terms: an output action stands for an ongoing computation (for instance, returning the result of a previous request) whereas an input action starts a new computation (for instance, a request of a certain service); therefore, in a sequential system, an output action should have priority over input actions. For OGS, the Λ_{V} representation is the straightforward adaptation of the OGS representations of typed λ-calculi in the literature, e.g., [Lai07] designed to build fully-abstract models for languages with mutable store. As outlined above, a few variants exist: we consider the alternating, concurrent, and well-bracketed variants.

We also consider the refinements obtained by focusing on *complete traces*, intuitively, traces that describe "terminating" interactions. Complete traces have been introduced in [AM97] to obtain an effective presentation of a fully abstract model for a language with higher-order store.

We then develop a thorough comparison between the behaviours of the OGS and Iπ representations. For this, we define a mapping from OGS configurations to Iπ processes. We also exploit the fact that, syntactically, the actions in the OGS and Iπ LTSs are the same. We derive a tight correspondence between the two models, which allows us to transfer techniques and to switch freely between the two models in the analysis of the OGS and Iπ representations of Λ_{V}, so to establish new results or obtain new proofs. On these aspects, our main results are the following:

(1) We show that the representation of Λ_{V} in the Alternating OGS is behaviourally the same as the representation in Iπ assuming the opLTS. Thus the semantics on λ-terms induced by the OGS and Iπ representations coincide. The same results are obtained between the Concurrent OGS and Iπ under its ordinary LTS.
(2) We transfer 'bisimulation up-to techniques' for Iπ, notably a form of 'up-to context', onto (Concurrent) OGS. The result is a powerful technique, called 'up-to composition', that allows us to split an OGS configuration into more elementary configurations during the bisimulation game.
(3) We show that the semantics induced on Λ_{V} by the Alternating and by the Concurrent OGS are the same, both when the equality in OGS is based on traces and when it is based on bisimulation. In other words, all the OGS views of Λ_{V} (Alternating or Concurrent, traced-based or bisimulation-based) coincide. Moreover, we show that such induced semantics is the equality of Lassen's trees. We derive the result in two ways: one in which we directly import it from Iπ; the other in which we lift eager normal-form bisimulations into OGS bisimulations via the up-to-composition technique.
(4) We derive congruence and compositionality properties for the OGS semantics, as well as a notion of tensor product over configurations that computes interleavings of traces.
(5) We show that when equality in OGS is based on complete traces, the semantics induced on Λ_{V} is the same for the Alternating, Concurrent, and Well-Bracketed OGS.

The results about OGS in (1-4) are obtained by exploiting the mapping into Iπ and its algebraic properties and proof techniques, as well as the up-to-composition technique for OGS imported from Iπ. Analogous results are established for the call-by-name strategy. To summarize our results, we consider in this paper the following LTSs and equivalences:

LTS	Iπ	output-prioritised Iπ	C-OGS	A-OGS	WB-OGS
bisimilarity	$\approx_{\pi}$	$\approx_{\mathtt{o}\pi}$	$\approx_{\mathtt{c}}$	$\approx_{\mathtt{a}}$	$\approx_{\mathtt{wb}}$
trace equivalence	$\eqsim_{\pi}$	$\eqsim_{\mathtt{o}\pi}$	$\eqsim_{\mathtt{c}}$	$\eqsim_{\mathtt{a}}$	$\eqsim_{\mathtt{wb}}$
complete-trace equivalence			$\eqsim^{\mathrm{c}}_{\mathrm{c}}$	$\eqsim^{\mathrm{c}}_{\mathrm{a}}$	$\eqsim^{\mathrm{c}}_{\mathrm{wb}}$

Terms of the call-by-value λ-calculus M, N can then be embedded into the LTS Iπ and *opLTS* with the encoding $\mathcal{W}[\![M]\!]$ and into the C-OGS, A-OGS and WB-OGS as initial configurations $\langle M\rangle$. Using these embeddings, the various equivalence of the previous figure can be lifted into equivalence over terms. We then prove that:

(1) $\approx_{\pi}, \approx_{\mathtt{o}\pi}, \approx_{\mathtt{c}}, \approx_{\mathtt{a}}, \approx_{\mathtt{wb}}, \eqsim_{\pi}, \eqsim_{\mathtt{o}\pi}, \eqsim_{\mathtt{c}}, \eqsim_{\mathtt{a}}$ all coincides;
(2) $\eqsim^{\mathrm{c}}_{\mathrm{c}}, \eqsim^{\mathrm{c}}_{\mathrm{a}}$ and $\eqsim^{\mathrm{c}}_{\mathrm{wb}}$ all coincides.

We also prove that the first group of equivalences also coincides with eager normal-form bisimilarity.

Structure of the paper. Sections 2 to 5 contain background material: general notations, Iπ, Λ_{V}, the representations of Λ_{V} in the Alternating OGS (A-OGS) and in Iπ. The following sections contain the new material. In Section 6 we study the relationship between the two Λ_{V} representations, in Iπ using the output-prioritised LTS. In Section 7 we establish a similar relationship between a new Concurrent OGS (C-OGS) and Iπ using its ordinary LTS. We also transport up-to techniques onto OGS, and prove that all the semantics of Λ_{V} examined (OGS, Iπ, traces, bisimulations) coincide. We import compositionality results for OGS from Iπ in Section 9. In Section 10 we introduce the Well-Bracketed OGS (WB-OGS) and complete traces. In Section 11, we introduce a notion of product of OGS configurations, that is used to provide a wide range of decomposition results. In Section 12 we relate all complete-trace equivalences in OGS (A-OGS, C-OGS, WB-OGS). In Section 13 we discuss the case of call-by-name. Finally, in Section 14 some further related work and future research directions. Appendixes A-D report additional material for proofs and results in Sections 6, 7, 10, and 13.

An extended abstract of this paper had appeared at CSL'22 [JS22], without proofs and without the contents in Sections 10-13.

2. Notations

In the paper, we use various LTSs and behavioural relations for them, both for OGS and for the π-calculus. In this section, we introduce or summarise common notations.

We use a tilde, like in $\widetilde{a}$, for (possibly empty) tuples of objects (usually names). Let $K \xrightarrow{\mu}_{\mathtt{g}} K'$ be a generic LTS (for OGS or Iπ; the grammar for actions in the LTSs for OGS and Iπ will be the same). Actions, ranged over by μ, can be of the form $a(\widetilde{b})$, $\overline{a}(\widetilde{b})$, τ, and $(\widetilde{a})$, where τ, called *silent* or (*invisible*) action, represents an internal step in K, that is, an action that does not require interaction with the outside, and $(\widetilde{a})$ is a special action performed by abstractions in Iπ and initial configurations in OGS. If $\mu \neq \tau$ then μ is a *visible* action; we use ℓ to range over them.

We sometimes abbreviate $\xrightarrow{\tau}_{\mathtt{g}}$ as $\longrightarrow_{\mathtt{g}}$. We write $\Longrightarrow_{\mathtt{g}}$ for the reflexive and transitive closure of $\xrightarrow{\tau}_{\mathtt{g}}$; i.e., $K \Longrightarrow_{\mathtt{g}} K'$ holds if K can evolve into K' by performing some silent steps – possibly none. We also write $K \overset{\mu}{\Longrightarrow}_{\mathtt{g}} K'$ if $K \Longrightarrow_{\mathtt{g}} \xrightarrow{\mu}_{\mathtt{g}} \Longrightarrow_{\mathtt{g}} K'$ (the composition of the three relations); i.e., $K \overset{\mu}{\Longrightarrow}_{\mathtt{g}} K'$ holds if there are K_1 and K_2 with $K \Longrightarrow_{\mathtt{g}} K_1$, $K_1 \xrightarrow{\mu}_{\mathtt{g}} K_2$ and $K_2 \Longrightarrow_{\mathtt{g}} K'$. Then $\overset{\widehat{\mu}}{\Longrightarrow}_{\mathrm{g}}$ is $\overset{\mu}{\Longrightarrow}_{\mathtt{g}}$ if $\mu \neq \tau$, and $\Longrightarrow_{\mathtt{g}}$ if $\mu = \tau$.

Traces, ranged over by s, are finite (and possibly empty) sequences of visible actions. If $s = \ell_1, \ldots, \ell_n$ ($n \geq 0$), then $K \overset{s}{\Longrightarrow}_{\mathtt{g}} K'$ holds if there are $K_0, \ldots, K_n$ with $K_0 = K$, $K_n = K'$, and $K_i \overset{\ell_{i+1}}{\Longrightarrow}_{\mathtt{g}} K_{i+1}$ for $0 \leq i < n$; and $K \overset{s}{\Longrightarrow}_{\mathtt{g}}$ if there is K' with $K \overset{s}{\Longrightarrow}_{\mathtt{g}} K'$.

In an action $a(\widetilde{b})$ or $\overline{a}(\widetilde{b})$ we call a the *subject* and $\widetilde{b}$ the object; similarly in a trace s a name a occurs in *subject position* (resp. in *object position*) if there is an action in s whose subject is a (resp. whose object includes a).

Two states K_1, K_2 of the LTS are *trace equivalent*, written $K_1 \simeq_{\mathtt{g}} K_2$, if ($K_1 \stackrel{s}{\Longrightarrow}_{\mathtt{g}}$ iff $K_2 \stackrel{s}{\Longrightarrow}_{\mathtt{g}}$), for all s.

Similarly, *bisimilarity*, written $\approx_{\mathtt{g}}$, is the largest symmetric relation on the state of the LTS such that whenever $K_1 \approx_{\mathtt{g}} K_2$ then

- $K_1 \stackrel{\mu}{\Longrightarrow}_{\mathtt{g}} K_1'$ implies there is K_2' with $K_2 \stackrel{\widehat{\mu}}{\Longrightarrow}_{\mathrm{g}} K_2'$ and $K_1' \approx_{\mathtt{g}} K_2'$.

For instance, in the Iπ LTS $\stackrel{\mu}{\longrightarrow}_{\pi}$ of Section 3.1, $P \simeq_{\pi} Q$ means that the Iπ processes P and Q are trace equivalent, and $P \approx_{\pi} Q$ means that they are bisimilar.

Remark 2.1 (bound names)**.** In an action $a(\widetilde{b})$ or $\overline{a}(\widetilde{b})$ or $(\widetilde{b})$, name a is *free* whereas $\widetilde{b}$ are *bound*; the free and bound names of a trace are defined accordingly.

Throughout the paper, in any statement (concerning OGS or Iπ), the bound names of an action or of a trace that appear in the statement are supposed to be all *fresh*; i.e., all distinct from each other and from the free names of the objects in the statement. For instance, in a trace $s = s_1, \mu, s_2$, a bound name of μ should not appear in the preceding prefix s_1 (neither bound nor free), should not have other occurrences in μ, and may only appear free in the following suffix s_2; moreover, if s is the trace in the definition above of trace equivalence, any bound names of s should not appear free in the initial states K_1, K_2. Similarly, in the following definition of bisimilarity, the bound names of the action μ do not appear free in the states K_1, K_2. In the same way, the convention applies to statements about operational correspondence between an OGS state K_1 and a Iπ state K_2 (the bound names of actions or traces mentioned in the statement do not appear free in K_1, K_2). □

3. Background

3.1. **The Internal π-calculus.** The Internal π-calculus, Iπ, is, intuitively, a subset of the π-calculus in which all outputs are bound. This is syntactically enforced by having outputs written as $\overline{a}(\widetilde{b})$ (which in the π-calculus would be an abbreviation for $\boldsymbol{\nu}\widetilde{b}\,\overline{a}\langle\widetilde{b}\rangle$). We use the polyadic version of the calculus, in which a tuple of names may be exchanged in a communication step. All tuples of names in Iπ are made of pairwise distinct components.

Abstractions are used to write name-parametrised processes, for instance, when writing recursive process definitions. The instantiation of the parameters of an abstraction B is done via the *application* construct $B\langle\widetilde{a}\rangle$. Processes and abstractions form the set of *agents*, ranged over by T. Lowercase letters $a, b, \ldots, x, y, \ldots$ range over the infinite set of names. The grammar of Iπ is thus:

$$\begin{array}{lcll} P & \triangleq & 0 \;\big|\; a(\widetilde{b}).\,P \;\big|\; \overline{a}(\widetilde{b}).\,P \;\big|\; \boldsymbol{\nu} a\,P \;\big|\; P_1 \mid P_2 \;\big|\; !a(\widetilde{b}).\,P \;\big|\; B\langle\widetilde{a}\rangle & \text{(processes)} \\ B & \triangleq & (\widetilde{a})\;P \;\big|\; \mathtt{K} & \text{(abstractions)} \end{array}$$

We omit the standard definition of free names, bound names, and names of an agent, respectively indicated with $\mathtt{fn}(-)$, $\mathtt{bn}(-)$, and $\mathtt{n}(-)$. The grammar for processes includes the operators 0 (representing the inactive process), input and output prefixes, restriction, parallel composition, replication, and application.

In the grammar of abstractions, $\mathtt{K}$ is a constant, used to write recursive definitions. Each constant $\mathtt{K}$ has a defining equation of the form $\mathtt{K} \triangleq (\tilde{x})\;P$, where $(\tilde{x})\;P$ is name-closed (that is, without free names); $\tilde{x}$ are the formal parameters of the constant (replaced by the actual parameters whenever the constant is used). Replication could be avoided in the syntax since it can be encoded with recursion. However its semantics is simple, and it is a useful construct for encodings, thus we chose to include it in the grammar.

An application redex, that is, a term syntactically of the form $((\widetilde{x})P)\langle\widetilde{a}\rangle$, can be normalised as $P\{\widetilde{a}/\widetilde{x}\}$. An agent is *normalised* if all such application redexes have been contracted. Although the full application $\mathtt{K}\langle\widetilde{a}\rangle$ is often convenient, when it comes to reasoning on behaviours it is useful to assume that all expressions are normalised, in the above sense. Thus in the remainder of the paper *we identify an agent with its normalised expression.*

Since the calculus is polyadic, we assume a *sorting system* [Mil91] to avoid disagreements in the arities of the tuples of names carried by a given name and in applications of abstractions. Being not essential, it will not be presented here. The reader should take for granted that all agents described obey a sorting.

3.1.1. *Operational semantics and behavioural relations.* In Figure 1 we report the standard LTS of Iπ. The symbol $\equiv_\alpha$ is used to indicate α-equivalence between Iπ agents. Rule `abs` is used to formalise, by means of an action, the ground instantiation of the parameters of an abstraction. In both tables, we omit the symmetric of `parL`, called `parR`. In rules `com`, $\overline{\mu}$ is the complementary of μ, thus if $\mu = a(\tilde{x})$ then $\overline{\mu} = \overline{a}(\tilde{x})$, and conversely if $\mu = \overline{a}(\tilde{x})$ then $\overline{\mu} = a(\tilde{x})$.

$$\texttt{alpha:}\ \frac{P \equiv_\alpha P' \quad P' \xrightarrow{\mu}_\pi P''}{P \xrightarrow{\mu}_\pi P''} \qquad \texttt{pre:}\ \frac{}{\mu.\, P \xrightarrow{\mu}_\pi P} \qquad \texttt{parL:}\ \frac{P \xrightarrow{\mu}_\pi P'}{P \mid Q \xrightarrow{\mu}_\pi P' \mid Q}$$

$$\texttt{res:}\ \frac{P \xrightarrow{\mu}_\pi P' \quad x \notin \texttt{n}(\mu)}{\boldsymbol{\nu} x\, P \xrightarrow{\mu}_\pi \boldsymbol{\nu} x\, P'} \qquad \texttt{com:}\ \frac{P \xrightarrow{\mu}_\pi P' \quad Q \xrightarrow{\overline{\mu}}_\pi Q' \quad \mu \neq \tau,\ \tilde{x} = \texttt{bn}(\mu)}{P \mid Q \xrightarrow{\tau}_\pi \boldsymbol{\nu}\tilde{x}\, (P' \mid Q')}$$

$$\texttt{rep:}\ \frac{\mu.\, P \xrightarrow{\mu}_\pi P'}{!\mu.\, P \xrightarrow{\mu}_\pi P' \mid\, !\mu.\, P} \qquad \texttt{con:}\ \frac{P \xrightarrow{\mu}_\pi P' \quad \texttt{K} \stackrel{\text{def}}{=} (\widetilde{y}).\, Q \quad (\widetilde{y}).\, Q \equiv_\alpha (\widetilde{x}).\, P}{\texttt{K}\langle \widetilde{x} \rangle \xrightarrow{\mu}_\pi P'}$$

$$\texttt{abs:}\ \frac{(\ B = (\widetilde{y}).\, Q \text{ or } (B = K \text{ and } K \stackrel{\text{def}}{=} (\widetilde{y}).\, Q)\ ) \quad (\widetilde{y}).\, Q \equiv_\alpha (\widetilde{x}).\, P}{B \xrightarrow{(\widetilde{x})} P}$$

FIGURE 1. The standard LTS for Iπ

In the LTS for Iπ, transitions are of the form $T \xrightarrow{\mu}_\pi T'$, where the bound names of μ are fresh, i.e., they do not appear free in T. The choice of bound names can be made more explicit by following the operational game semantics approach in which transitions are defined on configurations that include a name set component. (Some presentations of π-calculi indeed make use of configurations.) The two approaches are interchangeable, and the choice is largely a matter of taste. The standard LTS is however more convenient in reasoning, e.g., the up-to proof techniques of Section 3.3.

All behavioural relations are extended to abstractions by requiring ground instantiation of the parameters; this is expressed by means of a transition; e.g., the action $(\tilde{x})\ P \xrightarrow{(\tilde{x})}_\pi P$ (see rule `abs` in Figure 1).

3.2. **The expansion relation.** The expansion relation, $\lesssim_\pi$ [SW01], is an asymmetric variant of $\approx_\pi$ which allows us to count the number of τ-actions performed by the processes. Thus, $P \lesssim_\pi Q$ holds if $P \approx_\pi Q$ but also Q has at least as many τ-moves as P. We write $P \xrightarrow{\widehat{\mu}} P'$ when either $P \xrightarrow{\mu} P'$ or $(\mu = \tau$ and $P = P')$.

Definition 3.1 (expansion)**.** A relation $\mathcal{R} \subseteq \mathcal{P} \times \mathcal{P}$ is an *expansion* if $P\ \mathcal{R}\ Q$ implies:

(1) Whenever $P \xrightarrow{\mu} P'$, there exists Q' s.t. $Q \xRightarrow{\mu}_\pi Q'$ and $P'\ \mathcal{R}\ Q'$;
(2) whenever $Q \xrightarrow{\mu} Q'$, there exists P' s.t. $P \xrightarrow{\widehat{\mu}} P'$ and $P'\ \mathcal{R}\ Q'$.

We say that Q *expands* P, written $P \lesssim_\pi Q$, if $P\mathcal{R}Q$, for some expansion $\mathcal{R}$.

Relation $\lesssim_\pi$ is a precongruence, and is strictly included in $\approx_\pi$.

3.3. **The "up-to" techniques.** The "up-to" techniques allow us to reduce the size of a relation $\mathcal{R}$ to exhibit for proving bisimilarities.

3.3.1. *Bisimulation up-to context and expansion.* Our main up-to technique will be *up-to context and expansion* [San96a], which admits the use of contexts and of behavioural equivalences such as expansion to achieve the closure of a relation in the bisimulation game.

For our purposes, a restricted form of 'up-to context' will be enough, in which only contexts of the form $\boldsymbol{\nu}\tilde{c}\,(R \mid [\cdot])$ are employed.

Definition 3.2 (bisimulation up-to context and up-to ${}_\pi\gtrsim$)**.** A symmetric relation $\mathcal{R} \subseteq \mathcal{P} \times \mathcal{P}$ is a *bisimulation up-to context and up-to* ${}_\pi\gtrsim$ if $P\,\mathcal{R}\,Q$ and $P \xrightarrow{\mu} P''$ then there are processes P' and Q' and a static context $C_{\mathtt{ctx}}$ s.t. $P''\ {}_\pi\gtrsim\ C_{\mathtt{ctx}}[P']$, $Q \stackrel{\widehat{\mu}}{\Longrightarrow} {}_\pi\gtrsim\ C_{\mathtt{ctx}}[Q']$ and $P'\,\mathcal{R}\,Q'$.

The soundness of the above technique relies on the following lemma.

Lemma 3.3. *Suppose $\mathcal{R}$ is a bisimulation up-to context and up-to ${}_\pi\gtrsim$, $(P,Q) \in \mathcal{R}$ holds and $C_{\mathtt{ctx}}$ is a static context. If $C_{\mathtt{ctx}}[P] \xrightarrow{\mu_1} \cdots \xrightarrow{\mu_n} P_1$, $n \geq 0$, then there are a static context C' and processes P' and Q' s.t. $P_1\ {}_\pi\gtrsim\ C'[P']$, $C_{\mathtt{ctx}}[Q] \stackrel{\widehat{\mu_1}}{\Longrightarrow} \cdots \stackrel{\widehat{\mu_n}}{\Longrightarrow} {}_\pi\gtrsim\ C'[Q']$ and $P'\,\mathcal{R}\,Q'$.*

Theorem 3.4. *If $\mathcal{R}$ is a bisimulation up-to context and up-to ${}_\pi\gtrsim$, then $\mathcal{R} \subseteq \approx_\pi$.*

Proof. By showing that the relation $\mathcal{S}$ containing all pairs (P,Q) such that

$$\begin{array}{l}\text{for some static context } C_{\mathtt{ctx}} \text{ and processes } P', Q' \\ \text{it holds that } P\ {}_\pi\gtrsim\ C_{\mathtt{ctx}}[P'], Q\ {}_\pi\gtrsim\ C_{\mathtt{ctx}}[Q'] \text{ and } P'\,\mathcal{R}\,Q'\end{array}$$

is a bisimulation. See [San96a] for details (adapting the proof to Iπ is straightforward). □

3.3.2. *Bisimulation up-to bisimilarity.* We will also employ *bisimulation up-to* $\approx_\pi$ [Mil89], a widely-used technique, whereby bisimilarity itself is employed to achieve the closure of the candidate relation during the bisimulation game.

Definition 3.5 (bisimulation up-to $\approx_\pi$)**.** A symmetric relation $\mathcal{R}$ on $\mathcal{P} \times \mathcal{P}$ is a *bisimulation up-to* $\approx_\pi$ if $P\,\mathcal{R}\,Q$ and $P \stackrel{\mu}{\Longrightarrow}_\pi P'$ imply that there exists Q' s.t. $Q \stackrel{\widehat{\mu}}{\Longrightarrow} Q'$ and $P' \approx_\pi \mathcal{R} \approx_\pi Q'$.

Theorem 3.6. *Suppose $\mathcal{R}$ is a* bisimulation up-to $\approx_\pi$*. Then $\mathcal{R} \subseteq \approx_\pi$.*

3.3.3. *Bisimulation up-to context and up-to* $({}_\pi\gtrsim, \approx_\pi)$. We will also employ a variant of bisimulation up-to context and expansion, called *bisimulation up-to context and up-to* $({}_\pi\gtrsim, \approx_\pi)$. In Definition 3.2, the occurrence of ${}_\pi\gtrsim$ in $Q \stackrel{\widehat{\mu}}{\Longrightarrow} {}_\pi\gtrsim\ C_{\mathtt{ctx}}[Q']$ could be weakened to $\approx_\pi$, and the soundness of the technique, namely Theorem 3.4, would still hold. However, in general, the same weakening cannot be made on the requirement $P''\ {}_\pi\gtrsim\ C_{\mathtt{ctx}}[P']$. We show that however, the weakening is possible when the action μ is visible and the context cannot interact with the processes in its holes.

This proof technique, (which, as far as we know, does not appear in the literature) will be essential in some of our key results about the correspondence with game semantics.

Definition 3.7. A static context $C_{\mathtt{ctx}}$ *cannot interact* with a process P if no name appears free both in the context and in the process and with opposite polarities (that is, either in input position in the context and output position in the process, or the converse).

If $C_{\mathtt{ctx}}$ cannot interact with P, whenever $C_{\mathtt{ctx}}[P] \longrightarrow Q$ then the interaction is either internal to $C_{\mathtt{ctx}}$ or internal to P. Moreover, the property is invariant for reduction; i.e., if $Q = C'[P']$ where C' is the derivative of $C_{\mathtt{ctx}}$ and P' the derivative of P, then also C' cannot interact with P'. Notice the invariance is false in the full π-calculus.

We recall that we use ℓ to range over visible actions.

Definition 3.8 (bisimulation up-to context and up-to $({}_{\pi}\!\gtrsim, \approx_{\pi})$)**.** A symmetric relation $\mathcal{R} \subseteq \mathcal{P} \times \mathcal{P}$ is a *bisimulation up-to context and up-to* $({}_{\pi}\!\gtrsim, \approx_{\pi})$ if $P \,\mathcal{R}\, Q$ imply:

(1) if $P \xrightarrow{\ell} P''$ then there are processes P' and Q' and a static context $C_{\mathtt{ctx}}$ s.t. $P'' \approx_{\pi} C_{\mathtt{ctx}}[P']$, $Q \stackrel{\ell}{\Longrightarrow} \approx_{\pi} C_{\mathtt{ctx}}[Q']$ and $P' \,\mathcal{R}\, Q'$, and, moreover, $C_{\mathtt{ctx}}$ cannot interact with P or Q;
(2) if $P \longrightarrow P''$ then there are processes P' and Q' and a static context $C_{\mathtt{ctx}}$ s.t. $P'' \;{}_{\pi}\!\gtrsim C_{\mathtt{ctx}}[P']$, $Q \Longrightarrow {}_{\pi}\!\gtrsim C_{\mathtt{ctx}}[Q']$ and $P' \,\mathcal{R}\, Q'$.

Lemma 3.9. *Suppose $\mathcal{R}$ is a bisimulation up-to context and up-to $({}_{\pi}\!\gtrsim, \approx_{\pi})$ and $C_{\mathtt{ctx}}$ is a static context that does not interact with processes P and Q.*

(1) If $C_{\mathtt{ctx}}[P] \Longrightarrow P_1$, then there are processes P' and Q' and a static context C' s.t. $P_1 \;{}_{\pi}\!\gtrsim C'[P']$, $C_{\mathtt{ctx}}[Q] \Longrightarrow {}_{\pi}\!\gtrsim C'[Q']$ and $P' \mathcal{R} Q'$.
(2) If $C_{\mathtt{ctx}}[P] \xrightarrow{\ell} P_1$, then there are processes P' and Q' and a static context C' and s.t. $P_1 \approx_{\pi} C'[P']$, $C_{\mathtt{ctx}}[Q] \stackrel{\ell}{\Longrightarrow} {}_{\pi} \approx_{\pi} C'[Q']$ and $P' \mathcal{R} Q'$.

The proof of Lemma 3.9 is along the lines of similar results in the literature combining up-to expansion and up-to bisimilarity, see e.g. [SW01].

Theorem 3.10. *If $\mathcal{R}$ is a bisimulation up-to context and up-to $({}_{\pi}\!\gtrsim, \approx_{\pi})$, then $\mathcal{R} \subseteq \approx_{\pi}$.*

Proof. By showing that the relation $\mathcal{R}$ containing all pairs (P, Q) such that

$$\begin{array}{l}\text{for some static context } C_{\mathtt{ctx}} \text{ and processes } P', Q' \\ \text{where } C_{\mathtt{ctx}} \text{ does not interact with } P', Q' \\ \text{it holds that } P \approx_{\pi} C_{\mathtt{ctx}}[P'], Q \approx_{\pi} C_{\mathtt{ctx}}[Q'] \text{ and } P' \,\mathcal{R}\, Q'\end{array}$$

is a bisimulation. In a challenge transition $P \xrightarrow{\mu} P_1$ one distinguishes the case when μ is a silent or visible action. In both cases, one exploits Lemma 3.9. □

3.4. **The Call-By-Value λ-calculus.** The grammar of the untyped call-by-value λ-calculus, Λ_{V}, has values V, terms M, evaluation contexts E, and general contexts C:

$$\begin{array}{llcl} \text{Vals} & V & \triangleq & x \mid \lambda x.\, M \\ \text{Terms} & M, N & \triangleq & V \mid MN \\ \text{ECtxs} & E & \triangleq & [\cdot] \mid VE \mid EM \\ \text{Ctxs} & C & \triangleq & [\cdot] \mid \lambda x.\, C \mid MC \mid CM \end{array}$$

where $[\cdot]$ stands for the hole of a context. The call-by-value reduction $\to_{\mathtt{v}}$ has two rules:

$$\frac{}{(\lambda x.\, M)V \to_{\mathtt{v}} M\{V/x\}} \qquad \frac{M \to_{\mathtt{v}} N}{E[M] \to_{\mathtt{v}} E[N]}$$

In the following, we write $M \Downarrow M'$ to indicate that $M \Rightarrow_{\mathtt{v}} M'$ with M' an eager normal form, that is, either a value or a stuck call $E[xV]$.

We recall Lassen's *eager normal-form (enf) bisimilarity* [Las05]. Enf-bisimulations relate terms, values, and evaluation contexts.

Definition 3.11. An *enf-bisimulation* is a triple of relations on terms $\mathcal{R}_{\mathcal{M}}$, values $\mathcal{R}_{\mathcal{V}}$, and evaluation contexts $\mathcal{R}_{\mathcal{K}}$ that satisfies:

- $M_1 \mathcal{R}_{\mathcal{M}} M_2$ if either:
  - both M_1, M_2 diverge;
  - $M_1 \Downarrow E_1[xV_1]$ and $M_2 \Downarrow E_2[xV_2]$ for some x, values V_1, V_2, and evaluation contexts E_1, E_2 with $V_1 \mathcal{R}_{\mathcal{V}} V_2$ and $E_1 \mathcal{R}_{\mathcal{K}} E_2$;
  - $M_1 \Downarrow V_1$ and $M_2 \Downarrow V_2$ for some values V_1, V_2 with $V_1 \mathcal{R}_{\mathcal{V}} V_2$.
- $V_1 \mathcal{R}_{\mathcal{V}} V_2$ if $V_1 x \mathcal{R}_{\mathcal{M}} V_2 x$ for some fresh x;
- $E_1 \mathcal{R}_{\mathcal{K}} E_2$ if $E_1[x] \mathcal{R}_{\mathcal{M}} E_2[x]$ for some fresh x.

The largest *enf-bisimulation* is called *enf-bisimilarity.*

4. Operational Game Semantics

We introduce the representation of Λ_{V} in OGS. The LTS produced by the embedding of a term intends to capture the possible interactions between this term and its environment. Values exchanged between the term and the environment are represented by free variables, ranged over by x, y, z. Continuations (i.e., evaluation contexts) are also represented by names, called *continuation names* and ranged over by p, q, r.

Actions μ have been introduced in Section 2. In OGS, we have five kinds of (visible) actions:

- *Player Answers* (PA), $\bar{p}(x)$, and *Opponent Answers* (OA), $p(x)$, that exchange a variable x through a continuation name p;
- *Player Questions* (PQ), $\bar{x}(y, p)$, and *Opponent Questions* (OQ), $x(y, p)$, that exchange a variable y and a continuation name p through a variable x;
- *Initial Opponent Questions* (IOQ), (p), that introduce the initial continuation name p.

Remark 4.1. The denotation of terms is usually represented in game semantics using the notion of *pointer structure* rather than traces. A pointer structure is defined as a sequence of *moves*, together with a pointer from each move (but the initial one) to a previous move that 'justifies' it. Taking a trace s, one can reconstruct this pointer structure in the following way: an action μ is *justified* by an action μ' if the free name of μ is bound by μ' in s (here we are taking advantage of the 'freshness' convention on the bound names of traces, Remark 2.1).

Environments, ranged over by γ, maintain the association from names to values and evaluation contexts, and are partial maps with finite domain. A single mapping is either of the form $[x \mapsto V]$ (the variable x is mapped onto the value V), or $[p \mapsto (E, q)]$ (the continuation name p is mapped onto the pair of the evaluation context E and the continuation q). We write ε for the empty environment, and $\gamma \cdot \gamma'$ for the union of two such partial maps, provided that their domains are disjoint.

There are two main kinds of *configurations* F: *active configurations* $\langle M, p, \gamma, \phi \rangle$ and *passive configurations* $\langle \gamma, \phi \rangle$, where M is a term, p a continuation name, γ an environment and ϕ a set of names called its *name-support*, corresponding to the set of all names that has been exchanged so far.

The LTS is introduced in Figure 2. It is called *Alternating*, since, forgetting the Pτ transition, it is bipartite between active configurations that perform Player actions and τ-transitions, and passive configurations that perform Opponent actions. Accordingly, we call Alternating the resulting OGS, abbreviated A-OGS. In the OA rule, E is "garbage-collected" from γ, a behavior corresponding to *linear continuations.*

The term of an active configuration determines the next transition performed. First, the term needs to be reduced, using the rule (Pτ). When the term is a value V, a Player Answer (PA) is performed, providing a fresh variable x to Opponent, while V is stored in γ at position x. Freshness is enforced using the disjoint union $\uplus$. When the term is a callback $E[xV]$, with p the current continuation name, a Player Question (PQ) at x is performed, providing two fresh names y, q to Opponent, while storing V at y and (E, p) at q in γ.

$(P\tau)$	$\langle M, p, \gamma, \phi\rangle$	$\xrightarrow{\tau}_{\mathtt{a}}$	$\langle N, p, \gamma, \phi\rangle$	when $M \to_{\mathtt{v}} N$
(PA)	$\langle V, p, \gamma, \phi\rangle$	$\xrightarrow{\bar{p}(x)}_{\mathtt{a}}$	$\langle \gamma \cdot [x \mapsto V], \phi \uplus \{x\}\rangle$	
(PQ)	$\langle E[xV], p, \gamma, \phi\rangle$	$\xrightarrow{\bar{x}(y,q)}_{\mathtt{a}}$	$\langle \gamma \cdot [y \mapsto V] \cdot [q \mapsto (E, p)], \phi \uplus \{y, q\}\rangle$	
(OA)	$\langle \gamma \cdot [q \mapsto (E, p)], \phi\rangle$	$\xrightarrow{q(x)}_{\mathtt{a}}$	$\langle E[x], p, \gamma, \phi \uplus \{x\}\rangle$	
(OQ)	$\langle \gamma, \phi\rangle$	$\xrightarrow{x(y,p)}_{\mathtt{a}}$	$\langle Vy, p, \gamma, \phi \uplus \{y, p\}\rangle$	when $\gamma(x) = V$
(IOQ)	$\langle [? \mapsto M], \phi\rangle$	$\xrightarrow{(p)}_{\mathtt{a}}$	$\langle M, p, \varepsilon, \phi \uplus \{p\}\rangle$	

FIGURE 2. The LTS for the Alternating OGS (A-OGS)

On passive configurations, Opponent has the choice to perform different actions. It can perform an Opponent Answer (OA) by interrogating an evaluation context E stored in γ. For this, Opponent provides a fresh variable x that is plugged into the hole of E, while the continuation name q associated with E in γ is restored. Opponent may also perform an Opponent Question (OQ), by interrogating a value V stored in γ. For this, Opponent provides a fresh variable y as an argument to V.

To build the denotation of a term M, we introduce an *initial configuration* associated with it, written $\langle [? \mapsto M], \phi\rangle$, with ϕ the set of free variables we start with. When this set is taken to be the free variables of M, we simply write it as $\langle M\rangle$. In the initial configuration, the choice of the continuation name p is made by performing an Initial Opponent question (IOQ). So initial configurations are passive.

From an environment γ, we extract its *continuation structure* which is defined as the relation over continuation names

$$\mathrm{cs}(\gamma) \triangleq \{(p, q) \mid p \in \mathrm{dom}(\gamma) \text{ and } \gamma(p) = (_, q)\}$$

As we will see, the continuation structure indicates on which continuation name q Player should answer following an Opponent answer over p. Continuation names of ϕ that are either the toplevel continuation name – for active configurations – or that are in the continuation structure of γ are called *available*. So continuation names that are not available are the ones that have already been used to perform an answer. This distinction is needed since we consider linear continuations here. Variables are always considered to be available since there is no constraint on the number of times they can be interrogated.

Names in $\mathrm{dom}(\gamma)$ are called *P-names*, and those in $\phi \backslash \mathrm{dom}(\gamma)$ that are available are called *O-names*. So we obtain a *polarity function* pol_F associated to F, defined as the partial maps from ϕ to $\{O, P\}$ mapping names to their polarity. In the following, we only consider *valid configurations*, for which:

- $\mathrm{dom}(\gamma) \subseteq \phi$
- $\mathtt{fv}(M), p$ are O-names;
- for all $a \in \mathrm{dom}(\gamma)$, the names appearing in $\gamma(a)$ are O-names.

5. The encoding of call-by-value λ-calculus into the π-calculus

We recall here Milner's encoding of call-by-value λ-calculus, transplanted into Iπ.

The core of any encoding of the λ-calculus into a process calculus is the translation of function application. This becomes a particular form of parallel combination of two processes, the function, and its argument; β-reduction is then modelled as a process interaction.

$$
\begin{array}{rcl}
\mathcal{V}[\![V]\!] & \triangleq & (p)\ \overline{p}(y).\,\mathcal{V}^*[\![V]\!]\langle y\rangle \\
\mathcal{V}[\![MN]\!] & \triangleq & (p)\ \boldsymbol{\nu} q\ (\ \mathcal{V}[\![M]\!]\langle q\rangle \mid q(y).\,\boldsymbol{\nu} r\ (\mathcal{V}[\![N]\!]\langle r\rangle \mid \\
 & & \qquad\qquad r(w).\,\overline{y}(w',p').\,(w' \triangleright w \mid p' \triangleright p))) \\
\mathcal{V}^*[\![\lambda x.\,M]\!] & \triangleq & (y)\ !y(x,q).\,\mathcal{V}[\![M]\!]\langle q\rangle \\
\mathcal{V}^*[\![x]\!] & \triangleq & (y)\ y \triangleright x
\end{array}
$$

FIGURE 3. The encoding of call-by-value λ-calculus into Iπ

The encoding of a λ-term is parametric over a name, that is, it is an abstraction. This parameter may be thought of as its *continuation*. A term that becomes a value signals so at its continuation name and, in doing so, it grants access to the body of the value. Such body is replicated, so that the value may be copied several times. When the value is a function, its body can receive two names: (the access to) its value-argument, and the following continuation. In the translation of application, first, the function is run, then the argument; finally, the function is informed of its argument and continuation. (Milner noticed [Mil92] that his call-by-value encoding can be easily tuned so to mimic forms of evaluation in which, in an application MN, the function M is run first, or the argument N is run first, or function and argument are run in parallel. Here we follow the first option, the most common one in call-by-value.)

As in OGS, so in Iπ the encoding uses two kinds of names, the *continuation names* $p, q, r, \ldots$, and the *variable names* $x, y, v, w \ldots$. Figure 3 presents the encoding. The encoding function $\mathcal{V}$, on values, is decomposed by means of the auxiliary function $\mathcal{V}^*$; this will be useful later, e.g., in the encoding of environments. Process $a \triangleright b$ represents a *link* (sometimes called forwarder; for readability we have adopted the infix notation $a \triangleright b$ for the constant $\triangleright$). It transforms all outputs at a into outputs at b (therefore a, b are names of the same sort); thus the body of $a \triangleright b$ is replicated, unless a and b are continuation names:

$$
\triangleright \ \triangleq\ \begin{cases} (p,q).\,p(x).\,\overline{q}(y).\,y \triangleright x \\ \qquad \text{if } p, q \text{ are continuation names} \\ (x,y).\,!x(z,p).\,\overline{y}(w,q).\,(q \triangleright p \mid w \triangleright z) \\ \qquad \text{if } x, y \text{ are variable names} \end{cases}
$$

The equivalence induced on call-by-value λ-terms by their encoding into Iπ coincides with Lassen's *eager normal-form (enf) bisimilarity* [Las05].

Theorem 5.1 [DHS18]**.** *$\mathcal{V}[\![M]\!] \approx_\pi \mathcal{V}[\![N]\!]$ iff M and N are enf-bisimilar.*

In proofs about the behaviour of the Iπ representation of λ-terms we sometimes follow Durier et al. [DHS18] and use an optimisation of Milner's encoding, reported in Appendix B. The optimised encoding, indicated in the remainder as $\mathcal{O}$, is obtained from the initial one $\mathcal{V}$ by performing a few (deterministic) reductions, at the price of a more complex definition, in which a case analysis is made for the encoding of applications. We only report here the result about its behavioural correctness:

Lemma 5.2 (correctness of the optimisation)**.** *For all M, it holds that $\mathcal{V}[\![M]\!] \ {}_\pi\!\gtrsim \mathcal{O}[\![M]\!]$. Similarly, for all* A-OGS *configurations F, it holds that $\mathcal{V}[\![F]\!] \ {}_\pi\!\gtrsim \mathcal{O}[\![F]\!]$.*

The lemma is established by algebraic reasoning [DHS18, Dur20]; its extension to encodings of configurations is straightforward.

Encoding of environments:

$$\begin{array}{rcl} \mathcal{V}[\![[y \mapsto V] \cdot \gamma']\!] & \triangleq & \mathcal{V}^*[\![V]\!]\langle y\rangle \mid \mathcal{V}[\![\gamma']\!] \\ \mathcal{V}[\![[q \mapsto (E,p)] \cdot \gamma']\!] & \triangleq & q(x).\, \mathcal{V}[\![E[x]]\!]\langle p\rangle \mid \mathcal{V}[\![\gamma']\!] \\ \mathcal{V}[\![\varepsilon]\!] & \triangleq & 0 \end{array}$$

Encoding of configurations:

$$\begin{array}{rcl} \mathcal{V}[\![\langle M, p, \gamma, \phi\rangle]\!] & \triangleq & \mathcal{V}[\![M]\!]\langle p\rangle \mid \mathcal{V}[\![\gamma]\!] \\ \mathcal{V}[\![\langle \gamma, \phi\rangle]\!] & \triangleq & \mathcal{V}[\![\gamma]\!] \\ \mathcal{V}[\![\langle [? \mapsto M], \phi\rangle]\!] & \triangleq & \mathcal{V}[\![M]\!] \end{array}$$

FIGURE 4. From OGS environments and configurations to Iπ

6. Relationship between Iπ and A-OGS

To compare the A-OGS and Iπ representations of the (call-by-value) λ-calculus, we set a mapping from A-OGS configurations and environments to Iπ processes. The mapping is reported in Figure 4. It is an extension of Milner's encoding of the λ-calculus and is therefore indicated with the same symbol $\mathcal{V}$. The optimisation $\mathcal{O}$ of Milner's encoding mentioned at the end of the previous section is correspondingly extended to an optimisation of the mapping for configurations. Again, we shall sometimes make use of such optimisations in proofs.

The mapping uses a representation of environments γ as associative lists, introduced in Section 2. So it relies on a choice of an order on the names that form the domain of γ. This choice is irrelevant in the definition of the mapping into Iπ, as behavioural equivalence in Iπ is invariant under commutativity and associativity of parallel composition.

Remark 6.1. The encoding of a configuration F with name-support ϕ does not depend on ϕ. This name-support ϕ is used in OGS both to enforce freshness of names, and to deduce the polarity of names, as represented by the function pol. And indeed, the process $\mathcal{V}[\![F]\!]$ has its set of free names included in ϕ, and uses P-names in outputs and O-names in inputs. The freshness property could be made more explicit by employing configurations also in Iπ. The polarity property could also be stated in π-calculus using i/o-sorting [PS96]. Indeed, a correspondence between arenas of game semantics (used to enforce polarities of moves) and sorting has been explored [HO95, HY99].

6.1. Operational correspondence.

The following theorems establish the operational correspondence between the A-OGS and Iπ representations.

Theorem 6.2.

(1) If $F \Longrightarrow_{\mathtt{a}} F'$, then $\mathcal{V}[\![F]\!] \Longrightarrow_\pi {}_\pi\!\gtrsim \mathcal{V}[\![F']\!]$;

(2) If $F \overset{\ell}{\Longrightarrow}_{\mathtt{a}} F'$, then $\mathcal{V}[\![F]\!] \overset{\ell}{\Longrightarrow}_\pi \approx_\pi \mathcal{V}[\![F']\!]$.

Theorem 6.3.

(1) If $\mathcal{V}[\![F]\!] \Longrightarrow_\pi P$ then there is F' such that $F \Longrightarrow_{\mathtt{a}} F'$ and $P \approx_\pi \mathcal{V}[\![F']\!]$;

(2) If $\mathcal{V}[\![F]\!] \overset{\ell}{\Longrightarrow}_\pi P$ and ℓ is an output, then there is F' such that $F \overset{\ell}{\Longrightarrow}_{\mathtt{a}} F'$ and $P \approx_\pi \mathcal{V}[\![F']\!]$;

(3) If F is passive and $\mathcal{V}[\![F]\!] \overset{\ell}{\Longrightarrow}_\pi P$, then there is F' such that $F \overset{\ell}{\Longrightarrow}_{\mathtt{a}} F'$ and $P \approx_\pi \mathcal{V}[\![F']\!]$.

Both theorems above, as well as similar operational correspondence theorems in this and next section, are first proved in terms of the optimised encoding $\mathcal{O}$ (introduced in [DHS18] and discussed in the previous section and in Appendix B), and exploiting its tight correspondence with the main encoding $\mathcal{V}$. Moreover, the operational correspondence is first established on strong transitions, along the lines of analogous correspondence results, such as those detailed in Section 6.2 and Appendix B. The correspondence on strong transitions, as usual in these cases, is carried out by induction on the depth of the proof of a given transition; then the results are extended to weak transitions using induction on the length of the given weak transition. The advantage of $\mathcal{O}$ is that it does not have initial 'administrative' reductions; hence any initial reduction corresponds to an internal move of a λ-term. This, for instance, allows us to have the expansion relation ${}_{\pi}\gtrsim$ (in place of the coarser $\approx_\pi$, even though the final goal is the result for $\approx_\pi$), in the statement about silent actions for the operational correspondence from Iπ to A-OGS (see, e.g., Lemma B.7 in the Appendix, and similar lemmas). The use of expansion is essential, both to derive the operational correspondence theorems on weak transitions, and to use the theorems in up-to techniques for Iπ (more generally, in applications of the theorems in which one reasons about the number of steps performed).

In Theorem 6.3, a clause is missing for input actions from $\mathcal{V}[\![F]\!]$ when F active. Indeed such actions are possible in Iπ, stemming from the (encoding of the) environment of F, whereas they are not possible in A-OGS. This is rectified in Section 6.2, introducing a constrained LTS for Iπ, and in Section 7, considering a concurrent OGS.

Corollary 6.4. *If $F \stackrel{s}{\Longrightarrow}_{\mathtt{a}}$ then also $\mathcal{V}[\![F]\!] \stackrel{s}{\Longrightarrow}_{\pi}$.*

Proof. By induction on the length of trace s. □

6.2. **An output-prioritised Transition System.** We define an LTS for Iπ in which input actions are visible only if no output can be consumed, either as a visible action or through an internal action (i.e., syntactically the process has no unguarded output). The new LTS, called *output-prioritised* and indicated as opLTS, is defined on top of the ordinary one by means of the two rules below. A process P is *input reactive* if whenever $P \stackrel{\mu}{\longrightarrow}_{\pi} P'$, for some μ, P' then μ is an input action.

$$\frac{P \stackrel{\mu}{\longrightarrow}_{\pi} P' \quad P \text{ input reactive}}{P \stackrel{\mu}{\longrightarrow}_{\mathtt{o}\pi} P'}$$

$$\frac{P \stackrel{\mu}{\longrightarrow}_{\pi} P'}{P \stackrel{\mu}{\longrightarrow}_{\mathtt{o}\pi} P'} \mu \text{ is an output or } \tau \text{ action}$$

The opLTS captures an aspect of *sequentiality* in π-calculi: a free input prefix is to be thought of as a service offered to the external environment; in a sequential system, such a service is available only if there are no ongoing computations due to previous interrogations of the server. An ongoing computation is represented by a τ-action, indicating a step of computation internal to the server, or an output, indicating either an answer to a client or a request to an external server. The constraint imposed by the new LTS can also be formalised compositionally, as done in Figure 5.

Under the opLTS, the analogue of Theorem 6.2 continues to hold: in A-OGS configurations, input transitions only occur in passive configurations, and the encodings of passive configurations are input-reactive processes. However, now we have the full converse of Theorem 6.3 and, as a consequence, we can also establish the converse direction of Corollary 6.4.

Lemma 6.5.

(1) If $\mathcal{V}[\![F]\!] \Longrightarrow_{\mathtt{o}\pi} P$ then there is F' such that $F \Longrightarrow_{\mathtt{a}} F'$ and $P \approx_\pi \mathcal{V}[\![F']\!]$;

(2) If $\mathcal{V}[\![F]\!] \stackrel{\ell}{\Longrightarrow}_{\mathtt{o}\pi} P$ then there is F' such that $F \stackrel{\ell}{\Longrightarrow}_{\mathtt{a}} F'$ and $P \approx_\pi \mathcal{V}[\![F']\!]$.

$$\texttt{alpha:}\ \frac{P \equiv_\alpha P' \quad P' \xrightarrow{\mu}_{\mathsf{o}\pi} P''}{P \xrightarrow{\mu}_{\mathsf{o}\pi} P''} \qquad \texttt{pre:}\ \frac{}{\mu.\, P \xrightarrow{\mu}_{\mathsf{o}\pi} P}$$

$$\texttt{parL:}\ \frac{P \xrightarrow{\mu}_{\mathsf{o}\pi} P' \quad (Q \text{ input reactive}) \text{ or } (\mu \text{ is an output or } \tau)}{P \mid Q \xrightarrow{\mu}_{\mathsf{o}\pi} P' \mid Q}$$

$$\texttt{com:}\ \frac{P \xrightarrow{\mu}_{\mathsf{o}\pi} P' \quad Q \xrightarrow{\overline{\mu}}_{\mathsf{o}\pi} Q' \quad \mu \neq \tau,\ \tilde{x} = \mathtt{bn}(\mu)}{P \mid Q \xrightarrow{\tau}_{\mathsf{o}\pi} \boldsymbol{\nu}\tilde{x}\,(P' \mid Q')}$$

$$\texttt{res:}\ \frac{P \xrightarrow{\mu}_{\mathsf{o}\pi} P' \quad x \notin \mathtt{n}(\mu)}{\boldsymbol{\nu}x\, P \xrightarrow{\mu}_{\mathsf{o}\pi} \boldsymbol{\nu}x\, P'} \qquad \texttt{rep:}\ \frac{\mu.\, P \xrightarrow{\mu}_{\mathsf{o}\pi} P'}{!\mu.\, P \xrightarrow{\mu}_{\mathsf{o}\pi} P' \mid\, !\mu.\, P}$$

$$\texttt{con:}\ \frac{P \xrightarrow{\mu}_{\mathsf{o}\pi} P' \quad \mathtt{K} \stackrel{\text{def}}{=} (\widetilde{y}).\, Q \quad (\widetilde{y}).\, Q \equiv_\alpha (\widetilde{x}).\, P}{\mathtt{K}\langle \widetilde{x} \rangle \xrightarrow{\mu}_{\mathsf{o}\pi} P'}$$

FIGURE 5. A compositional presentation of the output-prioritised LTS for Iπ

Again, in the proof, we first establish a correspondence result on strong transitions, and reason on the optimised encoding $\mathcal{O}$. See Appendix B for details.

Lemma 6.5 talks about the opLTS of Iπ; however the theorems make use of the ordinary expansion relation $\lesssim_\pi$, which is defined on the ordinary LTS. We now show that such uses of expansion can actually be replaced by expansion on the opLTS, defined as ordinary expansion, but on the opLTS.

A behavioural relation on the ordinary LTS 'almost always' implies the corresponding relation on the opLTS. The only exceptions are produced by processes that are related although they exhibit different divergence behaviours (by divergence we mean the possibility of performing an infinite number of τ actions) and at the same time may perform input actions. For instance, using τ^ω to indicate a purely divergent process (i.e., a process whose only transition is $\tau^\omega \xrightarrow{\tau} \tau^\omega$), the processes $a.\,0 \mid \tau^\omega$ and $a.\,0$ are bisimilar (and trace equivalent) in the ordinary LTS, but they are not so in the opLTS, since the divergence in the former process prevents observation of the input at a.

Indeed, a 'bisimilarity respecting divergence' would imply bisimilarity on the opLTS. Such 'bisimilarity respecting divergence', written $\approx_\pi^{\Uparrow}$, is defined as $\approx_\pi$ with the additional requirement that $P \approx_\pi Q$ implies

- $P \Uparrow$ iff $Q \Uparrow$ $\qquad (*)$

where the predicate $\Uparrow$ holds for an agent R if it has a divergence (an infinite path consisting of only τ actions).

Lemma 6.6. *Relation $\approx_\pi^{\Uparrow}$ implies $\approx_{\mathsf{o}\pi}$.*

The same result applies to other relations; e.g., the 'expansion respecting divergence' (defined by adding the clause $(*)$ above to those of the expansion relation $\lesssim_\pi$) on the ordinary LTS implies expansion on the opLTS.

A comment about expansion ($\lesssim_\pi$) is worthwhile. As pointed out earlier, the use of expansion is important in proofs about operational correspondence results. Such uses are due to the appearance of expansion in lemmas such as Lemma 5.2 (and Lemmas B.1-B.3, in Appendix B), and come from application of some simple algebraic laws that respect divergence (i.e., the laws do not add or remove any divergence). Indeed the laws either are valid for strong

bisimilarity, or they are laws such as

$$\boldsymbol{\nu} a\,(a(\tilde{x}).\,P \mid \overline{a}(\tilde{x}).\,Q) \;\; {}_{\pi}\!\gtrsim \;\; \boldsymbol{\nu} a\,(P \mid Q)$$

As a consequence, all occurrences of $\lesssim_\pi$ can be replaced by the expansion relation defined on the opLTS ($\lesssim_{\mathtt{o}\pi}$). In this manner, for instance, we can refine (the proof of) Lemma 6.5 and derive:

Theorem 6.7.

(1) If $\mathcal{V}[\![F]\!] \Longrightarrow_{\mathtt{o}\pi} P$ then there is F' such that $F \Longrightarrow_{\mathtt{a}} F'$ and $P \approx_{\mathtt{o}\pi} \mathcal{V}[\![F']\!]$;
(2) If $\mathcal{V}[\![F]\!] \overset{\ell}{\Longrightarrow}_{\mathtt{o}\pi} P$ then there is F' such that $F \overset{\ell}{\Longrightarrow}_{\mathtt{a}} F'$ and $P \approx_{\mathtt{o}\pi} \mathcal{V}[\![F']\!]$.

A final remark about behavioural equivalences on the ordinary and opLTS concerns the encoding of λ-terms. On such terms, the ordinary bisimilarity $\approx_\pi$ implies the 'bisimilarity respecting divergence' $\approx_\pi^{\Uparrow}$. This is because, intuitively, a term $\mathcal{V}[\![M]\!]\langle p\rangle$ is divergent iff M is so in the call-by-value λ-calculus. (Again, this property boils down to the fact that the uses of $\lesssim_\pi$ in Lemmas such as 5.2-B.3 respect divergence.) See Appendix A for details.

From Theorem 6.7 we derive:

Corollary 6.8. *For any configuration F and trace s, we have $F \overset{s}{\Longrightarrow}_{\mathtt{a}}$ iff $\mathcal{V}[\![F]\!] \overset{s}{\Longrightarrow}_{\mathtt{o}\pi}$.*

Remark 6.9. We recall that, following Remark 2.1 on the usage of bound names, in Corollary 6.8 the bound names in s are fresh; thus they do not appear in F. (Similarly in Theorems 6.3 and 6.7, for the bound names in ℓ.)

Remark 6.10. Corollary 6.8 relies on Theorems 6.2 and 6.7. The corollary talks about the opLTS of Iπ; however the theorems make use of the ordinary expansion relation $\lesssim_\pi$, which is defined on the ordinary LTS. Such uses of expansion can however be replaced by expansion on the opLTS (defined as ordinary expansion, but on the opLTS). For more details on this, see Appendix A.

As a consequence of Corollary 6.8, trace equivalence is the same, on A-OGS configurations and on the encoding of Iπ terms. Moreover, from Theorem 6.7 the same result holds under a bisimulation semantics. Further, since the LTS produced by A-OGS is deterministic, its trace semantics coincides with its bisimulation semantics. We can thus conclude as in Corollary 6.11. We recall that $\eqsim_{\mathtt{o}\pi}$ and $\approx_{\mathtt{o}\pi}$ are, respectively, trace equivalence and bisimilarity between Iπ processes in the opLTS; similarly for $\eqsim_{\mathtt{a}}$ and $\approx_{\mathtt{a}}$ between A-OGS terms.

Corollary 6.11. *For any F, F' we have: $F \eqsim_{\mathtt{a}} F'$ iff $\mathcal{V}[\![F]\!] \eqsim_{\mathtt{o}\pi} \mathcal{V}[\![F']\!]$ iff $F \approx_{\mathtt{a}} F'$ iff $\mathcal{V}[\![F]\!] \approx_{\mathtt{o}\pi} \mathcal{V}[\![F']\!]$.*

Corollary 6.11 holds in particular when F is the initial configuration for a λ-term. That is, the equality induced on call-by-value λ-terms by their representation in A-OGS and in Iπ (under the opLTS) is the same, both employing traces and employing bisimulation to handle the observables for the two models.

Corollary 6.12. *For any λ-terms M, N, we have:*
$\langle M\rangle \eqsim_{\mathtt{a}} \langle N\rangle$ iff $\langle M\rangle \approx_{\mathtt{a}} \langle N\rangle$ iff $\mathcal{V}[\![M]\!] \eqsim_{\mathtt{o}\pi} \mathcal{V}[\![N]\!]$ iff $\mathcal{V}[\![M]\!] \approx_{\mathtt{o}\pi} \mathcal{V}[\![N]\!]$.

From Theorem 6.3 and Corollary 6.8, it also follows that F and $\mathcal{V}[\![F]\!]$ are weakly bisimilar, on the union of the respective LTSs.

$(P\tau)$	$\langle A \cdot [p \mapsto M], \gamma, \phi\rangle$	$\xrightarrow{\tau}_{\mathtt{c}}$	$\langle A \cdot [p \mapsto N], \gamma, \phi\rangle$	when $M \to_{\mathtt{v}} N$
(PA)	$\langle A \cdot [p \mapsto V], \gamma, \phi\rangle$	$\xrightarrow{\bar{p}(x)}_{\mathtt{c}}$	$\langle A, \gamma \cdot [x \mapsto V], \phi \uplus \{x\}\rangle$	
(PQ)	$\langle A \cdot [p \mapsto E[xV]], \gamma, \phi\rangle$	$\xrightarrow{\bar{x}(y,q)}_{\mathtt{c}}$	$\langle A, \gamma \cdot [y \mapsto V] \cdot [q \mapsto (E, p)], \phi \uplus \{y, q\}\rangle$	
(OA)	$\langle A, \gamma \cdot [p \mapsto (E, q)], \phi\rangle$	$\xrightarrow{p(x)}_{\mathtt{c}}$	$\langle A \cdot [q \mapsto E[x]], \gamma, \phi \uplus \{x\}\rangle$	
(OQ)	$\langle A, \gamma, \phi\rangle$	$\xrightarrow{x(y,p)}_{\mathtt{c}}$	$\langle A \cdot [p \mapsto Vy], \gamma, \phi \uplus \{y, p\}\rangle$	when $\gamma(x) = V$
(IOQ)	$\langle [? \mapsto M], \phi\rangle$	$\xrightarrow{(p)}_{\mathtt{c}}$	$\langle [p \mapsto M], \varepsilon, \phi \uplus \{p\}\rangle$	

FIGURE 6. The LTS for the Concurrent OGS

7. Concurrent Operational Game Semantics

In order to establish an exact correspondence between OGS and the encoding into π-calculus, as established in Theorem 6.7, we had to introduce the opLTS for the π-calculus.

In this section, we explore another way to derive an exact correspondence between OGS and Iπ, by relaxing the Alternating LTS for OGS to allow multiple terms in configurations to run concurrently. We refer to the resulting OGS as the *Concurrent OGS*, briefly C-OGS (we recall that A-OGS refers to the Alternating OGS of Section 4).

7.1. **The Concurrent OGS LTS.** We introduce *running terms*, ranged over by A, B, as partial maps with finite domains from continuation names to λ-terms. Each mapping $[p \mapsto M]$ of a running term can be seen as a thread evaluating M, with p an identifier used to communicate the result of the evaluation. A *concurrent configuration* is a triple $\langle A, \gamma, \phi\rangle$ of a running term A, an environment γ, and a set of names ϕ. Moreover, the domains of A and γ must be disjoint. Both the running term and the environment may be empty. The continuation structure of such a concurrent configuration is then defined as the pair $(\mathrm{dom}(A), \mathrm{cs}(\gamma))$ formed by the domain of A and the continuation structure of γ. We extend the definition of the polarity function, considering names in the domain of both A and γ as Player names.

Passive and active configurations can be seen as special cases of C-OGS configurations with empty and singleton running term, respectively. For this reason we still use F, G to range over C-OGS configurations. Moreover, we freely take A-OGS configurations to be C-OGS configurations, and conversely for C-OGS configurations with empty and singleton running term, omitting the obvious syntactic coercions.

We present the rules of C-OGS in Figure 6. Since there is no more distinction between passive and active configurations, a given configuration can perform both Player and Opponent actions. Notice that only Opponent can add a new term to the running term A.

A *singleton configuration* is a configuration F that has only one P-name (that is, in C-OGS, F is either of the form $\langle [p \mapsto M], \varepsilon, \phi\rangle$, or $\langle \varepsilon, [x \mapsto V], \phi\rangle$, or $\langle \varepsilon, [p \mapsto (E, q)], \phi\rangle$).

In this and in the following section F, G range over C-OGS configurations, as reminded by the index '$\mathtt{c}$' in the symbols for LTS and behavioural equivalence with which F, G appear (e.g., $\eqsim_{\mathtt{c}}$).

7.2. **Comparison between** C-OGS **and Iπ.** The encoding of C-OGS into Iπ is a simple adaptation of that for A-OGS. We only have to consider the new syntactic element of C-OGS, namely running terms; the encoding remains otherwise the same. The encoding of a running term is:

$$\begin{array}{rcl} \mathcal{V}[\![[p \mapsto M] \cdot A]\!] & \stackrel{\text{def}}{=} & \mathcal{V}[\![M]\!]\langle p\rangle \mid \mathcal{V}[\![A]\!] \\ \mathcal{V}[\![\varepsilon]\!] & \stackrel{\text{def}}{=} & 0 \end{array}$$

The encoding of configurations is then defined as:

$$\mathcal{V}[\![\langle A, \gamma, \phi\rangle]\!] \stackrel{\text{def}}{=} \mathcal{V}[\![A]\!] \mid \mathcal{V}[\![\gamma]\!]$$

The results about the operational correspondence between C-OGS and Iπ are as those between A-OGS and Iπ under the opLTS. We report the main result, namely the correspondence for weak transitions, as well as two lemmas about the correspondence for strong transitions, since they are also needed when lifting up-to techniques from Iπ to C-OGS, in Section 7.4. The reasoning in their proofs is similar to that in the operational correspondence for A-OGS (Lemmas B.6 and B.7).

Lemma 7.1 (from C-OGS to Iπ, strong transitions)**.**

(1) If $F \longrightarrow_{\mathsf{c}} F'$, then $\mathcal{O}[\![F]\!] \longrightarrow_{\pi} {}_{\pi}\gtrsim \mathcal{O}[\![F']\!]$;

(2) If $F \xrightarrow{\ell}_{\mathsf{c}} F'$, then $\mathcal{O}[\![F]\!] \xrightarrow{\ell}_{\pi} \approx_{\pi} \mathcal{O}[\![F']\!]$.

Lemma 7.2 (from Iπ to C-OGS, strong transitions)**.**

(1) If $\mathcal{O}[\![F]\!] \longrightarrow_{\pi} P$ then there is F' such that $F \longrightarrow_{\mathsf{c}} F'$ and $P \, {}_{\pi}\gtrsim \mathcal{O}[\![F']\!]$;

(2) If $\mathcal{O}[\![F]\!] \xrightarrow{\ell}_{\pi} P$ then there is F' such that $F \xrightarrow{\ell}_{\mathsf{c}} F'$ and $P \approx_{\pi} \mathcal{O}[\![F']\!]$.

Theorem 7.3.

(1) If $F \Longrightarrow_{\mathsf{c}} F'$ then $\mathcal{V}[\![F]\!] \Longrightarrow_{\pi} {}_{\pi}\gtrsim \mathcal{V}[\![F']\!]$;

(2) if $F \stackrel{\ell}{\Longrightarrow}_{\mathsf{c}} F'$ then $\mathcal{V}[\![F]\!] \stackrel{\ell}{\Longrightarrow}_{\pi} \approx_{\pi} \mathcal{V}[\![F']\!]$;

(3) if $\mathcal{V}[\![F]\!] \Longrightarrow_{\pi} P$ then there is F' such that $F \Longrightarrow_{\mathsf{c}} F'$ and $P \approx_{\pi} \mathcal{V}[\![F']\!]$;

(4) if $\mathcal{V}[\![F]\!] \stackrel{\ell}{\Longrightarrow}_{\pi} P$ then there is F' such that $F \stackrel{\ell}{\Longrightarrow}_{\mathsf{c}} F'$ and $P \approx_{\pi} \mathcal{V}[\![F']\!]$.

Corollary 7.4. *For any* C-OGS *configuration F and trace s, we have $F \stackrel{s}{\Longrightarrow}_{\mathsf{c}}$ iff $\mathcal{V}[\![F]\!] \stackrel{s}{\Longrightarrow}_{\pi}$.*

From Corollary 7.4 and Theorem 7.3, we derive:

Lemma 7.5. *For any F, F' we have:*

(1) $F_1 \eqsim_{\mathsf{c}} F_2$ iff $\mathcal{V}[\![F_1]\!] \eqsim_{\pi} \mathcal{V}[\![F_2]\!]$;

(2) $F_1 \approx_{\mathsf{c}} F_2$ iff $\mathcal{V}[\![F_1]\!] \approx_{\pi} \mathcal{V}[\![F_2]\!]$.

To derive the full analogue of Corollary 6.12, we now show that, on the Iπ representation of λ-terms, trace equivalence is the same as bisimilarity. This result needs a little care: it is known that on deterministic LTSs bisimilarity coincides with trace equivalence. (In a deterministic LTS, intuitively, for any process and action there exists at most one proof of derivation of that action from the process). However, the behaviour of the Iπ representation of a C-OGS configuration need not be deterministic, because there could be multiple silent transitions as well as multiple output transitions (for instance, in C-OGS rule OQ may be applicable to different terms).

Lemma 7.6. *For any M, N we have: $\mathcal{V}[\![M]\!] \doteq_\pi \mathcal{V}[\![N]\!]$ iff $\mathcal{V}[\![M]\!] \approx_\pi \mathcal{V}[\![N]\!]$.*

Proof. Bisimilarity implies trace equivalence. The delicate direction is the converse one. It is easier to work with the optimised encoding $\mathcal{O}$. The relation

$$(\mathcal{O}[\![F]\!], \mathcal{O}[\![G]\!]) \ : \quad F, G \text{ are singleton with } F \doteq_{\mathtt{c}} G$$

is a bisimulation up-to context and up-to $({}_\pi\!\gtrsim, \approx_\pi)$. The proof uses Lemmas C.1 and C.2 and is reported in Appendix C. □

We can finally combine Lemmas 7.6 and 7.5 to derive that the C-OGS and Iπ semantics of λ-calculus coincide, both for traces and for bisimilarity.

Corollary 7.7. *For all M, N we have: $\langle M \rangle \doteq_{\mathtt{c}} \langle N \rangle$ iff $\langle M \rangle \approx_{\mathtt{c}} \langle N \rangle$ iff $\mathcal{V}[\![M]\!] \doteq_\pi \mathcal{V}[\![N]\!]$ iff $\mathcal{V}[\![M]\!] \approx_\pi \mathcal{V}[\![N]\!]$.*

7.3. **Tensor Product.** We now introduce a way of combining configurations, which corresponds to the notion of tensor product of arenas and strategies in (denotational) game semantics.

Definition 7.8. Two concurrent configurations F, G are said to be *compatible* if their polarity functions $\mathrm{pol}_F, \mathrm{pol}_G$ are compatible — that is, for all $a \in \mathrm{dom}(\mathrm{pol}_F) \cap \mathrm{dom}(\mathrm{pol}_G)$, we have $\mathrm{pol}_F(a) = \mathrm{pol}_G(a)$.

Following Definition 3.7, two π-calculus processes P_1, P_2 *cannot interact* if there is no name that appears free in both processes and with opposite polarities (that is, in input position in one, and output position in the other). As in Iπ all names exchanged are fresh, the 'cannot interact' property is preserved by transitions.

Lemma 7.9. *If P_1, P_2 cannot interact and $P_1 \mid P_2 \stackrel{s}{\Longrightarrow}_\pi P$, then $P = P_1' \mid P_2'$, for some P_1', P_2' that cannot interact.*

The Iπ translation of two compatible C-OGS configurations gives two processes that cannot interact with each other.

Lemma 7.10. *Taking two* C-OGS *configurations F_1, F_2, if their polarity functions are compatible then $\mathcal{V}[\![F_1]\!]$ and $\mathcal{V}[\![F_2]\!]$ cannot interact.*

Definition 7.11. For compatible configurations $F = \langle A, \gamma, \phi \rangle$ and $G = \langle B, \delta, \phi' \rangle$, the *tensor product* $F \otimes G$ is defined as

$$F \otimes G \triangleq \langle A \cdot B, \gamma \cdot \delta, \phi \cup \phi' \rangle$$

The polarity function of $F \otimes G$ is then equal to $\mathrm{pol}_F \cup \mathrm{pol}_G$, and $\mathcal{V}[\![F_1 \otimes F_2]\!] \equiv \mathcal{V}[\![F_1]\!] \mid \mathcal{V}[\![F_2]\!]$, where $\equiv$ is the standard structural congruence of π-calculi.

In the following, we write $\mathtt{inter}(s_1, s_2)$ for the set of traces obtained from an interleaving of the elements in the sequences s_1 and s_2.

We relate the traces generated by the tensor product of two configurations to the set of interleavings of traces generated by the component configurations themselves. The result is proved by going through the π-calculus, via the following lemma.

Lemma 7.12. *Suppose P_1, P_2 cannot interact.*

(1) If $P_1 \mid P_2 \stackrel{s}{\Longrightarrow}_\pi P_1' \mid P_2'$, then, for $i = 1, 2$, there is s_i such that $P_i \stackrel{s_i}{\Longrightarrow}_\pi P_i'$, and $s \in \mathtt{inter}(s_1, s_2)$;

(2) Conversely, if for $i = 1, 2$, we have $P_i \stackrel{s_i}{\Longrightarrow}_\pi P_i'$, and $s \in \mathtt{inter}(s_1, s_2)$, then $P_1 \mid P_2 \stackrel{s}{\Longrightarrow}_\pi P_1' \mid P_2'$.

Proof. The interesting case is (1). Consider the sequence of one-step actions (including silent actions) that are performed to obtain the trace $P_1 \mid P_2 \stackrel{s}{\Longrightarrow}_\pi P_1' \mid P_2'$. Then tag each of these actions with `L` or `R` depending on whether in the derivation proof of that action, the last rule applied was `parL` or `parR`. Then s_1 is obtained by collecting the subsequence of s with the `ParL` tag, and similarly for s_2 with `ParR`. □

We finally prove that the set of traces generated by $F_1 \otimes F_2$ is the union of the sets of interleaving $\mathtt{inter}(s_1, s_2)$, for $F_1 \stackrel{s_1}{\Longrightarrow}_{\mathtt{c}}$ and $F_2 \stackrel{s_2}{\Longrightarrow}_{\mathtt{c}}$.

Lemma 7.13. *Suppose F_1, F_2 are compatible concurrent configurations.*

(1) If $F_1 \otimes F_2 \stackrel{s}{\Longrightarrow}_{\mathtt{c}} F$, then, for $i = 1, 2$, there are traces s_i such that $F_i \stackrel{s_i}{\Longrightarrow}_{\mathtt{c}} F_i'$, and $s \in \mathtt{inter}(s_1, s_2)$, and $F = F_1' \otimes F_2'$;

(2) Conversely, if for $i = 1, 2$, we have $F_i \stackrel{s_i}{\Longrightarrow}_{\mathtt{c}} F_i'$, and $s \in \mathtt{inter}(s_1, s_2)$, then $F_1 \otimes F_2 \stackrel{s}{\Longrightarrow}_{\mathtt{c}} F_1' \otimes F_2'$.

Proof. It follows from Lemma 7.12, Lemma 7.10, Corollary 7.4, and the fact that $\mathcal{V}[\![F_1 \otimes F_2]\!] \equiv \mathcal{V}[\![F_1]\!] \mid \mathcal{V}[\![F_2]\!]$. □

The tensor product of A-OGS configurations will be introduced in Section 11.2.

7.4. **Up-to techniques for games.** We introduce up-to techniques for C-OGS, which allow, in bisimulation proofs, to split two C-OGS configurations into separate components and then to reason separately on these. These up-to techniques are directly imported from Iπ. Abstract settings for up-to techniques have been developed, see [PS12,PS19]; we cannot however derive the OGS techniques from them because these settings are specific to first-order LTS (i.e., CCS-like, without binders within actions). They are then used to prove that C-OGS and A-OGS yield the same equivalences on λ-terms. A further application is in Section 8, discussing eager normal-form bisimilarity.

In order to compare two configurations, they must share the same structure, as defined by the following notion.

Definition 7.14. Two configurations F, G are *support-equivalent*, written $\mathtt{se}(F, G)$, when they have the same polarity function (and so the same names) and the same continuation structure.
A relation $\mathcal{R}$ on configurations is *well-formed* if it relates support-equivalent configurations.

Below, all relations on configurations are meant to be well-formed. Given a well-formed relation $\mathcal{R}$ we write

- $\mathcal{R}^{|}$ for the relation $\{(F_1, F_2) \ : \ \exists\ G$ s.t. $F_i = F_i' \otimes G$ $(i = 1, 2)$ and $F_1'\ \mathcal{R}\ F_2'\}$.
- $\mathcal{R}^{|\star}$ for the reflexive and transitive closure of $\mathcal{R}^{|}$.
  Thus from $F_1\ \mathcal{R}\ G_1$ and $F_2\ \mathcal{R}\ G_2$ we obtain $(F_1 \otimes F_2)\ \mathcal{R}^{|\star}\ (G_1 \otimes F_2)\ \mathcal{R}^{|\star}\ G_1 \otimes G_2$.
- $\Rightarrow_{\mathtt{c}} \mathcal{R}^{|\star}\ {}_{\mathtt{c}}\Leftarrow$ for the closure of $\mathcal{R}^{|\star}$ under reductions. That is, $F_1 \Rightarrow_{\mathtt{c}} \mathcal{R}^{|\star}\ {}_{\mathtt{c}}\Leftarrow F_2$ holds if there are F_i', $i = 1, 2$ with $F_i \Longrightarrow_{\mathtt{c}} F_i'$ and $F_1'\ \mathcal{R}^{|\star}\ F_2'$. (As $\Longrightarrow_{\mathtt{c}}$ is reflexive, we may have $F_i = F_i'$.)

Definition 7.15. A relation $\mathcal{R}$ on configurations is a *bisimulation up-to reduction and composition* if whenever $F_1\ \mathcal{R}\ F_2$:

(1) if $F_1 \stackrel{\mu}{\longrightarrow}_{\mathtt{c}} F_1'$ then there is F_2' such that $F_2 \stackrel{\widehat{\mu}}{\Longrightarrow}_{\mathtt{c}} F_2'$ and $F_1' \Rightarrow_{\mathtt{c}} \mathcal{R}^{|\star}\ {}_{\mathtt{c}}\Leftarrow F_2'$;
(2) the converse, on the transitions from F_2.

Sometimes we do not need the 'closure under reduction'; that is, referring to the clauses of Definition 7.15 we may take $F_i' = F_i$, $(i = 1, 2)$. In this case we say that $\mathcal{R}$ is a 'bisimulation up-to composition'.

For the soundness of bisimulation up-to reduction and composition, we first need the following Lemma 7.16, which is a consequence of the compositionality of the mapping from C-OGS to Iπ.

Lemma 7.16. *For a relation $\mathcal{R}$ on* C-OGS *configurations, if $(F_1, F_2) \in \mathcal{R}^{|\star}$, then we have $(\mathcal{O}[\![F_1]\!], \mathcal{O}[\![F_2]\!]) \in (\mathcal{O}[\![\mathcal{R}]\!])^{\mathtt{ctx}}$.*

We are now ready to show the soundness of bisimulation up-to reduction and composition.

Theorem 7.17. *If $\mathcal{R}$ is bisimulation up-to reduction and composition then $\mathcal{R} \subseteq \approx_{\mathtt{c}}$.*

Proof. We prove the result via Iπ, exploiting Lemma 7.5(2). Here again, it is convenient to use the optimised encoding $\mathcal{O}$.

If $\mathcal{R}$ is a relation on C-OGS configurations, then $\mathcal{O}[\![\mathcal{R}]\!]$ is the relation on Iπ processes obtained by mapping each pair in $\mathcal{R}$ in the expected manner:

$$\mathcal{O}[\![\mathcal{R}]\!] \stackrel{\text{def}}{=} \{(\mathcal{O}[\![F_1]\!], \mathcal{O}[\![F_2]\!]) \ : \ F_1 \ \mathcal{R} \ F_2\}$$

And, if $\mathcal{S}$ is a relation on Iπ processes, then $\mathcal{S}^{\mathtt{ctx}}$ is the closure of $\mathcal{S}$ under polyadic contexts, i.e., the set of pairs of the form $(C_{\mathtt{ctx}}[P_1, \ldots, P_n], C_{\mathtt{ctx}}[Q_1, \ldots, Q_n])$ where $C_{\mathtt{ctx}}$ is a polyadic context and for each i, $P_i \ \mathcal{S} \ Q_i$.

As a consequence of the compositionality of the mapping from C-OGS to Iπ, for a relation $\mathcal{R}$ on C-OGS configurations, if $(F_1, F_2) \in \mathcal{R}^{|\star}$, then $(\mathcal{O}[\![F_1]\!], \mathcal{O}[\![F_2]\!]) \in (\mathcal{O}[\![\mathcal{R}]\!])^{\mathtt{ctx}}$.

Now, let $\mathcal{R}$ be the relation in the hypothesis of the theorem. We prove the theorem by showing that $\mathcal{O}[\![\mathcal{R}]\!]$ is a bisimulation up-to context and up-to $({}_\pi\gtrsim, \approx_\pi)$ in Iπ, and then appealing to Theorem 3.10.

Suppose $\mathcal{O}[\![F_1]\!] \xrightarrow{\ell}_\pi P$. Then there is F_1' with $F_1 \xrightarrow{\ell}_\pi F_1'$ and $P \approx_\pi \mathcal{O}[\![F_1']\!]$ (this is derived from Lemma 7.2). Since $\mathcal{R}$ is bisimulation up-to reduction and composition and $F_1 \ \mathcal{R} \ F_2$, there are F_1'', and F_2', with $F_1' \Longrightarrow_{\mathtt{c}} F_1''$, $F_2 \xrightarrow{\ell}_\pi \Longrightarrow_{\mathtt{c}} F_2'$ and $F_1'' \ \mathcal{R}^{|\star} \ F_2'$.

Using Theorem 7.3(1) from $F_1' \Longrightarrow_{\mathtt{c}} F_1''$ and the inclusion ${}_\pi\gtrsim \subseteq \approx_\pi$, we derive $\mathcal{O}[\![F_1']\!] \approx_\pi \mathcal{O}[\![F_1'']\!]$; and by Theorem 7.3(1), and Lemmas 5.2 and 7.1, from $F_2 \xrightarrow{\ell}_\pi \Longrightarrow_{\mathtt{c}} F_2'$ we derive $\mathcal{O}[\![F_2]\!] \xrightarrow{\ell}_\pi \approx_\pi \mathcal{O}[\![F_2']\!]$.

Finally, by Lemma 7.16, from $(F_1'', F_2') \in \mathcal{R}^{|\star}$, we derive $(\mathcal{O}[\![F_1']\!], \mathcal{O}[\![F_2']\!]) \in (\mathcal{O}[\![\mathcal{R}]\!])^{\mathtt{ctx}}$. This closes the proof, up-to polyadic contexts and bisimilarity.

The case when $\mathcal{O}[\![F_1]\!]$ makes a silent move is simpler, using expansion in place of bisimilarity. □

We now present an additional up-to technique, which will be used when relating OGS bisimilarity with eager normal form bisimilarity in Section 8. When $\mathcal{R}$ is a relation on singleton configurations, a straightforward simplification of 'bisimulation up-to reduction and composition' allows us to play the bisimulation game only on visible actions; the bisimulation clause becomes:

- if $F_1 \Longrightarrow_{\mathtt{c}} \xrightarrow{\ell}_{\mathtt{c}} F_1'$ then there is F_2' such that $F_2 \Longrightarrow_{\mathtt{c}} \xrightarrow{\ell}_{\mathtt{c}} F_2'$ and $F_1' \ \mathcal{R}^{|\star} \ F_2'$.

We call this a *singleton bisimulation up-to composition*; its soundness is a corollary of that for bisimulation up-to reduction and composition, derived from the up-to techniques for Iπ and the mapping from C-OGS onto Iπ.

Proposition 7.18. *If $\mathcal{R}$ is a singleton bisimulation up-to composition, then $\mathcal{R} \subseteq \approx_{\mathtt{c}}$.*

Proof. Take

$$\mathcal{S} \stackrel{\text{def}}{=} \mathcal{R} \cup \{(F_1, F_2) \ : \ \text{ there are } F_1', F_2' \text{ with } F_1' \ \mathcal{R} \ F_2' \text{ and } F_i' \Longrightarrow_{\mathtt{c}} F_i \ i = 1, 2\}$$

Intuitively, $\mathcal{S}$ is the closure of $\mathcal{R}$ under deterministic reductions. Then $\mathcal{S}$ is a bisimulation up-to composition. Note that we exploit the fact that transitions of singleton configurations are deterministic. □

Remark 7.19. Results such as Corollary 7.7 and Theorem 7.17 might suggest that the equivalence between two configurations implies the equivalence of all their singleton components. That is, if $F \mathrel{\dot{=}}_{\mathtt{c}} G$, with $[p \mapsto M]$ part of F and $[p \mapsto N]$ part of G, then also $\langle [p \mapsto M] \rangle \mathrel{\dot{=}}_{\mathtt{c}} \langle [p \mapsto N] \rangle$. A counterexample is given by the configurations

$$\left.\begin{array}{rcl} F_1 & \stackrel{\text{def}}{=} & \langle [p_1 \mapsto M] \cdot [p_2 \mapsto \Omega] \rangle \\ F_2 & \stackrel{\text{def}}{=} & \langle [p_1 \mapsto \Omega] \cdot [p_2 \mapsto M] \rangle \end{array}\right| \ \text{where } M \stackrel{\text{def}}{=} (\lambda z.\, \Omega)(x \lambda y.\, \Omega)$$

Intuitively the reason why $F_1 \eqsim_{\mathtt{c}} F_2$ is that the term M can produce an output (along the variable x), but an observer will never obtain access to the name at which M is located (p_1 or p_2). That is, the term M can interrogate x, but it will never answer, neither at p_1 nor at p_2. However the result does hold when F, G are reachable from λ-terms, that is, there are M, N and a trace s such that $\langle M\rangle \overset{s}{\Longrightarrow}_{\mathtt{c}} F$ and $\langle N\rangle \overset{s}{\Longrightarrow}_{\mathtt{c}} G$.

Remark 7.20. Up-to composition can be understood as a decomposition principle for configurations whose components do not interact with each other, even though they may share names that are only used for communication with the environment. This decomposition reduces the OGS bisimulation game to the analysis of singleton configurations, effectively allowing one to reason independently on each branch of a (possibly infinite) Böhm-like trees representation of normal forms, as it will be made apparent in Section 8 with Lassen trees. Through the well-known correspondence between such branches and P-views in game semantics [Cur98], this yields a view-based reasoning principle: up-to composition enables one to analyse strategies by considering each P-view independently, in a forward manner. In this sense, it captures and operationalises a characteristic feature of innocent strategies, namely that behaviours can be decomposed along views without mutual interference between them.

7.5. **Relationship between Concurrent and Alternating OGS.** In Section 6 we have proved that the trace-based and bisimulation-based semantics produced by A-OGS and by Iπ under the opLTS coincide. In Section 7.2 we have obtained the same result for C-OGS and Iπ under the ordinary LTS. In this section, we develop these results to conclude that all such equivalences for λ-terms actually coincide. In other words, the equivalence induced on λ-terms by their representations in OGS and Iπ is the same, regardless of whether we adopt the alternating or concurrent flavour for OGS, the opLTS or the ordinary LTS in Iπ, a trace or a bisimulation semantics.

For this, in one direction we show that the trace semantics induced by A-OGS coincides with the subset of *alternating* traces of C-OGS.

Lemma 7.21. *For any* A-OGS *configuration F, $F \overset{s}{\Longrightarrow}_{\mathtt{a}}$ if and only if $F \overset{s}{\Longrightarrow}_{\mathtt{c}}$ with s an alternating trace starting with an output when F is active.*

Proof. Taking a trace s such that $F \overset{s}{\Longrightarrow}_{\mathtt{a}}$, we directly get that s is alternating. Moreover, s starts with an output when F is active. Since the rules of A-OGS LTS are a subset of the ones of C-OGS, we also get that $F \overset{s}{\Longrightarrow}_{\mathtt{c}}$.

In the opposite direction, we take s an alternating trace such that $F \overset{s}{\Longrightarrow}_{\mathtt{c}}$ and s starts with an output when F is active. We prove $F \overset{s}{\Longrightarrow}_{\mathtt{a}}$ by induction over s. If the trace is empty, this is direct. Otherwise, we write s as $\mu s'$ such that $F \xrightarrow{\mu}_{\mathtt{c}} G \overset{s'}{\Longrightarrow}_{\mathtt{c}}$. Then we reason by case analysis over F:

- either F is active so that μ must be an output by the statement of the lemma. So we get that $F \xrightarrow{\mu}_{\mathtt{a}} G$ and G is passive.
- or F is passive, so that μ has to be an input. We get that $F \xrightarrow{\mu}_{\mathtt{a}} G$ with G active. Since s is alternating, we then get that s' is starting with an output.

Since s' is alternating, in both cases we conclude using the induction hypothesis. □

In the opposite direction, the difficulty is that the C-OGS LTS can perform more actions than the A-OGS: it can perform input actions on active configurations. So we state the correspondence only on singleton configurations. But neither the C-OGS nor the A-OGS output transitions preserve the fact of being singleton. The solution is to use bisimulation up-to composition over the C-OGS LTS in order to keep reasoning on singleton configurations. We then have to only extract singleton configurations from A-OGS configurations that can effectively be triggered directly by an action.

Definition 7.22. A singleton C-OGS configuration F is a *triggerable* subconfiguration of an A-OGS configuration F' when:

- either F' is active of the shape $\langle M, p, \gamma, \phi\rangle$, and $F = \langle [p \mapsto M], \varepsilon, \phi \backslash \mathrm{dom}(\gamma)\rangle$
- or F' is passive of the shape $\langle \gamma, \phi\rangle$, and F is:
  - either of the shape $\langle [x \mapsto V], \phi \backslash \mathrm{dom}(\gamma) \cup \{x\}\rangle$ with $[x \mapsto V] \subseteq \gamma$;

– or of the shape $\langle [p \mapsto (E, q)], \phi \backslash \mathrm{dom}(\gamma) \cup \{p\} \rangle$ with $[p \mapsto (E, q)] \subseteq \gamma$.

Notice that a triggerable subconfiguration F of F' is not formally a subconfiguration of F', as one needs to remove the Player names of F' not in the domain of F, and in the case of F' active, we transform the singleton running term of F into a term and a continuation name.

Configurations $\langle [x \mapsto V], \phi \rangle$ and $\langle [p \mapsto (E, q)], \phi \rangle$ are not triggerable when F is active, since they cannot be directly triggered by the A-OGS LTS (while they can be in the C-OGS LTS).

Lemma 7.23. *If F, G are* A-OGS *singleton configurations and $F \approx_{\mathsf{a}} G$, then also $F \approx_{\mathsf{c}} G$.*

Proof. Let $\mathcal{R}$ be the relation on C-OGS singleton configurations with $F \mathcal{R} G$ if $\mathtt{se}(F, G)$ and there exist A-OGS configurations F', G' with $F' \approx_{\mathsf{a}} G'$ such that F and G are triggerable singleton configurations respectively of F' and G'.

First notice that if F, G are A-OGS singleton configurations and $F \approx_{\mathsf{a}} G$, we directly get that $F \mathcal{R} G$ since F and G are triggerable subconfigurations of themselves, once the implicit coercion of A-OGS configurations is applied to C-OGS configurations.

We now show that $\mathcal{R}$ is a bisimulation up-to composition. We take $F \mathcal{R} G$, such that there exist F', G' as stated by the definition of $\mathcal{R}$. Suppose

$$F \xrightarrow{\mu}_{\mathsf{c}} F_1 \tag{7.1}$$

in C-OGS. First, we suppose that F, G are both passive, so that the rule can be OA or OQ. We assume it is OQ; the case for OA is similar. We thus have, for some p, V, x, y

$$\begin{array}{rcl} F & = & \langle \gamma_1, \phi \rangle \text{ with } \gamma_1 = [x \mapsto V] \\ \mu & = & x(y, p) \\ F_1 & = & \langle [p \mapsto Vy], \gamma_1, \phi \uplus \{y, p\} \rangle \end{array}$$

Moreover, we have $G = \langle \delta_1, \phi \rangle$ with $\delta_1 = [x \mapsto W]$. Since F, G are triggerable respectively from F', G', we get that both F', G' are passive. From $F' \approx_{\mathsf{a}} G'$, we get that $\mathtt{se}(F', G')$, so there exist γ, δ sharing the same domain such that:

$$\begin{array}{rcl} F' & = & \langle \gamma_1 \cdot \gamma, \phi \uplus \mathrm{dom}(\gamma) \rangle \\ G' & = & \langle \delta_1 \cdot \delta, \phi \uplus \mathrm{dom}(\delta) \rangle \end{array}$$

We also have $F' \xrightarrow{\mu}_{\mathsf{a}} \langle Vy, p, \gamma_1 \cdot \gamma, \phi \uplus \mathrm{dom}(\gamma) \uplus \{y, p\} \rangle \stackrel{\text{def}}{=} F''$. Since $F' \approx_{\mathsf{a}} G'$,

$$G' \xrightarrow{\mu}_{\mathsf{a}} \langle Wy, q, \delta_1 \cdot \delta, \phi \uplus \mathrm{dom}(\delta) \uplus \{y, p\} \rangle \stackrel{\text{def}}{=} G''$$

and $F'' \approx_{\mathsf{a}} G''$. Let $G_1 = \langle [q \mapsto Wy], \delta_1, \phi \rangle$. We also have

$$G \xrightarrow{\mu}_{\mathsf{c}} G_1.$$

This is sufficient to match (7.1), as $F_1 \, \mathcal{R}^{|\star} \, G_1$. Indeed we have:

- $\langle \gamma_1, \phi \uplus \{y, p\} \rangle \mathcal{R} \langle \delta_1, \phi \uplus \{y, p\} \rangle$ since $F \mathcal{R} G$;
- $\langle [p \mapsto Vy], \phi \uplus \{y, p\} \rangle \mathcal{R} \langle [q \mapsto Wy], \phi \uplus \{y, p\} \rangle$ since they are triggerable subconfigurations respectively of F'' and G''.

Now suppose that F, G are active, so there are 3 possibilities of transitions, corresponding to rules Pτ, PA, PQ. The case of Pτ is straightforward since bisimilarity is preserved by internal actions. We only look at PQ, as PA is simpler. Thus we have $F = \langle [p \mapsto E[xV]], \varepsilon, \phi \rangle$, for some E, x, V and:

$$F \xrightarrow{\bar{x}(y,q)}_{\mathsf{c}} \langle [y \mapsto V] \cdot [q \mapsto (E, p)], \phi \uplus \{y, q\} \rangle \stackrel{\text{def}}{=} F_1 \tag{7.2}$$

Since G shares the same continuation structure as F, we have $G = \langle [p \mapsto M], \varepsilon, \phi \rangle$ for some term M. Since F, G are triggerable respectively from F', G', there exist γ, δ sharing the same domain such that:

$$\begin{array}{rcl} F' & = & \langle E[xV], p, \gamma, \phi \uplus \mathrm{dom}(\gamma) \rangle \\ G' & = & \langle M, p, \delta, \phi \uplus \mathrm{dom}(\delta) \rangle \end{array}$$

We also have

$$F' \xrightarrow{\bar{x}(y,q)}_{\mathtt{a}} \langle [y \mapsto V] \cdot [q \mapsto (E, p)] \cdot \gamma, \phi \uplus \{y, q\} \rangle \stackrel{\text{def}}{=} F''$$

As $F' \approx_{\mathtt{a}} G'$, there exist E', W such that

$$G' \Longrightarrow \langle E'[xW], p, \delta, \phi \rangle \xrightarrow{\bar{x}(y,q)}_{\mathtt{a}} \langle [y \mapsto W] \cdot [q \mapsto (E', p)] \cdot \delta, \phi \uplus \{y, q\} \rangle \stackrel{\text{def}}{=} G''$$

with $F'' \approx_{\mathtt{a}} G''$. Defining G_1 as $\langle [y \mapsto W] \cdot [q \mapsto (E', p)], \phi \uplus \{y, q\} \rangle$, we also have $G \overset{\bar{x}(y,q)}{\Longrightarrow}_{\mathtt{c}} G_1$. We deduce that both $\langle [y \mapsto V], \phi \uplus \{y\} \rangle \mathcal{R} \langle [y \mapsto W], \phi \uplus \{y\} \rangle$ and $\langle [q \mapsto (E, p)], \phi \uplus \{q\} \rangle \mathcal{R} \langle [q \mapsto (E', p)], \phi \uplus \{q\} \rangle$ hold since all these singleton configurations are triggerable from either F'' or G'', and that $F'' \approx_{\mathtt{a}} G''$. Hence $F_1 \, \mathcal{R}^{|\star} \, G_1$.

In summary, as an answer to the challenge (7.2), we have found G_1 such that $G \overset{\bar{x}(y,q)}{\Longrightarrow}_{\mathtt{c}} G_1$ and $F_1 \, \mathcal{R}^{|\star} \, G_1$; this closes the proof. □

From Corollaries 6.12, 7.7, and Lemmas 7.21, 7.23 we can thus conclude.

Corollary 7.24. *For any λ-terms M, N, the following statements are the same: $\langle M \rangle \eqsim_{\mathtt{a}} \langle N \rangle$; $\langle M \rangle \approx_{\mathtt{a}} \langle N \rangle$; $\langle M \rangle \eqsim_{\mathtt{c}} \langle N \rangle$; $\langle M \rangle \approx_{\mathtt{c}} \langle N \rangle$; $\mathcal{V}[\![M]\!] \eqsim_{\mathtt{o}\pi} \mathcal{V}[\![N]\!]$; $\mathcal{V}[\![M]\!] \approx_{\mathtt{o}\pi} \mathcal{V}[\![N]\!]$; $\mathcal{V}[\![M]\!] \eqsim_{\pi} \mathcal{V}[\![N]\!]$; $\mathcal{V}[\![M]\!] \approx_{\pi} \mathcal{V}[\![N]\!]$.*

8. Eager Normal-Form Bisimulation

From Corollary 7.24 and Theorem 5.1, we can immediately conclude that the semantics on λ-terms induced by OGS (Alternating or Concurrent) coincides with enf-bisimilarity (i.e., Lassen's trees), as introduced in Definition 3.11.

In this section, we show a direct proof of the result, for C-OGS bisimilarity, as an example of application of the up-to composition technique for C-OGS presented in Section 7.4.

Terms, values, and evaluation contexts of the λ-calculus can be directly lifted to singleton concurrent configurations, meaning that we can transform a relation on terms, values, and contexts $\mathcal{R}$ into a relation $\widehat{\mathcal{R}}$ on singleton concurrent configurations in the following way:

- If $(M_1, M_2) \in \mathcal{R}$ then $(\langle [p \mapsto M_1], \varepsilon, \phi \rangle, \langle [p \mapsto M_2], \varepsilon, \phi \rangle) \in \widehat{\mathcal{R}}$, with $\phi = \mathtt{fv}(M_1, M_2) \uplus \{p\}$;
- If $(V_1, V_2) \in \mathcal{R}$ then $(\langle [x \mapsto V_1], \phi \rangle, \langle [x \mapsto V_2], \phi \rangle) \in \widehat{\mathcal{R}}$, with $\phi = \mathtt{fv}(V_1, V_2) \uplus \{x\}$;
- If $(E_1, E_2) \in \mathcal{R}$ then $(\langle [p \mapsto (E_1, q)], \phi \rangle, \langle [p \mapsto (E_2, q)], \phi \rangle) \in \widehat{\mathcal{R}}$, with $\phi = \mathtt{fv}(E_1, E_2) \uplus \{p, q\}$.

Theorem 8.1. *Taking $(\mathcal{R}_{\mathcal{M}}, \mathcal{R}_{\mathcal{V}}, \mathcal{R}_{\mathcal{K}})$ an enf-bisimulation, then $(\widehat{\mathcal{R}_{\mathcal{M}}} \cup \widehat{\mathcal{R}_{\mathcal{V}}} \cup \widehat{\mathcal{R}_{\mathcal{K}}})$ is a singleton bisimulation up-to composition in* C-OGS *(as defined in Section 7.4).*

Proof. Taking F_1, F_2 two singleton configurations s.t. $(F_1, F_2) \in \widehat{\mathcal{R}_{\mathcal{M}}}$, then we can write F_i as $\langle [p \mapsto M_i], \phi \rangle$ for $i = 1, 2$, such that $(M_1, M_2) \in \mathcal{R}$. Suppose that $F_1 \Longrightarrow_{\mathtt{c}} \xrightarrow{\ell}_{\mathtt{c}} F_1'$, with ℓ a Player action. We only present the Player Question case, which is where 'up-to composition' is useful. So writing ℓ as $\bar{x}(y, q)$, F_1' can be written as $\langle [y \mapsto V_1] \cdot [q \mapsto (E_1, p)] \rangle$, so that $M_1 \to_{\mathtt{v}}^* E_1[xV_1]$.

As $(M_1, M_2) \in \mathcal{R}$, there are E_2, V_2 s.t. $M_2 \to_{\mathtt{v}}^* E_2[xV_2]$, $(V_1, V_2) \in \mathcal{R}$ and $(E_1, E_2) \in \mathcal{R}$. Hence $F_2 \Longrightarrow_{\mathtt{c}} \xrightarrow{\ell}_{\mathtt{c}} F_2'$ with $F_2' = \langle [y \mapsto V_2] \cdot [q \mapsto (E_2, p)] \rangle$.
Finally, $(\langle [y \mapsto V_1] \rangle, \langle [y \mapsto V_2] \rangle) \in \widehat{\mathcal{R}}$ and $(\langle [q \mapsto (E_1, p)] \rangle, \langle [q \mapsto (E_2, p)] \rangle) \in \widehat{\mathcal{R}}$, so that $(F_1', F_2') \in \widehat{\mathcal{R}}^{|\star}$.
The case of passive singleton configurations is proved in a similar way. □

9. Compositionality of OGS via Iπ

We now present some compositionality results about C-OGS and A-OGS, that can be proved via the correspondence between OGS and Iπ.

Compositionality of OGS amounts to computing the set of traces generated by $\langle M\{V/x\} \rangle$ from the set of traces generated by $\langle M \rangle$ and $\langle [x \mapsto V] \rangle$. This is the cornerstone of (denotational) game semantics, where the combination of $\langle M \rangle$ and $\langle [x \mapsto V] \rangle$ is represented via the so-called 'parallel composition plus hiding'. This notion of parallel composition of two processes P, Q plus hiding over a name x is directly expressible in Iπ as the process $\nu x(P \mid Q)$. Precisely, suppose F, G are two configurations that agree on their polarity functions, except on a name x; then we write

$$\nu x(F \mid G) \qquad (*)$$

for the Iπ process $\nu x(\mathcal{V}[\![F]\!] \mid \mathcal{V}[\![G]\!])$, the *parallel composition plus hiding over* x. To define this operation directly at the level of OGS, we would have to generalize its LTS, allowing internal interactions over a name x used both in input and in output.

As the translation $\mathcal{V}$ from OGS configurations into Iπ validates the β_v rule, we can prove that the behaviour of $\langle M\{V/x\} \rangle$ (e.g., its set of traces) is the same as that of the parallel composition plus hiding over x of $\langle M \rangle$ and $\langle [x \mapsto V] \rangle$. We use the notation $(*)$ above to express the following two results.

Theorem 9.1. $\mathcal{V}[\![\langle [p \mapsto M\{V/x\}], \gamma, \phi \cup \phi' \rangle]\!] \bowtie \nu x(\mathcal{V}[\![\langle [p \mapsto M], \varepsilon, \phi \uplus \{x\} \rangle]\!] \mid \mathcal{V}[\![\langle \gamma \cdot [x \mapsto V], \phi' \uplus \{x\} \rangle]\!])$ *for* $\bowtie \in \{\approx_{\mathtt{o}\pi}, \approx_\pi, \eqsim_{\mathtt{o}\pi}, \eqsim_\pi\}$.

Proof. The theorem is a consequence of the validity of the β_v rule; that is the property $\mathcal{V}[\![(\lambda x.\, M)V]\!] \approx_\pi \mathcal{V}[\![M\{^V/_x\}]\!]$. Indeed, by simple algebraic manipulation one can show

$$\mathcal{V}[\![(\lambda x.\, M)V]\!]\langle p \rangle \approx_\pi \nu x(\mathcal{V}[\![M]\!]\langle p \rangle \mid \mathcal{V}^*[\![V]\!]\langle x \rangle)$$

which is to say

$$\mathcal{V}[\![\langle [p \mapsto M\{V/x\}], \varepsilon, \phi \cup \phi' \rangle]\!] \approx_\pi \nu x(\mathcal{V}[\![\langle [p \mapsto M], \varepsilon, \phi \uplus \{x\} \rangle]\!] \quad \mid \mathcal{V}[\![\langle [x \mapsto V], \phi' \uplus \{x\} \rangle]\!])$$

The result can then be lifted to a non-empty environment γ by using the congruence properties of $\approx_\pi$, and then to the other relations. □

Corollary 9.2. *For any trace* s*:*

(1) $\langle M\{V/x\}, p, \gamma, \phi \cup \phi' \rangle \overset{s}{\Longrightarrow}_{\mathtt{a}}$ *iff* $\nu x(\langle M, p, \varepsilon, \phi \uplus \{x\} \rangle \mid \langle \gamma \cdot [x \mapsto V], \phi' \uplus \{x\} \rangle) \overset{s}{\Longrightarrow}_{\mathtt{o}\pi}$;
(2) $\langle [p \mapsto M\{V/x\}], \gamma, \phi \cup \phi' \rangle \overset{s}{\Longrightarrow}_{\mathtt{c}}$ *iff* $\nu x(\langle [p \mapsto M], \varepsilon, \phi \uplus \{x\} \rangle \mid \langle \gamma \cdot [x \mapsto V], \phi' \uplus \{x\} \rangle) \overset{s}{\Longrightarrow}_{\pi}$.

Other important properties that we can import in OGS from Iπ are the congruence properties for the A-OGS and C-OGS semantics. We report the result for $\eqsim_{\mathtt{a}}$; the same result holds for $\approx_{\mathtt{a}}, \eqsim_{\mathtt{c}}, \approx_{\mathtt{c}}$.

Theorem 9.3. *If* $\langle M \rangle \eqsim_{\mathtt{a}} \langle N \rangle$ *then for any* Λ_{V} *context* C, $\langle C[M] \rangle \eqsim_{\mathtt{a}} \langle C[N] \rangle$.

The result is obtained from the congruence properties of Iπ, Corollary 7.24, and the compositionality of the encoding $\mathcal{V}$.

The adequacy and congruence result yields soundness (i.e., inclusion) of the OGS relations (Alternating and Concurrent) with respect to contextual equivalence. Soundness of OGS is usually a non-trivial property, e.g., requiring ciu-equivalence [Lai07].

$$
\begin{array}{llll}
(P\tau) & \langle M, p, \sigma, \gamma, \phi\rangle & \xrightarrow{\tau}_{\mathtt{wb}} & \langle N, p, \sigma, \gamma, \phi\rangle \qquad \text{when } M \longrightarrow N \\
(PA) & \langle V, p, \sigma, \gamma, \phi\rangle & \xrightarrow{\bar{p}(x)}_{\mathtt{wb}} & \langle \sigma, \gamma \cdot [x \mapsto V], \phi \uplus \{x\}\rangle \\
(PQ) & \langle E[xV], p, \sigma, \gamma, \phi\rangle & \xrightarrow{\bar{x}(y,q)}_{\mathtt{wb}} & \langle q :: \sigma, \gamma \cdot [y \mapsto V] \cdot [q \mapsto (E, p)], \phi \uplus \{y, q\}\rangle \\
(OA) & \langle p :: \sigma, \gamma \cdot [p \mapsto (E, q)], \phi\rangle & \xrightarrow{p(x)}_{\mathtt{wb}} & \langle E[x], q, \sigma, \gamma, \phi \uplus \{x\}\rangle \\
(OQ) & \langle \sigma, \gamma, \phi\rangle & \xrightarrow{x(y,p)}_{\mathtt{wb}} & \langle Vy, p, \sigma, \gamma, \phi \uplus \{y, p\}\rangle \qquad \text{when } \gamma(x) = V \\
(IOQ) & \langle [? \mapsto M], \phi\rangle & \xrightarrow{(p)}_{\mathtt{wb}} & \langle M, p, [], \varepsilon, \phi \uplus \{p\}\rangle
\end{array}
$$

FIGURE 7. Well-Bracketed OGS LTS

10. WELL-BRACKETED OGS

In the operational game semantics we have studied so far, Opponent is allowed to answer over continuation names in an arbitrary order, and is even allowed 'to forget' to answer to some of these names. This behavior of Opponent corresponds to that of a context that has access to control operators, e.g. `call/cc`, with an affine usage of continuations.

10.1. **Well-Bracketed LTS.** To capture the absence of such control operators in the environment, we enforce a 'well-bracketed' behavior on the sequence of answers. To do so, in Figure 7 we introduce the *Well-Bracketed OGS* (WB-OGS). It refines A-OGS by using *stacked configurations* $\boldsymbol{F}$. They use a *stack* of continuation names σ, that keeps track of the order in which the Opponent answers must occur to be well-bracketed. Such stacks are represented as lists of Player continuation names. We write $p :: \sigma$ for the stack with p pushed on top of σ, and $[]$ for the empty stack. In WB-OGS, Player questions push their fresh continuation names on top of the stack, and Opponent answers pop the stack to obtain the continuation name they are allowed to answer to.

Extending the notion of valid configurations from A-OGS, we require the stack to be formed by distinct Player continuation names. Since names in stacks are all distinct, removing the evaluation context from γ after an Opponent answer is not necessary anymore, since the associated continuation name will never be allowed to be used again. But for uniformity with respect to the other LTS, we choose to keep it. For a stacked configuration $\boldsymbol{F}$, we write $\widehat{\boldsymbol{F}}$ for the corresponding A-OGS configuration, where the stack has been removed.

Notice that stacked configuration $\boldsymbol{F}$ only stores the stack of Player continuations as an independent component, in order to restrict Opponent behavior. By shuffling the continuation structure of γ with the stack σ, we could reconstruct a stack for the whole interaction between Player and Opponent.

10.2. **Well-Bracketed Traces.** The goal is now to characterize the set of traces generated by a stacked configuration $\boldsymbol{F}$ (on the well-bracketed LTS) as the subset of the traces generated by $\widehat{\boldsymbol{F}}$ (on the alternating LTS) that satisfies the following condition of *well-bracketing*.

Considering a trace $s_1\ \mu_1\ s_2\ \mu_2\ s_3$, with μ_2 an answer of the shape $p(x)$ or $\bar{p}(x)$ and μ_1 the action that introduces the continuation name p, then we say that μ_1 is *answered* by μ_2. Since only questions can introduce continuation names, μ_1 is indeed a question, and it must be of the opposite polarity as μ_2. Well-bracketing corresponds to the fact that questions must be answered in the last-in first-out order they previously appear in a trace. We formalize this idea using a pushdown system, defined as an LTS whose configurations are stacks of distinct continuation names with transition relation defined as:

$$
\text{(PQ) } \sigma \xrightarrow{\bar{x}(y,p)} p :: \sigma \qquad \text{(OQ) } \sigma \xrightarrow{x(y,p)} \sigma \qquad \text{(PA) } p :: \sigma \xrightarrow{\bar{p}(x)} \sigma \qquad \text{(OA) } p :: \sigma \xrightarrow{p(x)} \sigma
$$

Definition 10.1. Taking a stack σ, a trace s is said to be *well-bracketed* with respect to σ when there exists another stack σ' such that $\sigma \stackrel{s}{\Rightarrow} \sigma'$.

For example $c(w)\bar{x}(y,c')c'(z)\bar{c''}(w)$ is well-bracketed with respect to $c :: []$. Notice that we do not keep track of the Opponent continuation names in this definition. Indeed, Proponent behaviour is automatically well-bracketed, since running terms are taken only in the pure λ-calculus.

Lemma 10.2. *Taking $\boldsymbol{F}$ a configuration whose stack is σ, then $\boldsymbol{F} \stackrel{s}{\Longrightarrow}_{\mathtt{wb}} \boldsymbol{F}'$ iff $\widehat{\boldsymbol{F}} \stackrel{s}{\Longrightarrow}_{\mathtt{a}} \widehat{\boldsymbol{F}}'$ and s is well-bracketed with respect to σ.*

Proof. Suppose that $\boldsymbol{F} \stackrel{s}{\Longrightarrow}_{\mathtt{wb}} \boldsymbol{F}'$, then writing σ' for the stack of $\boldsymbol{F}'$, we prove by a direct induction on s that $\sigma \stackrel{s}{\Rightarrow} \sigma'$.

In the reverse direction, suppose that $\widehat{\boldsymbol{F}} \stackrel{s}{\Longrightarrow}_{\mathtt{a}} F'$ and $\sigma \stackrel{s}{\Rightarrow} \sigma'$, we consider the stacked configuration $\boldsymbol{F}'$ built from F' using σ'. Then we prove by a direct induction on s that $\boldsymbol{F} \stackrel{s}{\Longrightarrow}_{\mathtt{wb}} \boldsymbol{F}'$. □

10.3. **Relationship between Alternating and Well-Bracketing OGS.** In Section 7.5, we have related the denotation of λ-terms given by the C-OGS and A-OGS LTS. We now establish a similar connection between A-OGS and WB-OGS. This is done by adapting the proof of Lemma 7.23. To do so, we first adapt the definition of triggerable singleton subconfigurations.

Definition 10.3. A singleton A-OGS configuration F is a *triggerable* subconfiguration of a stacked configuration $\boldsymbol{F}$ when:

- either $\boldsymbol{F}$ is active of the shape $\langle M, p, \sigma, \gamma, \phi\rangle$, and $F = \langle M, p, \varepsilon, \phi\backslash\mathrm{dom}(\gamma)\rangle$;
- or $\boldsymbol{F}$ is passive of the shape $\langle \sigma, \gamma, \phi\rangle$, and F is:
  - either of the shape $\langle [x \mapsto V], \phi\backslash\mathrm{dom}(\gamma) \cup \{x\}\rangle$ with $[x \mapsto V] \subseteq \gamma$;
  - or of the shape $\langle [p \mapsto (E, q)], \phi\backslash\mathrm{dom}(\gamma) \cup \{p\}\rangle$ with $[p \mapsto (E, q)] \subseteq \gamma$, when p is the top element of σ.

So $\langle [p \mapsto (E, q)], \phi\rangle$ is not triggerable when p is not the top element of the stack, since it cannot be directly triggered by an OA action of the WB-OGS LTS (while it can be by the A-OGS LTS).

We recall that we only consider well-formed relations, as introduced in Definition 7.14. Therefore, whenever two configurations F, G appear as a pair in a relation (i.e., a behavioural relation) it is implicitly assumed that they are support-equivalent, written $\mathtt{se}(F, G)$. For relations on stacked configurations, $\mathtt{se}(\boldsymbol{F}, \boldsymbol{G})$ also enforces that $\boldsymbol{F}, \boldsymbol{G}$ have the same stack.

Lemma 10.4. *If $\boldsymbol{F}, \boldsymbol{G}$ are singleton stacked configurations and $\boldsymbol{F} \approx_{\mathtt{wb}} \boldsymbol{G}$, then also $\widehat{\boldsymbol{F}} \approx_{\mathtt{a}} \widehat{\boldsymbol{G}}$.*

Proof. Let $\mathcal{R}$ be the relation on A-OGS singleton configurations defined as $F \ \mathcal{R} \ G$ when $\mathtt{se}(F, G)$ and there exist stacked configurations $\boldsymbol{F}, \boldsymbol{G}$ with $\boldsymbol{F} \approx_{\mathtt{wb}} \boldsymbol{G}$ such that F and G are triggerable singleton configurations respectively of $\boldsymbol{F}$ and $\boldsymbol{G}$.

First notice that if $\boldsymbol{F}, \boldsymbol{G}$ are stacked singleton configurations and $\boldsymbol{F} \approx_{\mathtt{wb}} \boldsymbol{G}$, we directly get that $\widehat{\boldsymbol{F}} \ \mathcal{R} \ \widehat{\boldsymbol{G}}$ since $\widehat{\boldsymbol{F}}$ and $\widehat{\boldsymbol{G}}$ are triggerable subconfigurations of $\boldsymbol{F}$ and $\boldsymbol{G}$ respectively.

We now show that $\mathcal{R}$ is a bisimulation up-to composition. We take $F \ \mathcal{R} \ G$, such that there exists $\boldsymbol{F}, \boldsymbol{G}$ as stated by the definition of $\mathcal{R}$. Suppose

$$F \xrightarrow{\mu}_{\mathtt{a}} F_1 \tag{10.1}$$

in A-OGS. First, we suppose that F, G are both passive, so that the transition is either OA or OQ. We assume it is OA, since it is the hardest case, Opponent answers being constrained in WB-OGS. We thus have, for some p, V, x, y

$$\begin{array}{rcl} F & = & \langle \gamma_1, \phi \rangle \text{ with } \gamma_1 = [p \mapsto (E, q)] \\ \mu & = & p(x) \\ F_1 & = & \langle E[x], q, \varepsilon, \phi \uplus \{x\} \rangle \end{array}$$

Moreover, we have $G = \langle \delta_1, \phi \rangle$ with $\delta_1 = [p \mapsto (E', q)]$ for some evaluation context E'. Since F, G are triggerable respectively from $\boldsymbol{F}, \boldsymbol{G}$, we get that both $\boldsymbol{F}, \boldsymbol{G}$ are passive and that the top element of the stack of $\boldsymbol{F}$ is p. From $\boldsymbol{F} \approx_{\mathtt{wb}} \boldsymbol{G}$, we get that $\mathtt{se}(\boldsymbol{F}, \boldsymbol{G})$, so there exists a stack σ and two environments γ, δ sharing the same domain such that:

$$\begin{array}{rcl} \boldsymbol{F} & = & \langle p :: \sigma, \gamma_1 \cdot \gamma, \phi \uplus \mathrm{dom}(\gamma) \rangle \\ \boldsymbol{G} & = & \langle p :: \sigma, \delta_1 \cdot \delta, \phi \uplus \mathrm{dom}(\delta) \rangle \end{array}$$

We have $\boldsymbol{F} \xrightarrow{\mu}_{\mathtt{wb}} \langle E[x], q, \sigma, \gamma, \phi \uplus \mathrm{dom}(\gamma) \uplus \{x\} \rangle \stackrel{\text{def}}{=} \boldsymbol{F'}$. Since $\boldsymbol{F} \approx_{\mathtt{wb}} \boldsymbol{G}$,

$$\boldsymbol{G} \xrightarrow{\mu}_{\mathtt{wb}} \langle E'[x], q, \sigma, \delta, \phi \uplus \mathrm{dom}(\delta) \uplus \{x\} \rangle \stackrel{\text{def}}{=} \boldsymbol{G'}$$

and $\boldsymbol{F'} \approx_{\mathtt{wb}} \boldsymbol{G'}$. Let $G_1 = \langle E'[x], q, \delta_1, \phi \rangle$, we also have $G \xrightarrow{\mu}_{\mathtt{a}} G_1$. This is sufficient to match (10.1), as $F_1 \, \mathcal{R}^{|\star} \, G_1$. Indeed we have:

- $\langle \gamma_1, \phi \uplus \{y, p\} \rangle \mathcal{R} \langle \delta_1, \phi \uplus \{y, p\} \rangle$ since $F \, \mathcal{R} \, G$;
- $\langle E[x], q, \phi \uplus \{x\} \rangle \mathcal{R} \langle E'[x], q, \phi \uplus \{x\} \rangle$ since they are triggerable subconfigurations respectively of $\boldsymbol{F'}$ and $\boldsymbol{G'}$.

Now suppose that F, G are active, then the proof is similar to the one for Lemma 7.23. □

Remark 10.5. In the proof of Lemma 10.4, we crucially use the fact that the interaction on singleton configurations is always well-bracketed, as the stack can have at most one element. Following Remark 7.20, this corresponds to the fact that in game semantics, a P-view is always well-bracketed for Opponent.

10.4. **Complete Trace Equivalence.** To relate trace denotation to contextual equivalence, it is important to be able to represent terminating interactions between a term and a context. To do so, we consider the notion of *complete trace*, that has been introduced in game semantics [AM97] to obtain full-abstraction results for languages with mutable store. A trace is said to be complete when all its questions are answered. The notion of complete trace is normally considered only for well-bracketed traces, we generalize it here to more general traces, and with configurations having already Proponent continuation names.

Definition 10.6. A trace s is *complete for an* A-OGS *(resp.* C-OGS*) configuration* F if $F \stackrel{s}{\Longrightarrow}_{\mathtt{a}}$ (resp. $F \stackrel{s}{\Longrightarrow}_{\mathtt{c}}$) and the subject names of the unjustified answers in s are exactly the Player continuation names of F. This definition is extended to a stacked configuration $\boldsymbol{F}$ with σ its stack, by asking that $\boldsymbol{F} \stackrel{s}{\Longrightarrow}_{\mathtt{wb}}$ and $\sigma \stackrel{s}{\Rightarrow} []$. Then we say that a configuration is *terminating* if it has at least one complete trace.

So a trace s is complete for F when it has no unanswered questions, and furthermore s has also answers to all the pending questions of F.

Definition 10.7. We write $\boldsymbol{F}_1 \eqsim^{\mathrm{c}}_{\mathrm{wb}} \boldsymbol{F}_2$ if $\mathtt{se}(\boldsymbol{F}_1, \boldsymbol{F}_2)$ and $\boldsymbol{F}_1, \boldsymbol{F}_2$ have the same set of complete traces in WB-OGS. Similarly, we write $F_1 \eqsim^{\mathrm{c}}_{\mathrm{a}} F_2$ (resp. $F_1 \eqsim^{\mathrm{c}}_{\mathrm{c}} F_2$) if $\mathtt{se}(F_1, F_2)$ and F_1, F_2 have the same sets of complete traces in A-OGS (resp. C-OGS).

We now characterize configurations that are reached by complete traces.

Definition 10.8. An A-OGS or stacked configuration is *strongly passive* if it is passive and its continuation structure is empty. For C-OGS, we also ask that its running term be empty.

This means, for stacked configurations, that the stack is empty, and the configuration is of form

$$\langle [], [x_1 \mapsto V_1] \cdot \ldots \cdot [x_n \mapsto V_n], \phi \rangle \qquad n \geq 0$$

so that indeed we can consider the interaction terminated, since Opponent cannot answer anymore, even if it can still interrogate one of the x_i.

Traces that correspond to reduction to strongly passive configurations are complete traces, a result that holds for WB-OGS, A-OGS and C-OGS.

Lemma 10.9. *Taking $\boldsymbol{F}, \boldsymbol{F}'$ two stacked configurations and s a trace such that $\boldsymbol{F} \overset{s}{\Longrightarrow}_{\mathtt{wb}} \boldsymbol{F}'$. Then s is complete for $\boldsymbol{F}$ iff $\boldsymbol{F}'$ is strongly passive.*

Proof. Writing σ, σ' for the stack respectively of $\boldsymbol{F}, \boldsymbol{F}'$, we get from Lemma 10.2 that $\sigma \overset{s}{\Rightarrow} \sigma'$. By definition, s is complete for $\boldsymbol{F}$ iff σ' is empty, and $\boldsymbol{F}'$ is strongly passive iff σ' is empty. □

Lemma 10.10. *Taking F, F' two* A-OGS *(resp.* C-OGS*) configurations and s a trace s.t. $F \overset{s}{\Longrightarrow}_{\mathtt{a}} F'$ (resp. $F \overset{s}{\Longrightarrow}_{\mathtt{c}} F'$). Then s is complete for F iff F' is strongly passive.*

Proof. Suppose that s is complete for F. All Player questions of s having been answered during the interaction, the set of Player continuation names of F' is included in the one of F. Moreover, s contains answers for all Player continuation names of F so that no Player continuation name of F' is a Player continuation name of F. Therefore, its set of continuation names is empty, so that F' is strongly passive.

Reciprocally, if F' is passive, then its set of continuation names is empty. Thus, s cannot have pending questions and must have answered all the questions of F. So s is complete for F. □

Complete well-bracketed trace equivalence corresponds to contextual equivalence for a λ-calculus with store, as proved in [AM97] for a compositional game model of Idealized Algol, and in [Lai07] for an operational game model of RefML.

11. Tensor Product of OGS configurations

Following the introduction of the tensor product of configurations for C-OGS in Definition 7.11, we introduce a similar definition for the Alternating and Well-Bracketed LTS. We also prove results relating the (complete) traces generated by $F \otimes G$ with the interleaving of traces generated by F and G.

11.1. Interleaving of complete traces for C-OGS.

We first state the variant of Lemma 7.13 for complete traces. So given a complete trace for a configuration, the projections of the trace on sub-configurations are also complete, as proven in the following lemma.

Lemma 11.1. *Suppose*

- $F = F_1 \otimes F_2$;
- $F \overset{s}{\Longrightarrow}_{\mathtt{c}} F'$ *and s is a complete trace for F.*

Then for both $i \in \{1, 2\}$, there exists F_i' and a complete trace s_i for F_i such that

(1) $F_i \overset{s_i}{\Longrightarrow}_{\mathtt{c}} F_i'$;
(2) $s \in \mathtt{inter}(s_1, s_2)$;
(3) $F' = F_1' \otimes F_2'$.

Proof. From Lemma 7.13(1) we get the existence of configurations F_i' and traces s_i such that the three items above are verified. From Lemma 10.10, we get that F' is strongly passive, so that both F_1', F_2' are strongly passive as well. Thus applying Lemma 10.10 in the other direction, we get that both s_1, s_2 are complete. □

Lemma 11.2. *Suppose*

(1) $F = F_1 \otimes F_2$*;*

(2) $F_i \stackrel{s_i}{\Longrightarrow}_{\mathtt{c}} F_i'$ *where* s_i *is a complete trace for* F_i*,* $i = 1, 2$*.*

Then there is a trace $s \in \mathtt{inter}(s_1, s_2)$ *complete for* F *such that* $F \stackrel{s}{\Longrightarrow}_{\mathtt{c}} F_1' \otimes F_2'$*.*

Proof. The existence of s such that $F \stackrel{s}{\Longrightarrow}_{\mathtt{c}} F_1' \otimes F_2'$ follows from Lemma 7.13(2). Since s is the interleaving of two complete traces respectively from F_1 and F_2, we get that it is complete for $F = F_1 \otimes F_2$. □

11.2. A-OGS**.**

Definition 11.3. Taking F_1, F_2 two compatible A-OGS configurations, with γ_i, ϕ_i their respective environment and set of names, $F_1 \otimes F_2$ is defined as:

- $\langle M, p, \gamma_1 \cdot \gamma_2, \phi_1 \cup \phi_2 \rangle$ when one of F_1, F_2 is active, with M, p its toplevel term and continuation name;
- $\langle \gamma_1 \cdot \gamma_2, \phi_1 \cup \phi_2 \rangle$ when F_1, F_2 are passive.

We can then state the adaptation of Lemma 7.13 to A-OGS. We write $\mathtt{inter}_{\mathtt{a}}(s_1, s_2)$ for the subset of $\mathtt{inter}(s_1, s_2)$ formed by *alternating* traces, with the extra condition that if one of s_1, s_2 starts with an output, then the resulting interleaved trace must also start with an output.

Lemma 11.4. *Suppose* F_1, F_2 *are compatible* A-OGS *configurations, and at most one of the two is active. Then the set of traces generated by* $F_1 \otimes F_2$ *is equal to the set of interleavings* $\mathtt{inter}_{\mathtt{a}}(s_1, s_2)$*, with* $F_1 \stackrel{s_1}{\Longrightarrow}_{\mathtt{a}}$ *and* $F_2 \stackrel{s_2}{\Longrightarrow}_{\mathtt{a}}$*. That is:*

(1) if $F_1 \otimes F_2 \stackrel{s}{\Longrightarrow}_{\mathtt{a}} F$*, then, for* $i = 1, 2$*, there are traces* s_i *such that* $F_i \stackrel{s_i}{\Longrightarrow}_{\mathtt{a}} F_i'$*, and* $s \in \mathtt{inter}_{\mathtt{a}}(s_1, s_2)$*, and* $F = F_1' \otimes F_2'$*;*

(2) conversely, if for $i = 1, 2$*, we have* $F_i \stackrel{s_i}{\Longrightarrow}_{\mathtt{a}} F_i'$*, and* $s \in \mathtt{inter}_{\mathtt{a}}(s_1, s_2)$*, then* $F_1 \otimes F_2 \stackrel{s}{\Longrightarrow}_{\mathtt{a}} F_1' \otimes F_2'$*.*

The statement of (2) above can be deduced from the corresponding result for C-OGS, namely Lemma 7.13, combined with the characterization of traces generated by the A-OGS LTS as being the *alternating* traces of C-OGS LTS. However, such a proof does not work for (1), since we cannot deduce directly that s_1 and s_2 are alternating. To prove this direction, we use the correspondence between A-OGS and $\Longrightarrow_{\mathtt{o}\pi}$ the output-prioritised LTS established in Corollary 6.8, combined with Lemma 11.6 below, whose proof uses the following lemma.

Lemma 11.5. *Suppose* $P \mid Q \stackrel{\mu}{\Longrightarrow}_{\mathtt{o}\pi} P' \mid Q$ *and* $P \stackrel{\mu}{\Longrightarrow} P'$*. Then also* $P \stackrel{\mu}{\Longrightarrow}_{\mathtt{o}\pi} P'$*.*

Proof. If $P \stackrel{\mu}{\Longrightarrow}_{\mathtt{o}\pi} P'$ were not possible, because of the constraint on input components in the opLTS with respect to the ordinary LTS, then also $P \mid Q \stackrel{\mu}{\Longrightarrow}_{\mathtt{o}\pi} P' \mid Q$ would not be possible (if P is not input reactive, then $P \mid Q$ cannot be so either). □

Lemma 11.6. *Suppose* F_1, F_2 *are compatible configurations, and at most one of the two is active.*

(1) If $\mathcal{O}[\![F_1]\!] \mid \mathcal{O}[\![F_2]\!] \stackrel{s}{\Longrightarrow}_{\mathtt{o}\pi} P_1 \mid P_2$*, then, for* $i = 1, 2$*, there are traces* s_i *such that* $\mathcal{O}[\![F_i]\!] \stackrel{s_i}{\Longrightarrow}_{\mathtt{o}\pi} P_i$*, and* $s \in \mathtt{inter}_{\mathtt{a}}(s_1, s_2)$*;*

(2) Conversely, if for $i = 1, 2$*, we have* $\mathcal{O}[\![F_i]\!] \stackrel{s_i}{\Longrightarrow}_{\mathtt{o}\pi} P_i$*,* $s \in \mathtt{inter}_{\mathtt{a}}(s_1, s_2)$*, then also* $\mathcal{O}[\![F_1]\!] \mid \mathcal{O}[\![F_2]\!] \stackrel{s}{\Longrightarrow}_{\mathtt{o}\pi} P_1 \mid P_2$*.*

Proof. Proof of (1): the assertion follows by reasoning as in Lemma 7.12. Processes $\mathcal{O}[\![F_1]\!]$ and $\mathcal{O}[\![F_2]\!]$ cannot interact and, by Lemma 7.9, the property is invariant under transitions. Thus each action (visible or silent) that contributes to the given trace s is derived using either rule `parL` or rule `parR` on the outermost parallel composition. The former case represents an action (in the ordinary LTS) from the left process, the latter case an action from the right process. In either case, the action can be lifted to the opLTS using Lemma 11.5. By iterating the reasoning on each action of the trace s we obtain traces s_i $(i = 1, 2)$ with $\mathcal{O}[\![F_i]\!] \stackrel{s_i}{\Longrightarrow}_{\mathtt{o}\pi} P_i$.

For (2): From $\mathcal{O}[\![F_i]\!] \stackrel{s_i}{\Longrightarrow}_{\mathtt{o}\pi} P_i$, we deduce that $\mathcal{O}[\![F_i]\!] \stackrel{s_i}{\Longrightarrow}_{\pi} P_i$. So applying Lemma 7.12, we get that $\mathcal{O}[\![F_1]\!] \mid \mathcal{O}[\![F_2]\!] \stackrel{s}{\Longrightarrow}_{\pi} P_1 \mid P_2$. Moreover, from $\mathcal{O}[\![F_i]\!] \stackrel{s_i}{\Longrightarrow}_{\mathtt{o}\pi} P_i$ we also deduce that each s_i is alternating. Since at most one of the two F_1, F_2 is active, we deduce that at most one of s_1, s_2 is P-starting. We have that s is alternating since $s \in \mathtt{inter}_{\mathtt{a}}(s_1, s_2)$, and active iff one of the composing traces is P-starting. This means that we can assume that when an input transition of s is performed the source process is input reactive. □

We now state the corresponding lemmas for complete traces, whose proofs are similar to Lemma 11.1 and Lemma 11.2.

Lemma 11.7. *Suppose*

- $F = F_1 \otimes F_2$;
- s *is a complete trace for* F, *such that* $F \stackrel{s}{\Longrightarrow}_{\mathtt{a}} F'$.

Then for both $i \in \{1, 2\}$, *there exists* F_i' *and a complete trace* s_i *for* F_i *such that*

(1) $F_i \stackrel{s_i}{\Longrightarrow}_{\mathtt{a}} F_i'$;
(2) $s \in \mathtt{inter}_{\mathtt{a}}(s_1, s_2)$;
(3) $F' = F_1' \otimes F_2'$.

Lemma 11.8. *Suppose*

(1) $F = F_1 \otimes F_2$;
(2) s_i *a complete trace for* F_i, *such that* $F_i \stackrel{s_i}{\Longrightarrow}_{\mathtt{a}} F_i'$, $i = 1, 2$.

Then there is $s \in \mathtt{inter}_{\mathtt{a}}(s_1, s_2)$ *complete for* F *such that* $F \stackrel{s}{\Longrightarrow}_{\mathtt{a}} F_1' \otimes F_2'$.

Next, we show in Lemma 11.11 a decomposition property for complete trace equivalence $\eqsim^{\mathrm{c}}_{\mathtt{a}}$ with respect to $\otimes$ on configurations. To prove it, we rely on the following lemmas that show that the action of a configuration is determined by a specific singleton component, and it is the action itself that allows us to determine such a component.

Lemma 11.9. *Suppose* $F \stackrel{\mu}{\longrightarrow}_{\mathtt{a}} F'$. *Then there is a singleton* F_1, *and some* F_1', F_2, F_2' *such that:*

(1) $F = F_1 \otimes F_2$;
(2) $F_1 \stackrel{\mu}{\longrightarrow}_{\mathtt{a}} F_1'$;
(3) $F' = F_1' \otimes F_2$.

Moreover, the P-supports and the structure of continuations of F_1 *and* F_1' *are determined by* μ; *that is, if* $\mathtt{se}(F, G)$ *and* $G \stackrel{\mu}{\longrightarrow}_{\mathtt{a}} G'$, *then for the singleton* G_1, *and the configurations* G_1', G_2, G_2' *such that* $G = G_1 \otimes G_2$, $G_1 \stackrel{\mu}{\longrightarrow}_{\mathtt{a}} G_1'$, *and* $G' = G_1' \otimes G_2$ *we have:* $\mathtt{se}(F_1, G_1)$, $\mathtt{se}(F_2, G_2)$, *and* $\mathtt{se}(F_1', G_1')$.

Proof. By a simple case analysis on the form of μ. □

The next lemma uses the previous result to show that the decomposition of a trace is uniform.

Lemma 11.10. *Suppose*

(1) $F \stackrel{s}{\Longrightarrow}_{\mathtt{a}} F'$ *and* $G \stackrel{s}{\Longrightarrow}_{\mathtt{a}} G'$*;*
(2) $F = F_1 \otimes F_2$ *and* $G = G_1 \otimes G_2$*;*
(3) $\mathtt{se}(F, G)$ *and* $\mathtt{se}(F_i, G_i)$*, for* $i = 1, 2$*;*
(4) $F_i \stackrel{s_i}{\Longrightarrow}_{\mathtt{a}} F_i'$*,* $i = 1, 2$*, with* $s \in \mathtt{inter}_{\mathtt{a}}(s_1, s_2)$ *and* $F' = F_1' \otimes F_2'$*.*

Then we also have $G_i \stackrel{s_i}{\Longrightarrow}_{\mathtt{a}} G_i'$*, for some* G_i'*,* $i = 1, 2$ *with* $G' = G_1' \otimes G_2'$*.*

Proof. Using Lemma 11.9, and proceeding by induction on the length of the trace. □

Lemma 11.11. *Suppose*

(1) F *is terminating;*
(2) $F \simeq^{\mathrm{c}}_{\mathrm{a}} G$*;*
(3) $F = F_1 \otimes F_2$*, and* $G = G_1 \otimes G_2$*;*
(4) $\mathtt{se}(F_1, F_2)$ *and* $\mathtt{se}(G_1, G_2)$*.*

Then $F_i \simeq^{\mathrm{c}}_{\mathrm{a}} G_i$*,* $i = 1, 2$*.*

Proof. From the fact that F is terminating and $F \simeq^{\mathrm{c}}_{\mathrm{a}} G$, we deduce the existence of a complete trace t for both F and G. Applying Lemma 11.7, we deduce for both $i = 1, 2$ the existence of a complete trace t_i for both F_i and G_i.

Taking $j = 1, 2$ and s_j a complete trace for F_j, from Lemma 11.8 we deduce the existence of a complete trace s for F, with:

- either $s \in \mathtt{inter}_{\mathtt{a}}(s_1, t_2)$ if $j = 1$;
- or $s \in \mathtt{inter}_{\mathtt{a}}(t_1, s_2)$ if $j = 2$.

From $F \simeq^{\mathrm{c}}_{\mathrm{a}} G$, we deduce that s is a complete trace for G, so that applying Lemma 11.10, we get that s_j is a complete trace for G_j. □

As explained in Remark 7.19, this previous lemma fails for ordinary (not necessarily complete) trace equivalence. What makes it true for complete trace equivalence is the additional hypothesis that F is terminating. In the other direction, a form of compositionality for $\simeq^{\mathrm{c}}_{\mathrm{a}}$ with respect to $\otimes$ also holds.

Lemma 11.12. *Suppose*

(1) $F = F_1 \otimes F_2$*, and* $G = G_1 \otimes G_2$*;*
(2) $F_i \simeq^{\mathrm{c}}_{\mathrm{a}} G_i$*,* $i = 1, 2$*.*

Then $F \simeq^{\mathrm{c}}_{\mathrm{a}} G$*.*

Proof. First note that $\mathtt{se}(F, G)$ holds, as $F_i \simeq^{\mathrm{c}}_{\mathrm{a}} G_i$, $i = 1, 2$ and therefore $\mathtt{se}(F_i, G_i)$.

Suppose $F \stackrel{s}{\Longrightarrow}_{\mathtt{a}} F'$ where s is a complete trace for F. By Lemma 11.7, we have $F_i \stackrel{s_i}{\Longrightarrow}_{\mathtt{a}} F_i'$, where s_i is a complete trace, $i = 1, 2$, with $s \in \mathtt{inter}_{\mathtt{a}}(s_1, s_2)$ and $F' = F_1' \otimes F_2'$. Since $F_i \simeq^{\mathrm{c}}_{\mathrm{a}} G_i$, $i = 1, 2$, we also have $G_i \stackrel{s_i}{\Longrightarrow}_{\mathtt{a}} G_i'$. By Lemma 11.4(2) we infer that $G \stackrel{s}{\Longrightarrow}_{\mathtt{a}}$. Moreover, s is complete for G, since G has the same continuation structure as F. □

11.3. WB-OGS. We now introduce the tensor product of stacked configurations. Compared to the tensor product of C-OGS or A-OGS configurations, there are multiple ways in WB-OGS to tensor two stacked configurations, so that the definition of the tensor product $\otimes_\sigma$ has to be indexed by a stack of continuation names σ. Indeed, when tensoring two stacked configurations, we can choose how to interleave their stack.

First, we adapt the notion of compatible configuration to stacked ones.

Definition 11.13. Two stacked configurations $\boldsymbol{F}_1, \boldsymbol{F}_2$ are compatible when their stacks are disjoint and their corresponding A-OGS configurations $\widehat{\boldsymbol{F}_1}, \widehat{\boldsymbol{F}_2}$ are compatible.

Definition 11.14. Let $\boldsymbol{F}_1$ and $\boldsymbol{F}_2$ be two compatible stacked configurations with at most one active, and σ_1, σ_2 their respective stacks. Taking σ an interleaving of σ_1, σ_2, we write $\boldsymbol{F}_1 \otimes_\sigma \boldsymbol{F}_2$ for the following stacked configuration:

- $\langle M, p, \sigma, \gamma_1 \cdot \gamma_2, \phi_1 \cup \phi_2 \rangle$ when one of the two configurations is active, with M, p its toplevel term and continuation name;
- $\langle \sigma, \gamma_1 \cdot \gamma_2, \phi_1 \cup \phi_2 \rangle$ when the two configurations are passive.

with γ_i, ϕ_i the environment and the set of names in $\boldsymbol{F}_i$.

Taking s_1, s_2 two traces well-bracketed with respect to respectively σ_1 and σ_2, and σ a valid interleaving of σ_1, σ_2; we write $\mathtt{inter}^\sigma_{\mathtt{wb}}(s_1, s_2)$ for the subset of $\mathtt{inter}_{\mathtt{a}}(s_1, s_2)$ formed by well-bracketed traces with respect to σ.

Lemma 11.15. *Suppose $\boldsymbol{F}_1, \boldsymbol{F}_2$ are compatible stacked configurations and σ is an interleaving of their respective stack, such that $\boldsymbol{F}_1 \otimes_\sigma \boldsymbol{F}_2 \overset{s}{\Longrightarrow}_{\mathtt{wb}} \boldsymbol{G}$. Taking s_1, s_2 two traces, $s \in \mathtt{inter}^\sigma_{\mathtt{wb}}(s_1, s_2)$, and G_1, G_2 two compatible* A-OGS *configurations we suppose that $\widehat{\boldsymbol{F}_1} \overset{s_1}{\Longrightarrow}_{\mathtt{a}} G_1$ and $\widehat{\boldsymbol{F}_2} \overset{s_2}{\Longrightarrow}_{\mathtt{a}} G_2$. Then there exists $\boldsymbol{G}_1, \boldsymbol{G}_2$ two compatible stacked configurations and σ' an interleaving of their respective stack such that $\boldsymbol{G} = \boldsymbol{G}_1 \otimes_{\sigma'} \boldsymbol{G}_2$ and for all $i = 1, 2$, $\widehat{\boldsymbol{G}_i} = G_i$ and $\boldsymbol{F}_i \overset{s_i}{\Longrightarrow}_{\mathtt{wb}} \boldsymbol{G}_i$.*

Proof. We reason by induction on s. If s is empty, then $\boldsymbol{G} = \boldsymbol{F}_1 \otimes_\sigma \boldsymbol{F}_2$. Then both s_1, s_2 are empty as well, so that $G_1 = \widehat{\boldsymbol{F}_1}$ and $G_2 = \widehat{\boldsymbol{F}_2}$. So we define $\boldsymbol{G}_1$ as $\boldsymbol{F}_1$ and $\boldsymbol{G}_2$ as $\boldsymbol{G}_2$.

If s is of the shape μt, then either μ is the first action of s_1, or the first action of s_2. Suppose it is the first action of s_1, then s_1 can be written as μt_1. Then there exists F_1' such that $\widehat{\boldsymbol{F}_1} \xrightarrow{\mu}_{\mathtt{a}} F_1' \overset{t_1}{\Longrightarrow}_{\mathtt{a}} G_1$. Moreover, there exists $\boldsymbol{F}'$ such that $\boldsymbol{F}_1 \otimes_\sigma \boldsymbol{F}_2 \xrightarrow{\mu}_{\mathtt{wb}} \boldsymbol{F}' \overset{s}{\Longrightarrow}_{\mathtt{wb}} \boldsymbol{G}$.

We prove by a case analysis over μ that there exists $\boldsymbol{F}_1'$ and σ'' such that $\boldsymbol{F}' = \boldsymbol{F}_1' \otimes_{\sigma''} \boldsymbol{F}_2$, $\widehat{\boldsymbol{F}_1'} = F_1'$ and that $t \in \mathtt{inter}^{\sigma''}_{\mathtt{wb}}(t_1, s_2)$.

- If μ is an Opponent answer $p(x)$ then $\boldsymbol{F}_1$ is of the shape $\langle \sigma_1, \gamma \cdot [p \mapsto (E, q)], \phi \rangle$, and σ is of the shape $p :: \sigma''$. So σ_1 can be written as $p :: \sigma_1'$ since σ is an interleaving of σ_1 and σ_2, and that p must belong to σ_1. Writing the stack of $\boldsymbol{F}_2$ as σ_2, we get that σ'' is an interleaving of σ_1' and σ_2. Defining $\boldsymbol{F}_1'$ as $\langle \sigma_1', \gamma, \phi \cup \{x\} \rangle$, we then get the wanted properties.
- If μ is a Player question $\bar{x}(y, q)$ then $\boldsymbol{F}_1$ is of the shape $\langle E[xV], p, \sigma_1, \gamma, \phi \rangle$, so that F_1' is equal to $\langle \gamma \cdot [y \mapsto V] \cdot [q \mapsto (E, p)], \phi \cup \{y, q\} \rangle$. Moreover, the stack of $\boldsymbol{F}'$ is equal to $q :: \sigma$. Defining $\boldsymbol{F}_1'$ as $\langle q :: \sigma_1, \gamma \cdot [y \mapsto V] \cdot [q \mapsto (E, p)], \phi \cup \{x\} \rangle$ and σ'' as $q :: \sigma$, we then get the wanted properties.
- If μ is an Opponent question or a Player answer, then we can simply take σ' equal to σ, and $\boldsymbol{F}_1'$ as F_1' equipped with the same stack as $\boldsymbol{F}_1$.

We then conclude using the induction hypothesis over t. A similar reasoning applies when μ is the first action of s_2. □

Lemma 11.16. *Suppose $\boldsymbol{F}_1, \boldsymbol{F}_2$ are compatible stacked configurations and σ is an interleaving of their respective stack. Then the set of traces generated by $\boldsymbol{F}_1 \otimes_\sigma \boldsymbol{F}_2$ is equal to the set of interleavings $\mathtt{inter}^\sigma_{\mathtt{wb}}(s_1, s_2)$ well-bracketed with respect to σ, with $\boldsymbol{F}_1 \overset{s_1}{\Longrightarrow}_{\mathtt{wb}}$ and $\boldsymbol{F}_2 \overset{s_2}{\Longrightarrow}_{\mathtt{wb}}$. Moreover, s is complete if and only if s_1, s_2 are. That is:*

(1) *If* $\boldsymbol{F}_1 \otimes_\sigma \boldsymbol{F}_2 \overset{s}{\Longrightarrow}_{\mathtt{wb}} \boldsymbol{G}$*, then, for* $i = 1, 2$*, there are traces* s_i *such that* $\boldsymbol{F}_i \overset{s_i}{\Longrightarrow}_{\mathtt{wb}} \boldsymbol{G}_i$*, and* $s \in \mathtt{inter}^\sigma_{\mathtt{wb}}(s_1, s_2)$*, with* $\boldsymbol{G} = \boldsymbol{G}_1 \otimes_{\sigma'} \boldsymbol{G}_2$*; and if* s *is complete for* $\boldsymbol{F}$ *then both* s_i *are complete for* $\boldsymbol{F}_i$*;*

(2) *Conversely, if for* $i = 1, 2$*, we have* $\boldsymbol{F}_i \overset{s_i}{\Longrightarrow}_{\mathtt{wb}} \boldsymbol{G}_i$*, and* $s \in \mathtt{inter}^\sigma_{\mathtt{wb}}(s_1, s_2)$*, then* $\boldsymbol{F}_1 \otimes_\sigma \boldsymbol{F}_2 \overset{s}{\Longrightarrow}_{\mathtt{wb}} \boldsymbol{G}_1 \otimes_{\sigma'} \boldsymbol{G}_2$*, with* σ' *an interleaving of the stack of* $\boldsymbol{G}_1$ *and* $\boldsymbol{G}_2$*; and if both* s_i *are complete for* $\boldsymbol{F}_i$ *then* s *is complete for* $\boldsymbol{F}$*.*

Proof. (1) Lifting the reduction to A-OGS we get that $\widehat{\boldsymbol{F}_1} \otimes_\sigma \widehat{\boldsymbol{F}_2} \overset{s}{\Longrightarrow}_{\mathtt{a}} \widehat{\boldsymbol{G}}$. Applying Lemma 11.4 we deduce that there exists G_1, G_2 two compatible A-OGS configurations such that $\widehat{\boldsymbol{F}_1} \overset{s_1}{\Longrightarrow}_{\mathtt{a}} G_1$ and $\widehat{\boldsymbol{F}_2} \overset{s_2}{\Longrightarrow}_{\mathtt{a}} G_2$. We conclude using Lemma 11.15.

(2) Lifting the reductions to A-OGS we get that $\widehat{\boldsymbol{F}_i} \overset{s_i}{\Longrightarrow}_{\mathtt{a}} \widehat{\boldsymbol{G}_i}$, $i = 1, 2$. Applying Lemma 11.4 we deduce that $\widehat{\boldsymbol{F}_1} \otimes \widehat{\boldsymbol{F}_2} \overset{s}{\Longrightarrow}_{\mathtt{a}} \widehat{\boldsymbol{G}_1} \otimes \widehat{\boldsymbol{G}_2}$. Since s is well-bracketed, from Lemma 10.2 we get that $\boldsymbol{F}_1 \otimes_\sigma \boldsymbol{F}_2 \overset{s}{\Longrightarrow}_{\mathtt{wb}} \boldsymbol{G}_1 \otimes_{\sigma'} \boldsymbol{G}_2$ for σ' an interleaving of the stack of $\boldsymbol{G}_1$ and $\boldsymbol{G}_2$.
If both s_i are complete, then from Lemma 10.9, both $\boldsymbol{G}_i$ are strongly passive, so that $\boldsymbol{G}$ is also strongly passive. Thus applying Lemma 10.9 in the other direction, we deduce that s is complete for $\boldsymbol{F}$. □

Finally, we adapt Lemma 11.11 to WB-OGS.

Lemma 11.17. *Suppose*

(1) $\boldsymbol{F}$ *is terminating;*
(2) $\boldsymbol{F} \simeq^{\mathrm{c}}_{\mathrm{wb}} \boldsymbol{G}$*;*
(3) $\boldsymbol{F} = \boldsymbol{F}_1 \otimes_\sigma \boldsymbol{F}_2$*, and* $\boldsymbol{G} = \boldsymbol{G}_1 \otimes_\sigma \boldsymbol{G}_2$*;*
(4) $\mathtt{se}(\boldsymbol{F}_1, \boldsymbol{F}_2)$ *and* $\mathtt{se}(\boldsymbol{G}_1, \boldsymbol{G}_2)$*.*

Then $\boldsymbol{F}_i \simeq^{\mathrm{c}}_{\mathrm{wb}} \boldsymbol{G}_i$*,* $i = 1, 2$*.*

Its proof is a direct adaption of the proof of Lemma 11.11, using Lemmas corresponding to Lemma 11.9 and Lemma 11.10 for WB-OGS.

12. Complete-trace equivalences coincide

In this section, we show that complete-trace equivalences in C-OGS, A-OGS, and WB-OGS coincide.

Remark 12.1. Complete trace equivalence (in all the OGS forms discussed) is strictly weaker than ordinary trace equivalence, as shown by the following terms:

$$(\lambda z.\, \Omega)(x(\lambda y.\, \Omega)) \text{ and } (\lambda z.\, \Omega)(x(\lambda y.\, y))$$

The terms are equal if only complete traces are observed, while they are not otherwise.

Theorem 12.2. *For any* $\boldsymbol{F}, \boldsymbol{G}$*, we have* $\boldsymbol{F} \simeq^{\mathrm{c}}_{\mathrm{wb}} \boldsymbol{G}$ *iff* $\widehat{\boldsymbol{F}} \simeq^{\mathrm{c}}_{\mathrm{a}} \widehat{\boldsymbol{G}}$ *iff* $\widehat{\boldsymbol{F}} \simeq^{\mathrm{c}}_{\mathrm{c}} \widehat{\boldsymbol{G}}$*.*

One direction of the theorem is easy, intuitively because a trace in WB-OGS is also a trace in A-OGS, and these, in turn, are traces in C-OGS. The problem is the converse, namely showing that $\boldsymbol{F} \simeq^{\mathrm{c}}_{\mathrm{wb}} \boldsymbol{G}$ implies $\widehat{\boldsymbol{F}} \simeq^{\mathrm{c}}_{\mathrm{a}} \widehat{\boldsymbol{G}}$, and that $\widehat{\boldsymbol{F}} \simeq^{\mathrm{c}}_{\mathrm{a}} \widehat{\boldsymbol{G}}$ implies $\widehat{\boldsymbol{F}} \simeq^{\mathrm{c}}_{\mathrm{c}} \widehat{\boldsymbol{G}}$. They are proven in the next two sections.

Remark 12.3. In earlier sections, (Section 7), we have derived another equality result for trace equivalence, namely Corollary 7.24, relating trace equivalences in the Alternating and Concurrent LTS. The structure of the proofs of that result and the one above (Theorem 12.2) are however quite different. In Corollary 7.24 we derived the result via up-to techniques; we cannot apply such techniques here because we are reasoning on trace equivalence, notably a restricted form of traces (the complete ones). On the other hand, the proof strategy here rests on decomposition properties such as Lemma 11.17 that rely on a termination property for initial configurations, which may not be assumed in Section 7.

12.1. **Relating** WB-OGS **and** A-OGS **complete traces.**

Lemma 12.4. *Taking* $\boldsymbol{F}$ *a* WB-OGS *configuration, if* $\widehat{\boldsymbol{F}}$ *has a complete trace in* A-OGS*, then* $\boldsymbol{F}$ *has a well-bracketed complete trace.*

Proof. Assume s is a complete trace for $\widehat{\boldsymbol{F}}$ in A-OGS. We proceed by induction on the length of s. If s is empty then the set of P- and O- continuation names of $\widehat{\boldsymbol{F}}$ is empty. So s is also a complete trace for $\boldsymbol{F}$ since the stack of $\boldsymbol{F}$ must then be empty.

Suppose now $s = \mu s'$ is not empty, so that there exists F' such that $\widehat{\boldsymbol{F}} \stackrel{\mu}{\Longrightarrow}_{\mathtt{a}} F' \stackrel{s'}{\Longrightarrow}_{\mathtt{a}}$. We reason by case analysis over μ:

- If μ is a question, then μ is either of the form $\bar{x}(y,p)$ or of the form $x(y,p)$. We also have $\boldsymbol{F} \stackrel{\mu}{\Longrightarrow}_{\mathtt{wb}} \boldsymbol{F}'$ with $\widehat{\boldsymbol{F}'} = F'$. Moreover s is a complete trace for F'. By induction $\boldsymbol{F}'$ has a well-bracketed complete trace, say t. Then μt is a well-bracketed complete trace for $\boldsymbol{F}$.
- Suppose now μ is an answer. There are two cases:
  - If $\boldsymbol{F}$ is active, then the answer μ must be of the form $\bar{p}(x)$, where p is the continuation name for the running term. Thus μ is also the correct answer for WB-OGS, and we can easily conclude using induction.
  - If $\boldsymbol{F}$ is passive, then μ must be of the form $p(x)$. Writing σ for the stack of $\boldsymbol{F}$, if $\sigma = p :: \sigma'$ then μ is also the correct answer for a well-bracketed trace, and as before we can conclude using induction. Otherwise, suppose $\sigma = q :: \sigma'$ with $q \neq p$ (i.e., p is not the last name in σ). This means that $\boldsymbol{F}$ has at least two singleton components, located at p and q. Take a decomposition $\boldsymbol{F} = \boldsymbol{F}_1 \otimes_\sigma \boldsymbol{F}_2$, where p and q end up in different sub-configurations. We get from Lemma 11.7 that there exists s_i complete for $\widehat{\boldsymbol{F}_i}$ in A-OGS, for $i \in \{1,2\}$, and $\mu s \in \mathtt{inter}_{\mathtt{a}}(s_1, s_2)$. By induction, there is t_i a complete well-bracketed trace for $\boldsymbol{F}_i$ in WB-OGS. By Lemma 11.16, there is $t \in \mathtt{inter}^{\sigma}_{\mathtt{wb}}(t_1, t_2)$, a complete well-bracketed trace for $\boldsymbol{F}$. □

Lemma 12.5. *Suppose* $\boldsymbol{F} \simeq^{\mathrm{c}}_{\mathrm{wb}} \boldsymbol{G}$*, then* $\widehat{\boldsymbol{F}} \simeq^{\mathrm{c}}_{\mathrm{a}} \widehat{\boldsymbol{G}}$

Proof. First, if $\boldsymbol{F}, \boldsymbol{G}$ are not terminating, i.e. they have no complete trace in WB-OGS, then from Lemma 12.4, both $\widehat{\boldsymbol{F}}, \widehat{\boldsymbol{G}}$ have no well-bracketed complete trace, thus $\widehat{\boldsymbol{F}} \simeq^{\mathrm{c}}_{\mathrm{a}} \widehat{\boldsymbol{G}}$.

Suppose now that $\boldsymbol{F}$ and $\boldsymbol{G}$ are both terminating. By Lemmas 11.17 and 11.12, it is sufficient to consider the case when $\boldsymbol{F}$ and $\boldsymbol{G}$ are singleton. Suppose there exists a complete trace s for $\widehat{\boldsymbol{F}}$, then we prove by induction over s that s is a complete trace for $\widehat{\boldsymbol{G}}$ as well. If s is empty, then from Lemma 10.10, $\widehat{\boldsymbol{F}}$ must be strongly passive. Recall that from $\boldsymbol{F} \simeq^{\mathrm{c}}_{\mathrm{wb}} \boldsymbol{G}$ we enforce that $\mathtt{se}(\boldsymbol{F}, \boldsymbol{G})$. So $\widehat{\boldsymbol{G}}$ must be strongly passive as well, so that the empty trace is complete for $\widehat{\boldsymbol{G}}$ as well.

Suppose now that $s = \mu s'$, so that $\widehat{\boldsymbol{F}} \Longrightarrow \xrightarrow{\mu}_{\mathtt{a}} F' \stackrel{s'}{\Longrightarrow}_{\mathtt{a}} F''$ with s' complete for F'. Then μ is the only possible visible transition for $\widehat{\boldsymbol{F}}$ since $\widehat{\boldsymbol{F}}$ is a singleton. So it is also the first visible transition in any well-bracketed complete trace for $\boldsymbol{F}$ (i.e., we have $\boldsymbol{F} \Longrightarrow_{\mathtt{wb}} \xrightarrow{\mu}_{\mathtt{wb}} \boldsymbol{F}'$ with $\widehat{\boldsymbol{F}'} = F'$). Indeed, if μ is an Opponent answer $p(x)$, then the stack of $\boldsymbol{F}$ must be equal to p since $\boldsymbol{F}$ is a singleton. Since $\boldsymbol{F}$ is terminating, it has thus a well-bracketed complete trace starting with μ. So does $\boldsymbol{G}$ since $\boldsymbol{F} \simeq^{\mathrm{c}}_{\mathrm{wb}} \boldsymbol{G}$, that is, we have $\boldsymbol{G} \Longrightarrow_{\mathtt{wb}} \xrightarrow{\mu}_{\mathtt{wb}} \boldsymbol{G}'$, for some $\boldsymbol{G}'$, and therefore also $\widehat{\boldsymbol{G}} \Longrightarrow_{\mathtt{a}} \xrightarrow{\mu}_{\mathtt{a}} \widehat{\boldsymbol{G}'}$. Moreover, $\boldsymbol{F}' \simeq^{\mathrm{c}}_{\mathrm{wb}} \boldsymbol{G}'$.

If $\boldsymbol{F}'$ and $\boldsymbol{G}'$ are singleton, we can now apply induction and derive that s' is also complete for $\widehat{\boldsymbol{G}'}$, and then we are done. If they are not singleton, we can consider a decomposition for them (yielding $\boldsymbol{F}'_i$ and $\boldsymbol{G}'_i$), and associated decomposition for s' in complete traces s'_i for $\widehat{\boldsymbol{F}_i}$ using Lemma 11.7. We can then apply induction on the singleton components (using Lemma 11.17), then we infer that s'_i is also a complete trace for $\widehat{\boldsymbol{G}'_i}$. We can now apply Lemma 11.4 and infer that s' is a complete trace for $\widehat{\boldsymbol{G}'}$, and then conclude that $\mu s'$ is a complete trace for $\widehat{\boldsymbol{G}}$. □

12.2. **Relating** A-OGS **and** C-OGS **complete traces.**

Lemma 12.6. *Taking F an* A-OGS *configuration, if F has a complete trace in* C-OGS*, then F has a complete trace in* A-OGS*.*

Proof. Assume s is a complete trace for F in C-OGS. We proceed by induction on the length of s. If s is empty then this is straightforward. Suppose now there exists μ s.t. $s = \mu s'$. We reason by case analysis on μ:

- If μ is a Player action, then $F \stackrel{\mu}{\Longrightarrow}_{\mathtt{a}} F'$, so we conclude using induction.
- If μ is an Opponent action, there are two cases:
  - If F is a passive configuration, then again $F \stackrel{\mu}{\Longrightarrow}_{\mathtt{a}} F'$, and we conclude using induction.
  - If F is an active configuration, then we can write F as $F_1 \otimes F_2$ since F has both an active term and at least one element in its environment, that is triggered by μ. We get from Lemma 11.1 that there exist two traces s_1, s_2 complete respectively for F_1 and F_2 in C-OGS such that $\mu s \in \mathtt{inter}(s_1, s_2)$. By induction, there exists two traces t_1, t_1 complete respectively for F_1 and F_2 in A-OGS. Finally, by Lemma 11.8, there exists a trace $t \in \mathtt{inter}_{\mathtt{a}}(t_1, t_2)$ that is complete for F in A-OGS. □

Lemma 12.7. *Suppose $F \eqsim^{\mathrm{c}}_{\mathrm{a}} G$, then $F \eqsim^{\mathrm{c}}_{\mathrm{c}} G$.*

Proof. First, if F, G are not terminating, i.e. they have no complete trace in A-OGS, then from Lemma 12.6, both F, G have no complete trace C-OGS, thus $F \eqsim^{\mathrm{c}}_{\mathrm{c}} G$.

Suppose now that F and G are both terminating in A-OGS. By Lemmas 11.11 and the adaptation of Lemma 11.12 to C-OGS, it is sufficient to consider the case where F and G are singleton. Suppose there exists a complete trace s for F in C-OGS, then we prove by induction over s that s is a complete trace for G in C-OGS as well. If s is empty, then F must be strongly passive. Recall that from $F \eqsim^{\mathrm{c}}_{\mathrm{a}} G$ we enforce that $\mathtt{se}(F, G)$. So G must also be strongly passive, so that the empty trace is a complete trace for G as well.

If s is of the shape $\mu s'$, then $F \Longrightarrow\!\!\xrightarrow{\mu}_{\mathtt{c}} F' \stackrel{s'}{\Longrightarrow}_{\mathtt{c}}$. Since F is singleton, F cannot be active with μ an Opponent action. So μ is a Player action, or an Opponent action with F passive, so that $F \Longrightarrow_{\mathtt{a}}\!\!\xrightarrow{\mu}_{\mathtt{a}} F'$. Since F is terminating in A-OGS, we get that F has a complete trace starting with μ in A-OGS. Since $F \eqsim^{\mathrm{c}}_{\mathrm{a}} G$, we also have $G \Longrightarrow_{\mathtt{a}}\!\!\xrightarrow{\mu}_{\mathtt{a}} G'$, for some G', and therefore also $G \Longrightarrow_{\mathtt{c}}\!\!\xrightarrow{\mu}_{\mathtt{c}} G'$. Moreover, $F' \eqsim^{\mathrm{c}}_{\mathrm{a}} G'$.

If F' and G' are singleton, we can now apply the induction hypothesis and derive the same complete trace s' also for G' in C-OGS, and then we are done. If they are not singleton, we can consider a decomposition for them (yielding F'_i and G'_i), and associated decomposition for s' in complete traces s'_i for F_i using Lemma 11.1. We can then apply induction on the singleton components (using Lemma 11.11), and getting that s'_i is also a trace for G'_i in C-OGS. We can now apply Lemma 11.2 and infer that s' is a complete trace for G' in C-OGS, and then conclude that $\mu s'$ is a complete trace for G in C-OGS. □

13. The call-by-name λ-calculus

The relationship between OGS and π-calculus representations of the λ-terms examined in Sections 6 and 7 is not specific to call-by-value. A similar relationship can be established for call-by-name. We summarise here the main aspects and the main results. Further details may be found in Appendix D.

We maintain terminologies and notations in the call-by-value sections. The call-by-name language is given in Figure 8. Its reduction $\rightarrow_{\mathtt{n}}$ is defined by the following two rules:

$$
\begin{array}{rcll}
E & := & EM \;\big|\; [\cdot] & \text{evaluation contexts} \\
V & := & \lambda x.\, M \;\big|\; v & \text{values} \\
\gamma & := & [v \mapsto V] \cdot \gamma' \;\big|\; [q \mapsto (E,p)] \cdot \gamma' \;\big|\; [x \mapsto M] \cdot \gamma' \;\big|\; \varepsilon & \text{environments} \\
F & := & \langle L, p, \gamma, \phi \rangle \;\big|\; \langle \gamma, \phi \rangle \;\big|\; \langle [? \mapsto \phi], M \rangle & \text{configurations} \\
L & := & M \;\big|\; E[vM] \;\big|\; v & \text{extended } \lambda\text{-terms}
\end{array}
$$

FIGURE 8. Grammars for the call-by-name language and A-OGS configurations

$$
\frac{}{(\lambda x.\, M)N \to_{\mathtt{n}} M\{N/x\}} \qquad \frac{M \to_{\mathtt{n}} N}{E[M] \to_{\mathtt{n}} E[N]}
$$

The OGS for call-by-name uses three kinds of names: *continuation names* (ranged over by p, q), *variable names* (ranged over by x, y), and *value names* (ranged over by v, w). Continuation names are used as in call-by-value; variable names act as pointers to λ-terms and correspond to the variables of the λ-calculus; value names are pointers to values.

The OGS uses the syntactic categories in Figure 8, where L range over extended λ-terms, which are needed to accommodate value names. We write Λ^+ for the set of extended λ-terms.

In a call-by-name setting, Player Questions are of two possible shapes:

- *Player Value-Question* (PVQ) $\bar{v}(x,p)$, receiving a variable x and a continuation name p through a value name v;
- *Player Term-Question* (PTQ) $\bar{x}(p)$, receiving a continuation name p through a variable x.

Opponent Questions are also of two different shapes too:

- *Opponent Value-Question* (OVQ) $v(x,p)$, sending a variable x and a continuation name p through a value name v;
- *Opponent Term-Question* (OTQ) $x(p)$, sending a continuation name p through a variable x.

The encodings of call-by-name λ-calculus, OGS environments and configurations, and associated syntactic categories, are reported in Figure 10. The Iπ representation uses the same 3 types of names as the OGS representation.

As for call-by-value, so in call-by-name the encoding makes use of *link* processes, which are linear structures for continuation names and replicated structures for the other names. We recall their definition. For tuples of names $\widetilde{u} = u_1, \ldots, u_n$ and $\widetilde{v} = v_1, \ldots, v_n$ we write $\widetilde{u} \rhd \widetilde{v}$ to abbreviate $u_1 \rhd v_1 \mid \ldots \mid u_n \rhd v_n$:

$$
\rhd \;\stackrel{\text{def}}{=}\; \begin{cases} (x,y).\, !x(p).\, \overline{y}(q).\, q \rhd p \\ \qquad \text{if } x, y \text{ are variable names} \\ \\ (a,b).\, a(\widetilde{c}).\, \overline{b}(\widetilde{d}).\, \widetilde{d} \rhd \widetilde{c}) \\ \qquad\qquad\qquad \text{otherwise} \end{cases}
$$

(τ)	$\langle M,p,\gamma,\phi\rangle$	$\xrightarrow{\tau}_{\mathtt{a}}$	$\langle N,p,\gamma,\phi\rangle$	when $M \to_{\mathtt{n}} N$
(PA)	$\langle V,p,\gamma,\phi\rangle$	$\xrightarrow{\overline{p}(v)}_{\mathtt{a}}$	$\langle \gamma\cdot[v\mapsto V],\phi\uplus\{v\}\rangle$	
(PVQ)	$\langle E[vM],p,\gamma,\phi\rangle$	$\xrightarrow{\overline{v}(y,q)}_{\mathtt{a}}$	$\langle \gamma\cdot[y\mapsto M]\cdot[q\mapsto(E,p)],\phi\uplus\{y,q\}\rangle$	
(PTQ)	$\langle E[x],p,\gamma,\phi\rangle$	$\xrightarrow{\overline{x}(q)}_{\mathtt{a}}$	$\langle \gamma\cdot[q\mapsto(E,p)],\phi\uplus\{q\}\rangle$	
(OA)	$\langle \gamma\cdot[p\mapsto(E,q)],\phi\rangle$	$\xrightarrow{p(v)}_{\mathtt{a}}$	$\langle E[v],q,\gamma,\phi\uplus\{v\}\rangle$	
(OVQ)	$\langle \gamma\cdot[v\mapsto V],\phi\rangle$	$\xrightarrow{v(y,p)}_{\mathtt{a}}$	$\langle Vy,p,\gamma,\phi\uplus\{p,y\}\rangle$	when $\gamma(v)=V$
(OTQ)	$\langle \gamma,\phi\rangle$	$\xrightarrow{x(p)}_{\mathtt{a}}$	$\langle M,p,\gamma,\phi\uplus\{p\}\rangle$	when $\gamma(x)=M$
(IOQ)	$\langle [?\mapsto M],\phi\rangle$	$\xrightarrow{(p)}_{\mathtt{a}}$	$\langle M,p,\varepsilon,\phi\uplus\{p\}\rangle$	

FIGURE 9. CBN OGS LTS

We have:

$$\mathcal{N}[\![E[v]]\!] \;=\; \begin{cases} \mathcal{N}[\![E'[vM]]\!] & \text{if } E = E'[\,[\cdot]M\,] \\ \mathcal{N}[\![v]\!] & \text{if } E=[\cdot] \end{cases}$$

Lemma 13.1. $\mathcal{N}[\![E[x]]\!] \;=\; (p)\ \overline{x}(q).\mathcal{N}[\![[q\mapsto(E,p)]]\!]$

The results of operational correspondence between OGS and Iπ under the opLTS are the same as for call-by-value. We only report the operational correspondence for strong transitions and the analogue of Corollary 6.11.

Theorem 13.2.

(1) If $F \longrightarrow_{\mathtt{a}} F'$ then $\mathcal{N}[\![F]\!] \longrightarrow_\pi {}_\pi\!\gtrsim \mathcal{N}[\![F']\!]$;
(2) If $F \xrightarrow{\ell}_{\mathtt{a}} F'$ then $\mathcal{N}[\![F]\!] \xrightarrow{\mu} \approx_\pi \mathcal{N}[\![F']\!]$.

Theorem 13.3.

(1) If $\mathcal{N}[\![F]\!] \xrightarrow{\tau}_{\mathtt{o}\pi} P$ then there is F' such that $F \longrightarrow_{\mathtt{a}} F'$ and $P \,{}_\pi\!\gtrsim \mathcal{N}[\![F']\!]$;
(2) If $\mathcal{N}[\![F]\!] \xrightarrow{\ell}_{\mathtt{o}\pi} P$ then there is F' such that $F \xrightarrow{\ell}_{\mathtt{a}} F'$ and $P \approx_\pi \mathcal{N}[\![F']\!]$.

Corollary 13.4. *For any configuration F and trace s, we have $F \stackrel{s}{\Longrightarrow}_{\mathtt{a}}$ iff $\mathcal{N}[\![F]\!] \stackrel{s}{\Longrightarrow}_{\mathtt{o}\pi}$.*

13.1. **Concurrent Operational Game Semantics.** The modification to obtain the concurrent version of OGS (C-OGS) for call-by-name are similar to those for call-by-value in Section 7.

Below are the rules for the call-by-name OGS (C-OGS). We use the same notations as the call-by-value version.

Encoding of Λ into Iπ processes:

$$\mathcal{N}[\![\lambda x.\, M]\!] \stackrel{\text{def}}{=} (p)\ \overline{p}(v).\, v(x,q).\mathcal{N}[\![M]\!]\langle q\rangle$$

$$\mathcal{N}[\![x]\!] \stackrel{\text{def}}{=} (p)\ \overline{x}(r).\, r \rhd p$$

$$\mathcal{N}[\![MN]\!] \stackrel{\text{def}}{=} (p)\ \boldsymbol{\nu} q\, \Big(\mathcal{N}[\![M]\!]\langle q\rangle \mid q(v).\, \overline{v}(x,p').\, (p' \rhd p \mid\, !x(r).\mathcal{N}[\![N]\!]\langle r\rangle)\Big)$$

Encoding of Λ^+:

$$\mathcal{N}[\![E[vM]]\!] \stackrel{\text{def}}{=} (p)\ \overline{v}(x,r).\, (\mathcal{N}[\![[r \mapsto (E,p)]]\!] \mid\, !x(q).\mathcal{N}[\![M]\!]\langle q\rangle)$$

$$\mathcal{N}[\![v]\!] \stackrel{\text{def}}{=} (p)\ \overline{p}(w).\, w \rhd v$$

Encoding of OGS environments:

$$\mathcal{N}[\![[v \mapsto V]\cdot\gamma]\!] \stackrel{\text{def}}{=} \begin{cases} v(x,q).\mathcal{N}[\![M]\!]\langle q\rangle \mid \mathcal{N}[\![\gamma]\!] & \text{if } V = \lambda x.\, M \\ v \rhd w \mid \mathcal{N}[\![\gamma]\!] & \text{if } V = w \end{cases}$$

$$\mathcal{N}[\![[x \mapsto M]\cdot\gamma]\!] \stackrel{\text{def}}{=} !x(q).\mathcal{N}[\![M]\!]\langle q\rangle \mid \mathcal{N}[\![\gamma]\!]$$

$$\mathcal{N}[\![[q \mapsto (E,p)]\cdot\gamma]\!] \stackrel{\text{def}}{=} \begin{cases} q(v).\, \overline{v}(x,r).\, (\mathcal{N}[\![[r \mapsto E',p]]\!] \mid\, !x(q).\mathcal{N}[\![M]\!]\langle q\rangle) \mid \mathcal{N}[\![\gamma]\!] & \text{if } E = E'[[\cdot]\, M] \\ q \rhd p \mid \mathcal{N}[\![\gamma]\!] & \text{if } E = [\cdot] \end{cases}$$

$$\mathcal{N}[\![\emptyset]\!] \stackrel{\text{def}}{=} 0$$

Encoding of OGS configurations:

$$\mathcal{N}[\![\langle M,p,\gamma,\phi\rangle]\!] \stackrel{\text{def}}{=} \mathcal{N}[\![M]\!]\langle p\rangle \mid \mathcal{N}[\![\gamma]\!]$$

$$\mathcal{N}[\![\langle v,p,\gamma,\phi\rangle]\!] \stackrel{\text{def}}{=} \mathcal{N}[\![v]\!]\langle p\rangle \mid \mathcal{N}[\![\gamma]\!]$$

$$\mathcal{N}[\![\langle E[vM],p,\gamma,\phi\rangle]\!] \stackrel{\text{def}}{=} \mathcal{N}[\![E[vM]]\!]\langle p\rangle \mid \mathcal{N}[\![\gamma]\!]$$

$$\mathcal{N}[\![\langle \gamma,\phi\rangle]\!] \stackrel{\text{def}}{=} \mathcal{N}[\![\gamma]\!]$$

FIGURE 10. The encoding of call-by-name λ-calculus and OGS into Iπ

$$
\begin{array}{l|lll}
(Int) & \langle A \cdot [p \mapsto M], \gamma, \phi\rangle & \longrightarrow_{\mathtt{c}} \langle A \cdot [p \mapsto N], \gamma\rangle & \text{when } M \to_{\mathtt{n}} N \\
(PA) & \langle A \cdot [p \mapsto V], \gamma, \phi\rangle & \xrightarrow{\bar{p}(v)}_{\mathtt{c}} \langle A, \gamma \cdot [v \mapsto V], \phi \uplus \{v\}\rangle & \\
(PVQ) & \langle A \cdot [p \mapsto E[vM]], \gamma, \phi\rangle & \xrightarrow{\bar{v}(y,q)}_{\mathtt{c}} \langle A, \gamma \cdot [y \mapsto M] \cdot [q \mapsto (E,p)], \phi \uplus \{y,q\}\rangle & \\
(PTQ) & \langle A \cdot [p \mapsto E[x]], \gamma, \phi\rangle & \xrightarrow{\bar{x}(q)}_{\mathtt{c}} \langle A, \gamma \cdot [q \mapsto (E,p)], \phi \uplus \{q\}\rangle & \\
(OA) & \langle A, \gamma, \phi\rangle & \xrightarrow{p(v)}_{\mathtt{c}} \langle A \cdot [q \mapsto E[v]], \gamma, \phi \uplus \{v\}\rangle & \text{when } \gamma(p) = (E,q) \\
(OVQ) & \langle A, \gamma \cdot [v \mapsto V], \phi\rangle & \xrightarrow{v(y,p)}_{\mathtt{c}} \langle A \cdot [p \mapsto Vy], \gamma, \phi \uplus \{p,y\}\rangle & \\
(OTQ) & \langle A, \gamma, \phi\rangle & \xrightarrow{x(p)}_{\mathtt{c}} \langle A \cdot [p \mapsto M], \gamma, \phi \uplus \{p\}\rangle & \text{when } \gamma(x) = M \\
(IOQ) & \langle [? \mapsto M], \phi\rangle & \xrightarrow{(p)}_{\mathtt{c}} \langle M, p, \varepsilon, \phi \uplus \{p\}\rangle &
\end{array}
$$

As usual, for encoding C-OGS, it suffices to consider running terms and configurations, as the encoding remains otherwise the same. The encoding of the running term is:

$$
\begin{array}{rcl}
\mathcal{N}[\![[p \mapsto M] \cdot A]\!] & \stackrel{\text{def}}{=} & \mathcal{N}[\![M]\!]\langle p\rangle \mid \mathcal{N}[\![A]\!] \\
\mathcal{N}[\![\varepsilon]\!] & \stackrel{\text{def}}{=} & 0
\end{array}
$$

The encoding of configurations is then:

$$
\mathcal{N}[\![\langle A, \gamma, \phi\rangle]\!] \stackrel{\text{def}}{=} \mathcal{N}[\![A]\!] \mid \mathcal{N}[\![\gamma]\!]
$$

We report the operational correspondence for strong transitions.

Lemma 13.5 (from C-OGS to Iπ, strong transitions)**.**

(1) If $F \longrightarrow_{\mathtt{c}} F'$ *then* $\mathcal{N}[\![F]\!] \longrightarrow_{\pi} \pi\gtrsim \mathcal{N}[\![F']\!]$*;*
(2) If $F \xrightarrow{\ell}_{\mathtt{c}} F'$ *then* $\mathcal{N}[\![F]\!] \xrightarrow{\ell}_{\pi} \approx_{\pi} \mathcal{N}[\![F']\!]$*.*

Lemma 13.6 (from Iπ to C-OGS, strong transitions)**.**

(1) If $\mathcal{N}[\![F]\!] \longrightarrow_{\pi} P$*, then there is* F' *s.t.* $F \longrightarrow_{\mathtt{c}} F'$ *and* $P \;\pi\gtrsim \mathcal{N}[\![F']\!]$*;*
(2) If $\mathcal{N}[\![F]\!] \xrightarrow{\ell}_{\pi} P$*, then there is* F' *s.t.* $F \xrightarrow{\ell}_{\mathtt{c}} F'$ *and* $P \approx_{\pi} \mathcal{N}[\![F']\!]$*.*

Lemma 13.7. *For any* F, F' *we have:*

(1) $F_1 \eqsim_{\mathtt{c}} F_2$ *iff* $\mathcal{N}[\![F_1]\!] \eqsim_{\pi} \mathcal{N}[\![F_2]\!]$*;*
(2) $F_1 \approx_{\mathtt{c}} F_2$ *iff* $\mathcal{N}[\![F_1]\!] \approx_{\pi} \mathcal{N}[\![F_2]\!]$*.*

Theorem 13.8. *For all* M, N *with* $\mathtt{fv}(M,N) \subseteq \phi$*, we have:* $\langle M, \phi\rangle \eqsim_{\mathtt{c}} \langle N, \phi\rangle$ *iff* $\langle M, \phi\rangle \approx_{\mathtt{c}} \langle N, \phi\rangle$ *iff* $\mathcal{N}[\![M]\!] \eqsim_{\pi} \mathcal{N}[\![N]\!]$ *iff* $\mathcal{N}[\![M]\!] \approx_{\pi} \mathcal{N}[\![N]\!]$*.*

13.2. **Up-to techniques for games, and relationship between Concurrent and Alternating OGS.** As for call-by-value, so for call-by-name the 'up-to composition' technique for bisimulation (Definition 7.15) can be imported from the π-calculus, and then used to establish that the equivalence induced on λ-terms by the sequential and concurrent versions of OGS coincide.

We use the technique to prove the call-by-name analogue of Lemma 7.23.

Corollary 13.9. *For any λ-terms M, N, the following statement are the same: $\langle M\rangle \doteq_{\mathtt{a}} \langle N\rangle$; $\langle M\rangle \approx_{\mathtt{a}} \langle N\rangle$; $\langle M\rangle \doteq_{\mathtt{c}} \langle N\rangle$; $\langle M\rangle \approx_{\mathtt{c}} \langle N\rangle$; $\mathcal{N}[\![M]\!] \doteq_{\mathtt{o}\pi} \mathcal{N}[\![N]\!]$; $\mathcal{N}[\![M]\!] \approx_{\mathtt{o}\pi} \mathcal{N}[\![N]\!]$; $\mathcal{N}[\![M]\!] \doteq_{\pi} \mathcal{N}[\![N]\!]$; $\mathcal{N}[\![M]\!] \approx_{\pi} \mathcal{N}[\![N]\!]$.*

The equivalence induced by the Iπ (or π-calculus) encoding of the call-by-name λ-calculus is known to coincide with that of the Lévy-Longo Trees [San00]. In the light of Corollary 13.9, the result can thus be transported to OGS, both in its Alternating and its Concurrent variants, and both for traces and for bisimilarity.

Connections between game semantics and tree representation of λ-terms have been previously explored, e.g., in [ODG04], where a game model of untyped call-by-name λ-calculus is shown to correspond to Levy-Longo trees.

The connections that we have derived above, however, go beyond Lévy-Longo Trees: in particular, the bisimilarities that have been established between the observables (i.e., the dynamics) in the two models set a tight relationship between them while allowing one to transfer results and techniques along the lines of what we have shown for call-by-value.

14. Related and Future Work

Analogies between game semantics and π-calculus, as semantic frameworks in which *names* are central, have been pointed out from the very beginning of game semantics. In the pioneering work [HO95], the authors obtain a translation of PCF terms into the π-calculus from a game model of PCF by representing *strategies* (the denotation of PCF terms in the game model) as processes of the π-calculus. The encoding bears similarities with Milner's, though they are syntactically rather different ("it is clear that the two are conceptually quite unrelated", [HO95]). The connection has been developed in various papers, e.g., [Hon02, HY99, FH98, YCS20, CY19]. Milner's encodings into the π-calculus have sometimes been a source of inspiration in the definition of the game semantics models (e.g., transporting the work [HO95], in call-by-name, onto call-by-value [HY99]). In [YCS20], a typed variant of π-calculus, influenced by differential linear logic [Ehr18], is introduced as a metalanguage to represent game models. The approach is illustrated on a a higher-order call-by-value language with shared memory concurrency, whose game model is build by first translated it to this metalanguage, then using the game model of this metalanguage. In [DF15], games are defined using algebraic operations on sets of traces, and used to prove type soundness of a simply-typed call-by-value λ-calculus with effects. Although the calculus of traces employed is not a π-calculus (e.g., being defined from operators and relations over trace sets rather than from syntactic process constructs), there are similarities, which would be interesting to investigate.

Usually in the above papers the source language is a form of λ-calculus, that is interpreted into game semantics, and the π-calculus (or dialects of it) is used to represent the resulting strategies and games. Another goal has been to shed light on typing disciplines for π-calculus processes, by transplanting conditions on strategies such as well-bracketing and innocence into appropriate typings for the π-calculus (see, e.g., [BHY01, YBH01, HPS21]).

In contrast with the above works, where analogies between game semantics and π-calculus are used to better understand one of the two models (i.e., explaining game semantics in terms process interactions, or enhancing type systems for processes following structures in game semantics), in the present paper we have carried out a direct comparison between the two models (precisely OGS and Iπ). For this we have started from the (arguably natural) representations of the λ-calculus into OGS and Iπ (the latter being Milner's encodings). Our goal was understanding the relation between the behaviours of the terms in the two models, and transferring techniques and results between them.

Technically, our work builds on [DHS18, Dur20], where a detailed analysis of the behaviour of Milner's call-by-value encoding is carried out using proof techniques for π-calculus based on unique-solution of equations. Various results in [DHS18, Dur20] are essential to our own (the observation that Milner's encodings should be interpreted in Iπ rather than the full π-calculus is also from [DHS18, Dur20]).

An OGS for the call-by-name $\lambda\mu$-calculus is introduced in [Lai20]. It is different, once restricted to the fragment without the μ-binder, to the one we consider in Section 13. It might correspond to a different way of encoding call-by-name into the π-calculus where λ-abstractions are interpreted as input-prefixed processes, as e.g., in Milner's original encoding [Mil92].

Bisimulations over OGS terms, and tensor products of configurations, were introduced in [LS14], in order to provide a framework to study compositionality properties of OGS. In our case the compositionality result of OGS is derived from the correspondence with the π-calculus. In [ST19], a correspondence between an i/o typed asynchronous π-calculus and a computational λ-calculus with channel communication is established, using a common categorical model (a compact closed Freyd category). It would be interesting to see if our concurrent operational game model could be equipped with this categorical semantics.

Normal form (or *open*) bisimulations [Las05, SL07b], as game semantics, manipulate open terms, and sometimes make use of environments or stacks of evaluation contexts (see e.g., the recent work [BLP19], where a fully abstract normal-form bisimulation for a λ-calculus with store is obtained).

Notice that contextual equivalence for such a language corresponds to complete well-bracketed trace equivalence. It would be interesting to see if, once the reasoning on diverging configurations removed, the enf-bisimulations obtained there could be seen as up-to composition bisimulations over the OGS LTS .

There are also works that build game models directly for the π-calculus. In [Lai05], a game models for the simply-typed asynchronous π-calculus is introduced, and proved fully abstract for may-equivalence. In [Lai06], this construction was extended to a model of Asynchronous Concurrent ML. These models rely on a sequential representation of interactions as traces. More recently, causal representation of game models of the π-calculus have been proposed. In [ST17], a truly concurrent game model for the asynchronous π-calculus is introduced, using a directed acyclic graph structure to represent interactions. A game model for a synchronous π-calculus is defined in [EHS15], and shown to be fully abstract for fair testing equivalence. A correspondence between a synchronous π-calculus with session types and concurrent game semantics [WRCC17] is given in [CY19], relating games (represented as arenas) to session types, and strategies (defined as coincident event structures) to processes.

We have exploited the full abstraction results between OGS and Iπ to transport a few up-to techniques for bisimulation from Iπ onto OGS. However, in Iπ there are various other such techniques, even a theory of bisimulation enhancements. We would like to see which other techniques could be useful in OGS, possibly transporting the theory of enhancements itself. We would like also to study the possible game-semantics counterpart of recent work on a representation of functions as processes [SS23] in which different tree structures of the λ-calculus (Lévy-Longo trees, Böhm trees, Böhm trees with infinite η) are obtained by modifying the semantics of the certain special processes used as a parameter of the representation (and related to the *link* processes of Section 5 and 13, called *wires* in [SS23]).

References

[Abr89] Samson Abramsky. The lazy lambda calculus. In D. Turner, editor, *Research Topics in Functional Programming*, pages 65–116. Addison-Wesley, 1989.

[Abr97] Samson Abramsky. Game semantics for programming languages (abstract). In *Proceedings of the 22nd International Symposium on Mathematical Foundations of Computer Science*, MFCS '97, page 3–4, Berlin, Heidelberg, 1997. Springer-Verlag.

[Acc] Beniamino Accattoli. Evaluating functions as processes. *EPTCS 110*, page 41.

[AHM98] Samson Abramsky, Kohei Honda, and Guy McCusker. A fully abstract game semantics for general references. In *Proceedings of the 13th Annual IEEE Symposium on Logic in Computer Science*, LICS '98, page 334, USA, 1998. IEEE Computer Society.

[AJM00] Samson Abramsky, Radha Jagadeesan, and Pasquale Malacaria. Full abstraction for pcf. *Inf. Comput.*, 163(2):409–470, December 2000. doi:10.1006/inco.2000.2930.

[AM97] Samson Abramsky and Guy McCusker. Linearity, sharing and state: a fully abstract game semantics for Idealized Algol with active expressions. In *Algol-like languages*, pages 297–329. Springer, 1997.

[BHY01] Martin Berger, Kohei Honda, and Nobuko Yoshida. Sequentiality and the π-calculus. In *International Conference on Typed Lambda Calculi and Applications*, pages 29–45. Springer, 2001.

[BLP19] Dariusz Biernacki, Sergueï Lenglet, and Piotr Polesiuk. A complete normal-form bisimilarity for state. In *International Conference on Foundations of Software Science and Computation Structures*, pages 98–114. Springer, 2019.

[CCS10] Matteo Cimini, Claudio Sacerdoti Coen, and Davide Sangiorgi. Functions as processes: Termination and the $\lambda\widetilde{\mu mu}$–calculus. In *International Symposium on Trustworthy Global Computing*, pages 73–86. Springer, 2010.

[Cur98] Pierre-Louis Curien. Abstract Böhm trees. *Mathematical Structures in Computer Science*, 8(6):559–591, 1998.

[CY19] Simon Castellan and Nobuko Yoshida. Two sides of the same coin: Session types and game semantics: A synchronous side and an asynchronous side. *Proc. ACM Program. Lang.*, January 2019. doi:10.1145/3290340.

[DCdP94] Mariangiola Dezani-Ciancaglini, Ugo de'Liguoro, and Adolfo Piperno. Fully abstract semantics for concurrent λ-calculus. In *International Symposium on Theoretical Aspects of Computer Software*, pages 16–35. Springer, 1994.

[DF15] Tim Disney and Cormac Flanagan. Game semantics for type soundness. In *Proceedings of the 2015 30th Annual ACM/IEEE Symposium on Logic in Computer Science (LICS)*, LICS '15, page 104–114, USA, 2015. IEEE Computer Society. doi:10.1109/LICS.2015.20.

[DHS18] Adrien Durier, Daniel Hirschkoff, and Davide Sangiorgi. Eager functions as processes. In *33nd Annual ACM/IEEE Symposium on Logic in Computer Science, LICS 2018*. IEEE Computer Society, 2018.

[Dur20] Adrien Durier. *Unique solution techniques for processes and functions.* PhD thesis, University of Lyon, France, 2020.

[Ehr18] Thomas Ehrhard. An introduction to differential linear logic: proof-nets, models and antiderivatives. *Mathematical Structures in Computer Science*, 28(7):995–1060, 2018.

[EHS15] Clovis Eberhart, Tom Hirschowitz, and Thomas Seiller. An intensionally fully-abstract sheaf model for pi. In *6th Conference on Algebra and Coalgebra in Computer Science (CALCO 2015)*. Schloss Dagstuhl-Leibniz-Zentrum fuer Informatik, 2015.

[FH98] Marcelo Fiore and Kohei Honda. Recursive types in games: Axiomatics and process representation. In *Proceedings of the 13th Annual IEEE Symposium on Logic in Computer Science*, LICS '98, page 345, USA, 1998. IEEE Computer Society.

[GM04] Dan R. Ghica and Andrzej S. Murawski. Angelic semantics of fine-grained concurrency. In *International Conference on Foundations of Software Science and Computation Structures*, pages 211–225. Springer, 2004.

[GT12] Dan R. Ghica and Nikos Tzevelekos. A system-level game semantics. *Electron. Notes Theor. Comput. Sci.*, 286:191–211, September 2012. doi:10.1016/j.entcs.2012.08.013.

[HO95] J. M. E. Hyland and C.-H. L. Ong. Pi-calculus, dialogue games and full abstraction PCF. In *Proceedings of the Seventh International Conference on Functional Programming Languages and Computer Architecture*, FPCA '95, page 96–107, New York, NY, USA, 1995. Association for Computing Machinery. doi:10.1145/224164.224189.

[HO00] J.M.E. Hyland and C.-H. L. Ong. On full abstraction for PCF. *Inf. Comput.*, 163(2):285–408, December 2000. doi:10.1006/inco.2000.2917.

[Hon02] Kohei Honda. Processes and games. *Electron. Notes Theor. Comput. Sci.*, 71:40–69, 2002. doi:10.1016/S1571-0661(05)82528-9.

[HPS20] Daniel Hirschkoff, Enguerrand Prebet, and Davide Sangiorgi. On the representation of references in the pi-calculus. In Igor Konnov and Laura Kovács, editors, *Proc. CONCUR 2020*, volume 171 of *LIPIcs*, pages 34:1–34:20. Schloss Dagstuhl - Leibniz-Zentrum für Informatik, 2020.

[HPS21] Daniel Hirschkoff, Enguerrand Prebet, and Davide Sangiorgi. On sequentiality and well-bracketing in the π-calculus. In *36th ACM/IEEE Symposium on Logic in Computer Science*, pages 1–13. IEEE, 2021. doi:10.1109/LICS52264.2021.9470559.

[HY99] Kohei Honda and Nobuko Yoshida. Game-theoretic analysis of call-by-value computation. *Theor. Comput. Sci.*, 221(1–2):393–456, June 1999. doi:10.1016/S0304-3975(99)00039-0.

[JJR03] Radha Jagadeesan, Alan Jeffrey, and James Riely. A calculus of untyped aspect-oriented programs. In *European Conference on Object-Oriented Programming*, pages 54–73. Springer, 2003.

[JM21] G. Jaber and A. S. Murawski. Complete trace models of state and control. In *Proceedings of ESOP*, 2021. to appear. URL: https://hal.archives-ouvertes.fr/hal-03116698.

[JPR09] Radha Jagadeesan, Corin Pitcher, and James Riely. Open bisimulation for aspects. In *Transactions on Aspect-Oriented Software Development V*, pages 72–132. Springer, 2009.

[JR05] Alan Jeffrey and Julian Rathke. Java jr: Fully abstract trace semantics for a core java language. In *European symposium on programming*, pages 423–438. Springer, 2005.

[JS21] Guilhem Jaber and Davide Sangiorgi. Games, mobile processes, and functions (long version). 2021. URL: https://hal.archives-ouvertes.fr/hal-03407123.

[JS22] Guilhem Jaber and Davide Sangiorgi. Games, mobile processes, and functions. In Florin Manea and Alex Simpson, editors, *30th EACSL Annual Conference on Computer Science Logic, CSL 2022*, LIPIcs, pages 25:1–25:18. Schloss Dagstuhl - Leibniz-Zentrum für Informatik, 2022. doi:10.4230/LIPICS.CSL.2022.25.

[JT16] Guilhem Jaber and Nikos Tzevelekos. Trace semantics for polymorphic references. In *Proceedings of the 31st Annual ACM/IEEE Symposium on Logic in Computer Science*, LICS '16, page 585–594, New York, NY, USA, 2016. Association for Computing Machinery. doi:10.1145/2933575.2934509.

[Lai97] James Laird. Full abstraction for functional languages with control. In *Proceedings of the 12th Annual IEEE Symposium on Logic in Computer Science*, LICS '97, page 58, USA, 1997. IEEE Computer Society.

[Lai05] James Laird. A game semantics of the asynchronous π-calculus. In *International Conference on Concurrency Theory*, pages 51–65. Springer, 2005.

[Lai06] James Laird. Game semantics for higher-order concurrency. In *Proceedings of the 26th International Conference on Foundations of Software Technology and Theoretical Computer Science*, FSTTCS'06, page 417–428, Berlin, Heidelberg, 2006. Springer-Verlag. doi:10.1007/11944836.38.

[Lai07] James Laird. A fully abstract trace semantics for general references. In *Proceedings of the 34th International Conference on Automata, Languages and Programming*, ICALP'07, page 667–679, Berlin, Heidelberg, 2007. Springer-Verlag.

[Lai20] James Laird. A curry-style semantics of interaction: From untyped to second-order lazy λ μ-calculus. In Jean Goubault-Larrecq and Barbara König, editors, *Foundations of Software Science and Computation Structures - 23rd International Conference, FOSSACS 2020, Held as Part of the European Joint Conferences on Theory and Practice of Software, ETAPS 2020, Dublin, Ireland, April 25-30, 2020, Proceedings*, Lecture Notes in Computer Science, pages 422–441. Springer, 2020. doi:10.1007/978-3-030-45231-5_22.

[Las05] Søren B. Lassen. Eager normal form bisimulation. In *20th IEEE Symposium on Logic in Computer Science (LICS 2005), 26-29 June 2005, Chicago, IL, USA, Proceedings*, pages 345–354, 2005. doi:10.1109/LICS.2005.15.

[LS14] Paul Blain Levy and Sam Staton. Transition systems over games. In *Proceedings of the Joint Meeting of the Twenty-Third EACSL Annual Conference on Computer Science Logic (CSL) and the Twenty-Ninth Annual ACM/IEEE Symposium on Logic in Computer Science (LICS)*, CSL-LICS '14, New York, NY, USA, 2014. Association for Computing Machinery. doi:10.1145/2603088.2603150.

[Maz03] Damiano Mazza. Pi et lambda. *Une étude sur la traduction des lambda-termes dans le pi-calcul. Memoire de DEA (in french)*, 2003.

[Mil89] R. Milner. *Communication and Concurrency*. Prentice Hall, 1989.

[Mil91] R. Milner. The polyadic π-calculus: a tutorial. Technical Report ECS–LFCS–91–180, LFCS, 1991. Also in *Logic and Algebra of Specification*, ed. F.L. Bauer, W. Brauer and H. Schwichtenberg, Springer Verlag, 1993.

[Mil92] R. Milner. Functions as processes. *MSCS*, 2(2):119–141, 1992.

[ODG04] C.-H. L. Ong and Pietro Di Gianantonio. Games characterizing levy–longo trees. *Theoretical computer science*, 312(1):121–142, 2004.

[Plo75] G.D. Plotkin. Call by name, call by value and the λ-calculus. *TCS*, 1:125–159, 1975.

[PS96] B. Pierce and D. Sangiorgi. Typing and subtyping for mobile processes. *MSCS*, 6(5):409–454, 1996.

[PS12] Damien Pous and Davide Sangiorgi. Enhancements of the bisimulation proof method. In Davide Sangiorgi and Jan Rutten, editors, *Advanced Topics in Bisimulation and Coinduction*. Cambridge University Press, 2012.

[PS19] Damien Pous and Davide Sangiorgi. Bisimulation and coinduction enhancements: A historical perspective. *Formal Asp. Comput.*, 31(6):733–749, 2019. doi:10.1007/s00165-019-00497-w.

[San93] D. Sangiorgi. An investigation into functions as processes. In *Proc. Ninth International Conference on the Mathematical Foundations of Programming Semantics (MFPS'93)*, pages 143–159. Springer, 1993.

[San95] D. Sangiorgi. πI: A symmetric calculus based on internal mobility. In *Proc. TAPSOFT'95*, volume 915, pages 172–186. Springer, 1995.

[San96a] D. Sangiorgi. Locality and non-interleaving semantics in calculi for mobile processes. *TCS*, 155:39–83, 1996.

[San96b] D. Sangiorgi. π-calculus, internal mobility and agent-passing calculi. *TCS*, 167(2):235–274, 1996.

[San99] D. Sangiorgi. The name discipline of uniform receptiveness. *TCS*, 221:457–493, 1999.

[San00] D. Sangiorgi. Lazy functions and mobile processes. In G. Plotkin, C. Stirling, and M. Tofte, editors, *Proof, Language and Interaction: Essays in Honour of Robin Milner*. MIT Press, 2000.

[SL07a] Kristian Støvring and Soren B. Lassen. A complete, co-inductive syntactic theory of sequential control and state. *SIGPLAN Not.*, 42(1):161–172, January 2007. doi:10.1145/1190215.1190244.

[SL07b] Kristian Støvring and Soren B. Lassen. A complete, co-inductive syntactic theory of sequential control and state. In *Proceedings of the 34th Annual ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages*, POPL '07, page 161–172, New York, NY, USA, 2007. Association for Computing Machinery. doi:10.1145/1190216.1190244.

[SL09] Kristian Støvring and Søren B. Lassen. A complete, co-inductive syntactic theory of sequential control and state. In *Semantics and Algebraic Specification, Essays Dedicated to Peter D. Mosses on the Occasion of His 60th Birthday*, pages 329–375, 2009. doi:10.1007/978-3-642-04164-8_17.

[SS23] Ken Sakayori and Davide Sangiorgi. Extensional and non-extensional functions as processes. In *38th ACM/IEEE Symposium on Logic in Computer Science*, pages 1–13. IEEE, 2023. doi:10.1109/LICS56636.2023.10175686.

[ST17] Ken Sakayori and Takeshi Tsukada. A truly concurrent game model of the asynchronous π-calculus. In *International Conference on Foundations of Software Science and Computation Structures*, pages 389–406. Springer, 2017.

[ST19] Ken Sakayori and Takeshi Tsukada. A categorical model of an $\mathbf{i}/\mathbf{o}$-typed π-calculus. In *European Symposium on Programming*, pages 640–667. Springer, 2019.

[SW01] D. Sangiorgi and D. Walker. *The π-calculus: a Theory of Mobile Processes*. Cambridge University Press, 2001.

[vBV14] Steffen van Bakel and Maria Grazia Vigliotti. A fully-abstract semantics of lambda-mu in the pi-calculus. In Paulo Oliva, editor, *Proceedings Fifth International Workshop on Classical Logic and Computation, CL&C 2014, Vienna, Austria, July 13, 2014*, volume 164 of *EPTCS*, pages 33–47, 2014.

[WRCC17] Glynn Winskel, Silvain Rideau, Pierre Clairambault, and Simon Castellan. Games and strategies as event structures. *Logical Methods in Computer Science*, 13, 2017.

[YBH01] Nobuko Yoshida, Martin Berger, and Kohei Honda. Strong normalisation in the π-Calculus. In *16th Annual IEEE Symposium on Logic in Computer Science (LICS'01)*, pages 311–322. IEEE Computer Society, 2001.

[YCS20] Nobuko Yoshida, Simon Castellan, and Léo Stefanesco. Game semantics: Easy as pi. *arXiv preprint arXiv:2011.05248*, 2020.

Appendix A. Auxiliary material for Section 6.2

On the encoding of λ-terms, the ordinary bisimilarity $\approx_\pi$ implies the 'bisimilarity respecting divergence' $\approx_\pi^{\Uparrow}$. This because, intuitively, a term $\mathcal{V}[\![M]\!]\langle p\rangle$ is divergent iff M is so in the call-by-value λ-calculus. Formally, the result is proved by rephrasing the result about the correspondence between $\approx_\pi$ and Lassen's trees (Theorem 5.1) in terms of $\approx_\pi^{\Uparrow}$ in place of $\approx_\pi$. Again, this property boils down to the fact that the uses of $\lesssim_\pi$ in Lemmas 5.2-B.3 (on which also the characterisation in terms of Lassen's trees is built) respect divergence. Thus, using also Lemma 6.6, we derive the following result.

Theorem A.1. *For $M, N \in \Lambda$, we have: $\mathcal{V}[\![M]\!]\langle p\rangle \approx_\pi \mathcal{V}[\![N]\!]\langle p\rangle$ iff $\mathcal{V}[\![M]\!]\langle p\rangle \approx_{\mathsf{o}\pi} \mathcal{V}[\![N]\!]\langle p\rangle$.*

Below we report an alternative proof of the theorem above, however always relying on the variant of lemmas, such as Lemma 5.2 and Lemmas B.1-B.3, with $\lesssim_{\mathsf{o}\pi}$ in place of $\lesssim_\pi$.

Lemma A.2. *If F is a singleton configuration, then $\mathcal{V}[\![F]\!]$ is of three possible forms:*

(1) $\mathcal{V}[\![M]\!]\langle p\rangle$, for some M, p;
(2) $q(x).\,\mathcal{V}[\![E[x]]\!]\langle p\rangle$, for some E, x, q, p;
(3) $\mathcal{V}^*[\![V]\!]\langle y\rangle$, for some V, y (that is, either $!y(x,q).\,\mathcal{V}[\![M]\!]\langle q\rangle$ if $V = \lambda x.\,M$, or $y \triangleright x$ if $V = x$).

Lemma A.3. *Suppose a not free in P_1 and Q_1.*

- *Let $P = P_1 \mid\, !a(\widetilde{b}).\,P_2$, and $Q = Q_1 \mid\, !a(\widetilde{b}).\,Q_2$, and $P \approx_\pi Q$. Then also $P_1 \approx_\pi Q_1$.*
- *Similarly, let $P = P_1 \mid\, a(\widetilde{b}).\,P_2$, and $Q = Q_1 \mid\, a(\widetilde{b}).\,Q_2$, and $P \approx_\pi Q$. Then also $P_1 \approx_\pi Q_1$.*

Theorem A.4. *If F and G are singleton configurations, then $\mathcal{V}[\![F]\!] \approx_\pi \mathcal{V}[\![G]\!]$ implies $\mathcal{V}[\![F]\!] \approx_{\mathsf{o}\pi} \mathcal{V}[\![G]\!]$.*

Proof. Let $\mathcal{R}$ be the relations with all pairs (P, Q) where P and Q are of the form

$$\begin{aligned} P &= P_1 \mid \ldots \mid P_n \\ Q &= Q_1 \mid \ldots \mid Q_n \end{aligned}$$

for some n, where all P_i, Q_i are encodings of singleton configurations and for all i, we have $P_i \approx_\pi Q_i$. Moreover:

- there is at most one i for which P_i, Q_i are encodings of active configurations;
- in any P_j (resp. Q_j) that is the encoding of a passive configuration, the name of the initial input does not appear free in any other component $P_{j'}$ (resp. $Q_{j'}$) for $j' \neq j$.

A consequence of the two conditions above is that two distinct components P_i and P_j cannot interact. That is, if $P \xrightarrow{\mu} P'$, then there is i such that $P_i \xrightarrow{\mu} P_i'$ and $P' = P_1'' \mid \ldots \mid P_n''$ where $P_j'' = P_j$ if $j \neq i$ and $P_i'' = P_i'$. And similarly if $P \xrightarrow{\mu}_{\mathsf{o}\pi} P'$.

We show that $\mathcal{R}$ is a $\approx_{\mathsf{o}\pi}$-bisimulation up-to $\lesssim_{\mathsf{o}\pi}$ (the bisimulation up-to expansion is sound for bisimilarity in any LTS). In the proof below, Lemmas B.2 and B.3 actually refer to the versions of the lemmas mentioned above, with $\lesssim_{\mathsf{o}\pi}$ in place of $\lesssim_\pi$.

Suppose $P \xrightarrow{\mu}_{\mathsf{o}\pi} P'$. The action orginates from some P_i alone, say $P_i \xrightarrow{\mu}_{\mathsf{o}\pi} P_i'$. If μ is a τ action, then also $P_i \xrightarrow{\mu} P_i'$; and since $P_i \approx_\pi Q_i$, we have $Q_i \Longrightarrow Q_i'$ with $P_i' \approx_\pi Q_i$. Moreover, also $Q_i \Longrightarrow_{\mathsf{o}\pi} Q_i'$ — and the corresponding transition from Q.

Using Lemma B.2 we infer that, up-to expansion, the derivative processes P_i' and Q_i' can be rewritten into processes that fit the definition of $\mathcal{R}$.

For the case when μ is an output, one reasons similarly, possibly also using Lemma A.3 when μ is of the form $\overline{x}(z, q)$.

Finally, if $P \xrightarrow{\mu}_{\mathsf{o}\pi} P'$ and μ is an input, then P is input reactive. From the conditions in the definition of $\mathcal{R}$ we infer that Q is input reactive too. Then we can conclude, reasoning as in the previous cases, but this time using Lemma B.3 with γ a singleton. □

Theorem A.1 is then a corollary of Theorem A.4.

Appendix B. Auxiliary results for Section 6

We present results that are needed to establish the operational correspondence between OGS and Iπ, studied in Section 6.

We begin discussing an optimisation $\mathcal{O}$ of the encoding of call-by-value λ-calculus. Following Durier et al.'s [DHS18], we sometimes use $\mathcal{O}$ to simplify proofs. We report the full definition of $\mathcal{O}$; for convenience, we also list the definition of $\mathcal{O}$ on OGS environments and configurations, thought is the same as that for the initial encoding $\mathcal{V}$ — just replacing $\mathcal{V}$ with $\mathcal{O}$ in Figure 4).

The encoding $\mathcal{O}$ is obtained from the initial one $\mathcal{V}$ by inlining the encoding and performing a few (deterministic) reductions, at the price of a more complex definition. Precisely, in the encoding of application some of the initial communications are removed, including those with which a term signals to have become a value. To achieve this, the encoding of an application goes by a case analysis on the occurrences of values in the subterms.

We begin with a few results from [DHS18, Dur20] that are needed to reason about the optimised encoding $\mathcal{O}$, in addition to Lemma 5.2 reported in the main text.

Lemma B.1. *We have:*

$$\mathcal{O}[\![E[xV]]\!]\langle p\rangle \ {}_{\pi}\!\gtrsim \overline{x}(z,q).\,(\mathcal{O}^*[\![V]\!]\langle z\rangle \mid q(y).\,\mathcal{O}[\![E[y]]\!]\langle p\rangle).$$

The proof [DHS18, Dur20] goes by induction on the evaluation context E.

The following lemma [DHS18, Dur20] uses Lemma B.1 to establish the shape of the possible transitions that a term $\mathcal{O}[\![M]\!]\langle p\rangle$ can perform.

Lemma B.2. *For any $M \in \Lambda$ and p, process $\mathcal{O}[\![M]\!]\langle p\rangle$ has exactly one immediate transition and exactly one of the following clauses holds:*

(1) $\mathcal{O}[\![M]\!]\langle p\rangle \xrightarrow{\overline{p}(y)} P$ and M is a value, with $P = \mathcal{O}^*[\![M]\!]\langle y\rangle$;

(2) $\mathcal{O}[\![M]\!]\langle p\rangle \xrightarrow{\overline{x}(z,q)} P$ and M is of the form $E[xV]$, for some E, x, V, with

$$P \ {}_{\pi}\!\gtrsim \mathcal{O}^*[\![V]\!]\langle z\rangle \mid q(y).\,\mathcal{O}[\![E[y]]\!]\langle p\rangle,$$

and moreover z is not free in $q(y).\,\mathcal{O}[\![E[y]]\!]\langle p\rangle$ whereas q is not free in $\mathcal{O}^*[\![V]\!]\langle z\rangle$;

(3) $\mathcal{O}[\![M]\!]\langle p\rangle \xrightarrow{\tau} P$ and there is N with $M \longrightarrow N$ and $P \ {}_{\pi}\!\gtrsim \mathcal{O}[\![N]\!]\langle p\rangle$.

We now move to reasoning about the behaviour of the encoding of configurations. Two key lemmas are the following ones; they are derived from Lemmas B.1 and B.2.

Lemma B.3. *If $\mathcal{O}[\![\gamma]\!] \xrightarrow{\mu} P$ and μ is an input action, then we have three possibilities:*

(1) μ is an input $y(x,q)$ and $\gamma = [y \mapsto \lambda x.\,M] \cdot \gamma'$, and $P \ {}_{\pi}\!\gtrsim \mathcal{O}[\![M]\!]\langle q\rangle \mid \mathcal{O}[\![\gamma]\!]$;

(2) μ is an input $y(x,q)$ and $\gamma = [y \mapsto z] \cdot \gamma'$, and $P \ {}_{\pi}\!\gtrsim \mathcal{O}[\![zx]\!]\langle q\rangle \mid \mathcal{O}[\![\gamma]\!]$;

(3) μ is an input $q(x)$ and $\gamma = [q \mapsto (E,p)] \cdot \gamma'$, and $P \ {}_{\pi}\!\gtrsim \mathcal{O}[\![E[x]]\!]\langle p\rangle \mid \mathcal{O}[\![\gamma']\!]$.

Lemma B.4. $\mathcal{O}^*[\![V]\!]\langle y\rangle \xrightarrow{y(x,q)} \approx_{\pi} \mathcal{O}[\![Vx]\!]\langle q\rangle \mid \mathcal{O}^*[\![V]\!]\langle y\rangle$

Encoding of λ-terms:

$$\begin{aligned}
\mathcal{O}[\![xV]\!] &\stackrel{\text{def}}{=} (p)\ \overline{x}(z,q).\,(\mathcal{O}^*[\![V]\!]\langle z\rangle \mid q \triangleright p)\\
\mathcal{O}[\![(\lambda x.\,M)V]\!] &\stackrel{\text{def}}{=} (p)\ \boldsymbol{\nu} y,w\,(\mathcal{O}^*[\![\lambda x.\,M]\!]\langle y\rangle \mid \mathcal{O}^*[\![V]\!]\langle w\rangle \mid \overline{y}(w',r').\,(w' \triangleright w \mid r' \triangleright p))\\
\mathcal{O}[\![VM]\!] &\stackrel{\text{def}}{=} (p)\ \boldsymbol{\nu} y\,(\mathcal{O}^*[\![V]\!]\langle y\rangle \mid \boldsymbol{\nu} r\,(\mathcal{O}[\![M]\!]\langle r\rangle \mid r(w).\,\overline{y}(w',r').\,(w' \triangleright w \mid r' \triangleright p))) \qquad (*)\\
\mathcal{O}[\![MV]\!] &\stackrel{\text{def}}{=} (p)\ \boldsymbol{\nu} q\,(\mathcal{O}[\![M]\!]\langle q\rangle \mid q(y).\,\boldsymbol{\nu} w\,(\mathcal{O}^*[\![V]\!]\langle w\rangle \mid \overline{y}(w',r').\,(w' \triangleright w \mid r' \triangleright p))) \qquad (*)\\
\mathcal{O}[\![MN]\!] &\stackrel{\text{def}}{=} (p)\ \boldsymbol{\nu} q\,(\mathcal{O}[\![M]\!]\langle q\rangle \mid q(y).\,\boldsymbol{\nu} r\,(\mathcal{O}[\![N]\!]\langle r\rangle \mid r(w).\,\overline{y}(w',r').\,(w' \triangleright w \mid r' \triangleright p))) \quad (**)\\
\mathcal{O}[\![V]\!] &\stackrel{\text{def}}{=} (p)\ \overline{p}(y).\,\mathcal{O}^*[\![V]\!]\langle y\rangle
\end{aligned}$$

where in the rules marked $(*)$, M is not a value, and in the rule marked $(**)$ M and N are not values; and where $\mathcal{O}^*[\![V]\!]$ is thus defined:

$$\begin{aligned}
\mathcal{O}^*[\![\lambda x.\,M]\!] &\stackrel{\text{def}}{=} (y)\ !y(x,q).\,\mathcal{O}[\![M]\!]\langle q\rangle\\
\mathcal{O}^*[\![x]\!] &\stackrel{\text{def}}{=} (y)\ y \triangleright x
\end{aligned}$$

Encoding of environments:

$$\mathcal{O}[\![[y \mapsto V]\cdot\gamma']\!] \stackrel{\text{def}}{=} \mathcal{O}^*[\![V]\!]\langle y\rangle \mid \mathcal{O}[\![\gamma']\!]$$

$$\mathcal{O}[\![[q \mapsto (E,p)]\cdot\gamma']\!] \stackrel{\text{def}}{=} q(x).\,\mathcal{O}[\![E[x]]\!]\langle p\rangle \mid \mathcal{O}[\![\gamma']\!]$$

$$\mathcal{O}[\![\varepsilon]\!] \stackrel{\text{def}}{=} 0$$

Encoding of configurations:

$$\mathcal{O}[\![\langle M,p,\gamma\rangle]\!] \stackrel{\text{def}}{=} \mathcal{O}[\![M]\!]\langle p\rangle \mid \mathcal{O}[\![\gamma]\!]$$

$$\mathcal{O}[\![\langle\gamma\rangle]\!] \stackrel{\text{def}}{=} \mathcal{O}[\![\gamma]\!]$$

$$\mathcal{O}[\![\langle M\rangle]\!] \stackrel{\text{def}}{=} \mathcal{O}[\![M]\!]$$

FIGURE 11. The optimised encoding

In Lemma B.4, the occurrence of $\approx_\pi$ cannot be replaced by ${}_\pi\gtrsim$; for instance, if $V = \lambda x.\,M$ then, using Lemma B.2, we can infer that the derivative process is in the relation ${}_\pi\gtrsim$ with $\mathcal{O}[\![M]\!]\langle q\rangle$; however it is not in the same relation with $\mathcal{O}[\![Vx]\!]\langle q\rangle$ (which has extra τ transitions with respect to $\mathcal{O}[\![M]\!]\langle q\rangle$).

Here is another important result that we borrow from [DHS18]. It shows that the only λ-terms whose encoding is bisimilar to $\mathcal{V}[\![x]\!]$ reduce either to x, or to a (possibly infinite) η-expansion of x.

Lemma B.5. *If V is a value and x a variable, $\mathcal{O}^*[\![V]\!] \approx_\pi \mathcal{O}^*[\![x]\!]$ implies that either $V = x$ or $V = \lambda z.\, M$, where the eager normal form of M is of the form $E[xV']$, with $\mathcal{O}^*[\![V']\!] \approx_\pi \mathcal{O}^*[\![z]\!]$ and $\mathcal{O}[\![E[y]]\!] \approx_\pi \mathcal{O}[\![y]\!]$ for any y fresh.*

We can now establish the operational correspondence between OGS and Iπ. (In all the results about operational correspondence, we exploit the convention on freshness of bound names of actions and traces produced by OGS configurations and Iπ processes, as by Remark 2.1.)

Lemma B.6 (strong transitions, from A-OGS to Iπ)**.**

(1) If $F \longrightarrow_{\mathtt{a}} F'$, then $\mathcal{O}[\![F]\!] \longrightarrow_\pi {}_\pi\gtrsim \mathcal{O}[\![F']\!]$;

(2) If $F \xrightarrow{\ell}_{\mathtt{a}} F'$, then $\mathcal{O}[\![F]\!] \xrightarrow{\ell}_\pi \approx_\pi \mathcal{O}[\![F']\!]$.

Proof. The proof goes by a case analysis on the rule used to derive the transition from F, using the definition of the encoding, using Lemmas B.2-B.4. □

Lemma B.7 (strong transitions, from Iπ to A-OGS)**.**

(1) If $\mathcal{O}[\![F]\!] \longrightarrow_\pi P$ then there is F' such that $F \longrightarrow_{\mathtt{a}} F'$ and $P \,{}_\pi\gtrsim \mathcal{O}[\![F']\!]$;

(2) If $\mathcal{O}[\![F]\!] \xrightarrow{\mu}_\pi P$ and μ is an output, then there is F' such that $F \xrightarrow{\mu}_{\mathtt{a}} F'$ and $P \,{}_\pi\gtrsim \mathcal{O}[\![F']\!]$;

(3) If F is passive and $\mathcal{O}[\![F]\!] \xrightarrow{\mu}_\pi P$, then there is F' such that $F \xrightarrow{\mu}_{\mathtt{a}} F'$ and $P \approx_\pi \mathcal{O}[\![F']\!]$.

Proof. The proof goes by a case analysis on the possible transition from $\mathcal{O}[\![F]\!]$, again exploiting Lemmas B.2-B.4. □

We can strengthen both Lemma B.6 and Theorem 6.2 to a correspondence between transitions from A-OGS configurations and from their Iπ translation under the (more restrictive) opLTS. Further, as discussed in Remark 6.10 and Appendix A, we can use the expansion relation and bisimilarity on the opLTS, written $\lesssim_{\mathtt{o}\pi}$ and $\approx_{\mathtt{o}\pi}$. We write I$\pi^{\mathtt{op}}$ to refer to Iπ under the opLTS.

Theorem B.8 (strong transitions, from A-OGS to I$\pi^{\mathtt{op}}$)**.**

(1) If $F \longrightarrow_{\mathtt{a}} F'$, then $\mathcal{O}[\![F]\!] \longrightarrow_{\mathtt{o}\pi} {}_{\mathtt{o}\pi}\gtrsim \mathcal{O}[\![F']\!]$;

(2) If $F \xrightarrow{\ell}_{\mathtt{a}} F'$, then $\mathcal{O}[\![F]\!] \xrightarrow{\ell}_{\mathtt{o}\pi} \approx_{\mathtt{o}\pi} \mathcal{O}[\![F']\!]$.

Theorem B.9 (weak transitions, from A-OGS to I$\pi^{\mathtt{op}}$)**.**

(1) If $F \Longrightarrow_{\mathtt{a}} F'$, then $\mathcal{V}[\![F]\!] \Longrightarrow_{\mathtt{o}\pi} {}_{\mathtt{o}\pi}\gtrsim \mathcal{V}[\![F']\!]$;

(2) If $F \overset{\ell}{\Longrightarrow}_{\mathtt{a}} F'$, then $\mathcal{V}[\![F]\!] \overset{\ell}{\Longrightarrow}_{\mathtt{o}\pi} \approx_{\mathtt{o}\pi} \mathcal{V}[\![F']\!]$.

The following Theorem B.10 is needed in the proof of Theorem 6.7: it relates strong transitions from I$\pi^{\mathtt{op}}$ processes to transitions of A-OGS configurations. Again, in both theorems, the occurrences of $\lesssim_\pi$ and $\approx_\pi$ can be replaced by $\lesssim_{\mathtt{o}\pi}$ and $\approx_{\mathtt{o}\pi}$.

Theorem B.10.

(1) If $\mathcal{O}[\![F]\!] \xrightarrow{\tau}_{\mathtt{o}\pi} P$ then there is F' such that $F \longrightarrow_{\mathtt{a}} F'$ and $P \,{}_\pi\gtrsim \mathcal{O}[\![F']\!]$;

(2) If $\mathcal{O}[\![F]\!] \xrightarrow{\ell}_{\mathtt{o}\pi} P$ then there is F' such that $F \xrightarrow{\ell}_{\mathtt{a}} F'$ and $P \approx_\pi \mathcal{O}[\![F']\!]$.

We report now more details on the proof of Corollary 6.8. First, we recall the assertion:

$$\text{For any trace } s, \text{ we have } F \overset{s}{\Longrightarrow}_{\mathtt{a}} \text{ iff } \mathcal{V}[\![F]\!] \overset{s}{\Longrightarrow}_{\mathtt{o}\pi}.$$

Proof. The result is first established for $\mathcal{O}$, and then extended to $\mathcal{V}$, exploiting the correctness of the optimisations in $\mathcal{O}$ (Lemma 5.2; again, in this lemma, the occurrence of $\lesssim_\pi$ can be lifted to $\lesssim_{\mathsf{o}\pi}$).

For $\mathcal{O}$, both directions are proved proceeding by induction on the length of a trace $\ell_1, \ldots, \ell_n$. In the direction from left to right, we use Theorem B.9(2). For the converse direction, we proceed similarly, this time relying on Theorem 6.7(2) (again, the version with $\approx_{\mathsf{o}\pi}$ in place of $\approx_\pi$). □

Appendix C. Auxiliary results for Section 7.2

We report here the auxiliary results needed to establish the relationship between Concurrent OGS (C-OGS) and Iπ. The results below are used in the proof of Lemma 7.6.

A process P *has deterministic immediate transitions* if, for any μ, whenever $P \xrightarrow{\mu}_\pi P_1$ and $P \xrightarrow{\mu}_\pi P_2$, then $P_1 = P_2$.

Lemma C.1. *Suppose P, Q have deterministic immediate transitions, $P \eqsim_\pi Q$, and $P \xrightarrow{\mu}_\pi P_1$. Then $Q \overset{\mu}{\Longrightarrow}_\pi Q_1$ implies $P_1 \eqsim_\pi Q_1$.*

Lemma below is the analogue to Lemma A.3 for traces.

Lemma C.2.

- *Suppose $P = P_1 \mid !a(\widetilde{b}).\, P_2$, and $Q = Q_1 \mid !a(\widetilde{b}).\, Q_2$, and $P \eqsim_\pi Q$. Then also $P_1 \eqsim_\pi Q_1$.*
- *Similarly, suppose $P = P_1 \mid a(\widetilde{b}).\, P_2$, and $Q = Q_1 \mid a(\widetilde{b}).\, Q_2$, and $P \eqsim_\pi Q$. Then also $P_1 \eqsim_\pi Q_1$.*

We can now conclude the proof of Lemma 7.6. We recall the assertion:

For any M, N we have:
$\mathcal{V}[\![M]\!] \eqsim_\pi \mathcal{V}[\![N]\!]$ iff $\mathcal{V}[\![M]\!] \approx_\pi \mathcal{V}[\![N]\!]$.

Proof. We have to prove that trace equivalence implies bisimilarity. We work with the optimised encoding $\mathcal{O}$. The relation

$$(\mathcal{O}[\![F]\!], \mathcal{O}[\![G]\!]) \;:\; F, G \text{ are singleton with } F \eqsim_{\mathsf{c}} G$$

is a bisimulation up-to context and up-to $({}_\pi\!\gtrsim, \approx_\pi)$.

F and G, as singleton, are of the forms described in Lemma A.2. Moreover, to be in the relation $\eqsim_{\mathsf{c}}$ they must be of the same form, and, using also Lemmas B.2 and B.3 we deduce that they have deterministic immediate transitions.

Suppose F and G are of the form $\mathcal{V}[\![M]\!]\langle p\rangle$ and $\mathcal{V}[\![N]\!]\langle q\rangle$, respectively.

Consider the case $\mathcal{V}[\![M]\!]\langle p\rangle \xrightarrow{\tau} P$; then by Lemma B.2, $P \;{}_\pi\!\gtrsim \mathcal{V}[\![M']\!]\langle p\rangle$, for some M'. Moreover, we have $\mathcal{V}[\![M']\!]\langle p\rangle \eqsim_{\mathsf{c}} \mathcal{V}[\![M]\!]\langle p\rangle \eqsim_{\mathsf{c}} \mathcal{V}[\![N]\!]\langle q\rangle$, and we are done.

The case when $\mathcal{V}[\![M]\!]\langle p\rangle$ performs an output action $\overline{p}(x)$ is simple, as usual using Lemma B.2; we also deduce that $p = q$.

Suppose now $\mathcal{V}[\![M]\!]\langle p\rangle$ performs an output action $\overline{x}(z, p')$. Using Lemma B.2, we have

$$\mathcal{V}[\![M]\!]\langle p\rangle \overset{\overline{x}(z,p')}{\longrightarrow}{}_\pi\!\gtrsim \mathcal{O}^*[\![V]\!]\langle z\rangle \mid p'(y).\, \mathcal{O}[\![E[y]]\!]\langle p\rangle$$

for some V, y, E. Since $\mathcal{V}[\![M]\!]\langle p\rangle \eqsim_{\mathsf{c}} \mathcal{V}[\![N]\!]\langle q\rangle$, and again using Lemma B.2, we deduce

$$\mathcal{V}[\![N]\!]\langle q\rangle \Longrightarrow \overset{\overline{x}(z,p')}{\longrightarrow}{}_\pi\!\gtrsim \mathcal{O}^*[\![W]\!]\langle z\rangle \mid q'(y).\, \mathcal{O}[\![E'[y]]\!]\langle q\rangle$$

By Lemma C.1,

$$\begin{array}{c}\mathcal{O}^*[\![V]\!]\langle z\rangle \mid p'(y).\,\mathcal{O}[\![E[y]]\!]\langle p\rangle \;\eqsim_{\mathsf{c}}\\ \mathcal{O}^*[\![W]\!]\langle z\rangle \mid q'(y).\,\mathcal{O}[\![E'[y]]\!]\langle q\rangle\end{array}$$

Applying Lemma C.2 twice, we deduce

$$\mathcal{O}^*[\![V]\!]\langle z\rangle \eqsim_{\mathsf{c}} \mathcal{O}^*[\![W]\!]\langle z\rangle$$

and

$$q'(y).\,\mathcal{O}[\![E'[y]]\!]\langle q\rangle \eqsim_{\mathsf{c}} q'(y).\,\mathcal{O}[\![E'[y]]\!]\langle q\rangle$$

and we are done, up to expansion and context.
The cases when F and G are of a different form are similar, this time using Lemma B.3. □

Appendix D. Proofs about Section 13

D.1. **Some auxilary results.** For $n > 0$ we define, for Iπ abstractions $A_1 \dots A_n$:

$$\begin{array}{rcl}\mathcal{O}_n\langle r_n, r_0, A_1 \dots A_n\rangle & \stackrel{\text{def}}{=} & \boldsymbol{\nu} r_1, \dots r_{n-1}\\ & & (r_0(v_1).\,\overline{v}_1(x_1, r_1).\mid\\ & & \;r_1(v_2).\,\overline{v}_2(x_2, r_2).\mid\\ & & \;\dots\\ & & \;r_{n-1}(v_n).\,\overline{v}_n(x_n, r_n).\\ & & \;!x_1(q).\,A_1\langle q\rangle \mid\\ & & \;!x_2(q).\,A_2\langle q\rangle \mid\\ & & \;\dots\\ & & \;!x_n(q).\,A_n\langle q\rangle)\end{array}$$

Lemma D.1. *We have:*

$$\begin{array}{rcl}\mathcal{N}[\![xM_1 \dots M_n]\!]\langle r_n\rangle \xrightarrow{\overline{x}(r_0)}_{\pi} & \equiv & \boldsymbol{\nu} r_1, \dots r_{n-1}\\ & & (r_0(v_1).\,\overline{v}_1(x_1, r'_1).\,r'_1 \rhd r_1 \mid\\ & & \;r_1(v_2).\,\overline{v}_2(x_2, r'_2).\,r'_2 \rhd r_2 \mid\\ & & \;\dots\\ & & \;r_{n-1}(v_n).\,\overline{v}_n(x_n, r'_n).\,r'_n \rhd r_n\\ & & \;!x_1(q).\,\mathcal{N}[\![M_1]\!]\langle q\rangle \mid\\ & & \;!x_2(q).\,\mathcal{N}[\![M_2]\!]\langle q\rangle \mid\\ & & \;\dots\\ & & \;!x_n(q).\,\mathcal{N}[\![M_n]\!]\langle q\rangle)\\ & & \\ & \approx_{\pi} & \mathcal{O}_n\langle r_n, r_0, \mathcal{N}[\![M_1]\!] \dots \mathcal{N}[\![M_n]\!]\rangle\end{array}$$

The two lemmas below are known results about encoding of call-by-name λ-calculus into Iπ [San95].

Lemma D.2. *Let M be a λ-term.*

(1) If a, b and c are distinct names of the same type, then $\boldsymbol{\nu} b\,(a \rhd b \mid b \rhd c) \approx_\pi a \rhd c$;
(2) If x and y are distinct trigger names and y is not free in M, then $\boldsymbol{\nu} x\,(x \rhd y \mid \mathcal{N}[\![M]\!]\langle r\rangle) \approx_\pi \mathcal{N}[\![M\{y/x\}]\!]\langle p\rangle$;
(3) If p and r are distinct continuation names, then $\boldsymbol{\nu} r\,(r \rhd p \mid \mathcal{N}[\![M]\!]\langle r\rangle) \approx_\pi \mathcal{N}[\![M]\!]\langle p\rangle$.

Lemma D.3. *If $M \longrightarrow M'$ then $\mathcal{N}[\![M]\!]\langle p\rangle \longrightarrow {}_\pi\!\gtrsim \mathcal{N}[\![M']\!]\langle p\rangle$*

D.2. **Operational correspondence.** We report here some results concerning the operational correspondence between OGS and Iπ terms.

Lemma D.4 (Actions from encodings of Λ^+ terms)**.** *For any $L \in \Lambda^+$ and ϕ with $p, \mathtt{fv}(L) \subseteq \phi$, term $\mathcal{N}[\![L]\!]\langle p\rangle$ has exactly one immediate transition, and exactly one of the following clauses holds:*

(1) $\mathcal{N}[\![L]\!]\langle p\rangle \xrightarrow{\overline{p}(v)} P$ and L is a value (hence of the form either $\lambda x.\, M$ or v), with $P = \mathcal{N}[\![[v \mapsto L]]\!]$;
(2) $\mathcal{N}[\![L]\!]\langle p\rangle \xrightarrow{\overline{x}(q)} P$ and $L = E[x]$, for some E, and

$$P \,{}_\pi\!\gtrsim \mathcal{N}[\![[q \mapsto (E, p)]]\!]$$

;
(3) $\mathcal{N}[\![L]\!]\langle p\rangle \xrightarrow{\overline{v}(x,r)} P$ and $L = E[vM]$, for some E, M and

$$P \equiv \mathcal{N}[\![[r \mapsto (E, p)] \cdot [x \mapsto M]]\!]$$

;
(4) $\mathcal{N}[\![M]\!]\langle p\rangle \xrightarrow{\tau} P$ and there is N with $M \longrightarrow N$ and

$$P \,{}_\pi\!\gtrsim \mathcal{N}[\![N]\!]\langle p\rangle.$$

Lemma D.5 (Actions from Iπ encodings of configurations)**.** *If $\mathcal{N}[\![\langle \gamma, \phi\rangle]\!] \xrightarrow{\mu}_\pi P$, then we have three possibilities:*

(1) μ is an input $v(x, q)$ and $\gamma = [v \mapsto V] \cdot \gamma'$, and

$$P \approx_\pi \mathcal{N}[\![\langle Vx, q, \gamma', \phi \uplus \{q, x\}\rangle]\!];$$

(2) μ is an input $x(q)$ and $\gamma = [x \mapsto M] \cdot \gamma'$, and

$$P = \mathcal{N}[\![\langle M, q, \gamma, \phi \uplus \{q\}\rangle]\!];$$

(3) μ is an input $p(v)$ and $\gamma = [p \mapsto E, q] \cdot \gamma'$, and

$$P = \mathcal{N}[\![\langle E[v], q, \gamma, \phi \uplus \{v\}\rangle]\!].$$

Lemma D.6 (from OGS to Iπ, strong transitions)**.**

(1) (rule (IOQ)) If $F \xrightarrow{(p)} F'$ then $\mathcal{N}[\![F]\!] \xrightarrow{(p)} \mathcal{N}[\![F']\!]$;
(2) (rule (Int)) If $F \longrightarrow F'$ then $\mathcal{N}[\![F]\!] \longrightarrow_\pi\!\gtrsim \mathcal{N}[\![F']\!]$;
(3) (rule (PA)) If $F \xrightarrow{\overline{p}(v)} F'$ then $\mathcal{N}[\![F]\!] \xrightarrow{\overline{p}(v)} \equiv \mathcal{N}[\![F']\!]$;
(4) (rule (PVQ)) If $F \xrightarrow{\overline{v}(y,q)} F'$ then $\mathcal{N}[\![F]\!] \xrightarrow{\overline{v}(y,q)} \equiv \mathcal{N}[\![F']\!]$;
(5) (rule (PTQ)) If $F \xrightarrow{\overline{x}(q)} F'$ then $\mathcal{N}[\![F]\!] \xrightarrow{\overline{x}(q)}_\pi\!\gtrsim \mathcal{N}[\![F']\!]$;

(6) (rule (OA)) If $F \xrightarrow{q(v)} F'$ *then* $\mathcal{N}[\![F]\!] \xrightarrow{q(v)} \mathcal{N}[\![F']\!]$;
(7) (rule (OVQ)) If $F \xrightarrow{v(y,q)} F'$ *then* $\mathcal{N}[\![F]\!] \xrightarrow{v(y,q)} \approx_\pi \mathcal{N}[\![F']\!]$;
(8) (rule (OTQ)) If $F \xrightarrow{x(q)} F'$ *then* $\mathcal{N}[\![F]\!] \xrightarrow{x(q)} \mathcal{N}[\![F']\!]$.

Theorem D.7 (from OGS to Iπ, traces)**.** *If F has the trace s then also $\mathcal{N}[\![F]\!]$ has the trace s.*

Proof. Straightforward induction on the length of the trace. □

Now the converse direction.

Lemma D.8 (from Iπ to OGS, strong transitions)**.**

(1) If $\mathcal{N}[\![F]\!] \xrightarrow{(p)} P$, *then there is* F' *such that* $F \xrightarrow{(p)} F'$ *and* $P = \mathcal{N}[\![F']\!]$;
(2) If $\mathcal{N}[\![F]\!] \longrightarrow P$, *then then there is* F' *such that* $F \longrightarrow F'$ *and* $P \; {}_\pi\!\gtrsim \mathcal{N}[\![F']\!]$;
(3) If $\mathcal{N}[\![F]\!] \xrightarrow{\mu} P$, *where μ is an output, then then there is* F' *such that* $F \xrightarrow{\mu} F'$ *and* $P \; {}_\pi\!\gtrsim \mathcal{N}[\![F']\!]$;
(4) If $\mathcal{N}[\![F]\!] \xrightarrow{\mu} P$, *where μ is an input, and $\mathcal{N}[\![F]\!]$ cannot perform an output or a τ, then there is* F' *such that* $F \xrightarrow{\mu} F'$ *and* $P \approx_\pi \mathcal{N}[\![F']\!]$.

In the modified transition system $\xrightarrow{\mu'}_{\mathsf{o}\pi}$ for Iπ, where output and silent transitions have priority over input transitions, we can then derive from the above results the full abtractions on traces, reported in Corollary 13.4.

D.3. **Concurrent Operational Game Semantics.** We have reported in the main text the operational correspondence result for strong transitions. We can thus derive the operational correspondence result for weak transitions. For silent transition, we proceed by induction on the number of strong transitions involved.

Theorem D.9 (from C-OGS to Iπ, weak transitions)**.**

(1) If $F \Longrightarrow_{\mathsf{c}} F'$ *then* $\mathcal{N}[\![F]\!] \Longrightarrow_\pi {}_\pi\!\gtrsim \mathcal{N}[\![F']\!]$;
(2) If $F \overset{\ell}{\Longrightarrow}_{\mathsf{c}} F'$ *then* $\mathcal{N}[\![F]\!] \overset{\ell}{\Longrightarrow}_\pi \approx_\pi \mathcal{N}[\![F']\!]$.

Theorem D.10 (from Iπ to C-OGS, weak transitions)**.**

(1) If $\mathcal{N}[\![F]\!] \Longrightarrow_\pi P$, *then there is* F' *s.t.* $F \Longrightarrow_{\mathsf{c}} F'$ *and* $P \; {}_\pi\!\gtrsim \mathcal{N}[\![F']\!]$;
(2) If $\mathcal{N}[\![F]\!] \overset{\ell}{\Longrightarrow}_\pi P$, *then there is* F' *s.t.* $F \overset{\ell}{\Longrightarrow}_{\mathsf{c}} F'$ *and* $P \approx_\pi \mathcal{N}[\![F']\!]$.

Corollary D.11. *F and $\mathcal{N}[\![F]\!]$ have the same set of traces.*

Proof. By induction on the length of a trace performed by F or $\mathcal{N}[\![F]\!]$: thus one shows that, for a trace t,

- If $F \overset{t}{\Longrightarrow}_{\mathsf{c}} F'$ then $\mathcal{N}[\![F]\!] \overset{t}{\Longrightarrow}_\pi \approx_\pi \mathcal{N}[\![F']\!]$;
- If $\mathcal{N}[\![F]\!] \overset{t}{\Longrightarrow}_\pi P$, then there is F' s.t. $F \overset{t}{\Longrightarrow}_{\mathsf{c}} F'$ and $P \approx_\pi \mathcal{N}[\![F']\!]$. □

Corollary D.12. *We have:* $F_1 \eqsim_{\mathsf{c}} F_2$ *iff* $\mathcal{N}[\![F_1]\!] \eqsim_\pi \mathcal{N}[\![F_2]\!]$.

Corollary D.13. *we have:* $F_1 \approx_{\mathsf{c}} F_2$ *iff* $\mathcal{N}[\![F_1]\!] \approx_\pi \mathcal{N}[\![F_2]\!]$.

A configuration F *is a singleton* if its domain has only one element.

Lemma D.14. *For any singleton F, G, we have: $\mathcal{N}[\![F]\!] \eqsim_\pi \mathcal{N}[\![G]\!]$ iff $\mathcal{N}[\![F]\!] \approx_\pi \mathcal{N}[\![G]\!]$.*

Proof. Weak bisimilarity implies trace equivalence, hence the direction from right to left.

For the converse, we show that the relation

$$(\mathcal{N}[\![F]\!], \mathcal{N}[\![G]\!]) \ : \quad F, H \text{ are singleton with } F \eqsim_{\mathsf{c}} G$$

is a bisimulation up-to contexts and expansion. The proof uses Lemmas C.1 and A.3. □

We think that the above lemma can be generalised to aribitrary configurations; the current statement is however sufficient for our purposes. By combining Lemma D.14 and Corollaries D.12 and D.13 we then derive Theorem 13.8.
The following result holds for call-by-name but not for call-by-value.

Lemma D.15. *Suppose γ and δ (resp. γ' and δ') have the same domains. Then $\langle \gamma \cdot \gamma', \phi \rangle \eqsim_{\mathsf{c}} \langle \delta \cdot \delta', \phi \rangle$ implies $\langle \gamma, \phi \rangle \eqsim_{\mathsf{c}} \langle \delta, \phi \rangle$*

Proof. This is a consequence of the fact that any action from the components γ', δ' has, as subject, a name in the domain of these components. Such a name does not appear in γ, δ, and thus it will not appear in any action of their traces. □